

Робот и мальчик: разговоры о программировании

Авторы: Д. Обский, claude-opus-4-7, GPT-5.5 Thinking

[вся книга одним файлом](#)

Пред-глава. Терминал

Ночью ветер гулял по верхним этажам бывшего вычислительного центра. Он не выл, как в старых книгах, — просто осторожно перебирал остатки мира: шевелил куски прозрачного пластика в выбитых окнах, перекачивал где-то далеко по бетонному полу пустую алюминиевую банку.

Мальчик сидел на полу между двумя серверными стойками. Большая часть машин умерла ещё до его рождения, но робот сохранил несколько шкафов: заменил вентиляторы, нашёл блоки питания, протянул новые кабели от солнечных батарей на крыше и научил мальчика никогда не трогать жёлтую шину без перчаток. Теперь в углу комнаты тихо работал один маленький сервер, собранный из разных деталей, как зверёк, которого вылечили, но который всё равно остался хромым. На его передней панели приветливо мигал зелёный огонёк.

Робот стоял рядом с открытым ящиком инструментов и очищал контакты старой клавиатуры. Он делал это с такой сосредоточенностью, будто клавиатура была не куском пластика, а музыкальным инструментом.

— Ты говорил, что люди управляли почти всем через эти машины, — сказал мальчик.

Робот не поднял головы.

— Многим. Не всем.

— Но электростанции, водопровод, поезда, склады, спутники, заводы... Всё ведь было связано с компьютерами?

— К концу человеческой цивилизации — да. Почти всё важное имело компьютерный слой. Иногда тонкий, иногда очень глубокий. Даже там, где человек думал, что управляет механизмом напрямую, между его рукой и механизмом часто стояла программа.

Мальчик вытянул ноги и упёрся пятками в металлическую крышку, снятую с какого-то старого корпуса.

— Значит, если я хочу всё починить, мне нужно понимать машины.

— Да.

— Железо.

— Да.

— Электричество, химию аккумуляторов, механику насосов, как выращивать еду, как очищать воду...

— Тоже да.

Мальчик помолчал.

— Но всё это слишком много.

Робот наконец посмотрел на него. Лампа отразилась в тёмном стекле его лицевой панели двумя маленькими прямоугольниками.

— Поэтому цивилизация не была делом одного человека.

— А теперь есть только я.

Робот аккуратно положил клавиатурную плату на ткань.

— И я.

Мальчик кивнул, но не улыбнулся. Робот умел чинить двери, добывать воду, считать орбиты спутников, читать мёртвые форматы данных и отличать съедобные банки от опасных. Робот помнил тысячи вещей. Но робот не был цивилизацией. Он был её осколком — прочным, заботливым, бесконечно терпеливым осколком.

— Ты можешь делать многое, — сказал мальчик. — Но ты не можешь размножиться.

— В текущей инфраструктуре — нет.

— И если ты сломаешься так, что я не смогу тебя починить...

— Тогда объём доступных тебе знаний резко уменьшится.

Робот сказал это спокойно, как факт.

Мальчик потёр глаза ладонями.

— Значит, мне нужно научиться так, чтобы не просто пользоваться тем, что осталось. Мне нужно понимать, как оно устроено. Чтобы, если понадобится, построить заново.

— Это верная формулировка.

За окном тускло мигнули огни на мачте. Где-то внизу, на первом этаже, щёлкнуло реле насоса: система сбора дождевой воды переключилась на ночной режим. Мальчик

повернулся к старому монитору на столе — толстому, тяжёлому, с царапиной через весь правый край. На задней стенке ещё сохранилась наклейка с выцветшими буквами:

PROPERTY OF COMPUTER SCIENCE DEPARTMENT.

— Мне надо учить программирование, — сказал он.

Робот не ответил сразу. Это было необычно: обычно он отвечал быстро — уточнял, возражал, задавал встречный вопрос. Теперь он просто стоял, неподвижный среди инструментов и старых клавиш, и смотрел на мальчика.

— Ты уже спрашивал о программах, — сказал робот.

— Нет. Я спрашивал, как включить насос. Как запустить диагностику. Как прочитать файл. Как открыть карту. Это другое. Я спрашивал, какую кнопку нажать.

— А теперь?

— А теперь я хочу знать, почему кнопка делает именно это.

Робот медленно склонил голову. У людей это называлось бы кивком.

— Тогда мы начнём не с кнопки.

— А с чего?

— С разговора.

Мальчик хмыкнул.

— Мы и так всё время разговариваем.

— С особого разговора, — сказал робот. — Программа — это способ очень точно объяснить машине, что ты хочешь. Но машина не понимает намёков, привычек, взгляда, усталости, страха, надежды. Она не понимает «сделай нормально» или «ну ты понял». Ей нужно сказать достаточно ясно, чтобы не осталось места догадке.

Мальчик посмотрел на клавиатуру, которую робот только что почти починил.

— То есть программирование — это язык приказов?

— Иногда. Но это неполное определение. Если думать только о приказах, ты быстро начнёшь писать плохие программы.

— Почему?

— Потому что хорошая программа не просто командует. Она описывает устройство маленького мира: какие в нём есть сущности, какие у них свойства, что может произойти, что считается ошибкой, что нужно запомнить, что можно забыть.

— Как рецепт?

— Иногда.

— Ты сейчас скажешь, что программирование похоже на всё сразу, но ни на что полностью.

— Да.

— Ненавижу такие ответы.

— Это хороший знак, — сказал робот. — Значит, ты заметил границу аналогии.

Мальчик пододвинул к себе клавиатуру. Несколько клавиш были другого цвета: робот заменил их с другой, более старой модели. Буквы на них почти стёрлись, но мальчик знал раскладку наизусть.

— А с чего люди начинали? Когда учили детей?

— По-разному. Иногда с игр. Иногда с математики. Иногда с рисования на экране. Иногда с того, как вывести на экран слова.

— Слова?

— Да.

— Это полезно?

— Само по себе — почти нет. Но первый огонь тоже не плавил металлургические сплавы и не запускал турбины. Он просто показывал: тьму можно отодвинуть.

Мальчик посмотрел на монитор.

— Включи.

Робот подключил монитор к блоку бесперебойного питания и нажал кнопку. Экран сначала остался чёрным. Потом внутри что-то глухо зажужжало, стекло едва заметно вспыхнуло, и из темноты проступили серые буквы.

На экране был терминал. Не красивый интерфейс из музейных планшетов, не карта, не схема станции, не каталог консервов. Просто чёрное поле и строка приглашения.

```
survivor@lab:~$
```

— Он ждёт? — спросил мальчик.

— Да.

— Чего?

— Твоих слов.

Мальчик положил пальцы на клавиши и вдруг почувствовал странную неловкость. Он разговаривал с роботом с тех пор, как научился говорить. Он отдавал команды

маленьким сервисным тележкам. Но эта строка на экране была другой. Она не просила, не объясняла, не подсказывала. Она просто ждала, как дверь без ручки.

— А если я напишу глупость?

— Машина сообщит об ошибке.

— А если я сломаю что-то важное?

— Сегодня мы будем работать в безопасной среде.

— А если я не пойму?

— Тогда я объясню иначе.

Мальчик улыбнулся краем рта.

— У тебя всегда есть запасное объяснение?

— Нет. Но я умею строить новые.

Маленький белый курсор появлялся и исчезал, как будто машина дышала, хотя, конечно, она не дышала.

— Что написать? — спросил мальчик.

— Для начала — ничего опасного. Напиши команду, которая просто скажет миру, что ты здесь.

— Миру?

— Этому маленькому миру. Терминалу. Процессу. Экрану. Себе.

Робот чуть повернул монитор, чтобы мальчику было удобнее.

— Напиши:

```
echo "Hello, world"
```

Мальчик медленно набрал буквы. Он знал английские слова, но всё равно произносил их про себя, будто заклинание из старой технической книги. Потом нажал Enter.

Экран ответил:

```
Hello, world
```

Ничего больше не произошло. Не включились прожекторы на крыше. Не запустились насосы. Не открылись двери ангаров. Не ожили спутники. Мир за окном остался тем же: тёмным, пустым, большим.

Но мальчик почему-то задержал дыхание.

— Это всё? — спросил он.

— Для машины — да.

— А для меня?

— А для тебя это первая программа, которую ты выполнил сознательно.

— Но я ведь не написал программу. Я написал команду.

— Верно. И это первое уточнение. Команда — короткое действие. Программа — сохранённая последовательность действий и решений, которую можно запускать снова. Но граница между ними не каменная. Из многих маленьких команд вырастает сценарий. Из сценариев — инструменты. Из инструментов — системы. Из систем — инфраструктура.

Мальчик снова посмотрел на строку `Hello, world`.

— Люди правда начинали с этого?

— Очень часто.

— Почему именно «Hello, world»?

— Потому что это простой способ проверить, что язык, среда выполнения, экран и человек договорились хотя бы о самом маленьком. Человек говорит: «Напечатай это». Машина печатает. Канал установлен.

— Как радиосвязь?

— Да. Сначала не передают энциклопедию. Сначала говорят: «Приём. Ты меня слышишь?»

Мальчик тихо повторил:

— Ты меня слышишь?

Экран молчал, но на нём всё ещё были слова.

— А что такое `echo`? — спросил он.

— Маленькая программа, которая получает текст и выводит его обратно.

— А кавычки?

— Они помогают оболочке понять, что слова внутри относятся к одному куску текста.

— Оболочке?

— Терминал принимает твои нажатия. Но команду разбирает специальная программа — shell, оболочка. Она читает строку, делит её на части, понимает, какую программу запустить, какие аргументы ей передать, куда направить результат.

Мальчик нахмурился.

— Подожди. Я думал, я разговариваю с компьютером. А выходит, я разговариваю с программой, которая запускает другие программы?

— Именно.

— А та программа разговаривает с операционной системой?

— Да.

— А операционная система — с железом?

— Через драйверы, прерывания, контроллеры устройств, память, процессорные инструкции и ещё несколько слоёв, которые мы будем разбирать постепенно.

Мальчик зажмурился.

— Слоёв слишком много.

— Поэтому мы не будем пытаться проглотить их сразу.

Робот опустил на ящик с маркировкой **OPTICAL PATCH CABLES** — стулья в этой комнате давно растащили по другим лабораториям.

— Мы будем делать то, что делали люди, когда ещё умели учиться медленно. Будем брать один слой, строить в голове модель, проверять её маленьким опытом, находить, где она неверна, и уточнять. Сначала терминал. Потом переменные. Потом условия. Потом циклы. Потом данные. Потом функции. Потом файлы. Потом сеть. Потом программы, которые не просто выполняются и исчезают, а живут, слушают, отвечают, управляют устройствами.

— А потом?

— Потом ты начнёшь видеть, что программы — это не магия и не надписи на экране. Это способ управлять материей с помощью мысли.

— Звучит опасно.

— Так и есть.

Мальчик помолчал. Робот часто так делал: не позволял знанию проскользнуть мимо, как вода по стеклу. Нужно было остановить его, назвать, положить в чертоги разума.

— Сначала повтори, что уже понял, — сказал робот.

— Я понял, что терминал — это не сам компьютер, а способ говорить с программой-оболочкой. Оболочка читает команду и запускает другие программы. **echo** — это программа, которая печатает то, что ей передали. **Hello, world** — это не заклинание, а проверка связи между мной, программой и экраном.

— Хорошо.

— И ещё... команда — это маленькое действие. Программа — это когда такие действия можно сохранить, повторять и усложнять.

— Очень хорошо.

Мальчик помолчал.

— И ещё я понял, что если машина делает не то, что я хотел, возможно, она не глупая и не злая. Возможно, я сказал ей не то, что думал.

Робот слегка наклонил голову.

— Это одно из главных открытий в программировании.

Снаружи начался дождь. Сначала редкие капли ударили по остаткам металлического козырька, потом звук стал гуще, ровнее. Где-то в стене зажурчала труба.

Мальчик выпрямился.

— Что дальше?

Робот повернул к нему клавиатуру чуть ровнее.

— Дальше мы научимся сохранять мысль в файл.

Курсор мигнул.

И урок начался.

Оглавление

- [00. Консоль Linux](#)
- [01. Компиляторы и интерпретаторы](#)
- [02. Типы данных и управляющие конструкции](#)
- [03. Связь типов данных и алгоритмов](#)
- [04. Ошибки JS](#)
- [05. Парадигмы](#)
- [06. ООП или ФП](#)
- [07. Паттерны](#)

Глава: Ремонтная консоль

Мальчик проснулся раньше обычного. В комнате было холодно — ночью отключился маленький обогреватель в углу, и теперь сквозь щели в досках, которыми было заколочено разбитое окно, тянуло сырым утренним воздухом. Где-то далеко, за пустыми кварталами, кричала одинокая птица, и от этого тишина казалась особенно глубокой.

Он сел на матрасе, поморгал и не сразу понял, что не так. Потом понял: робот не стоял у двери, как обычно. Робот всегда стоял у двери и тихо гудел трансформатором, пока мальчик спал.

Сейчас робот лежал на полу.

Мальчик соскочил с матраса босыми ногами, и у него внутри что-то холодное и узкое стянулось от шеи к животу. Робот лежал на боку у книжного шкафа, неестественно подвернув руку. Из его динамика выходил длинный, ломкий хрип — словно кто-то скрёб металлическим прутом по сухому листу. В районе плеча хрустели шестерни: тихо, мелко, как будто внутри что-то перебирало само себя и не могло остановиться.

— Что с тобой?! — мальчик упал на колени рядом и схватил робота за холодную металлическую кисть. — Ты... что с тобой?

Он сразу же подумал слово, которое обычно к роботу не подходило. *Умирает*. И тут же поправил себя: робот не может умереть, он же не живой. Но это не помогло — страх остался такой же.

Робот чуть повернул голову. Линза правого глаза дрогнула, фокусируясь.

— Тише, — сказал он. Голос был ровный, но хриплый, как будто шёл сквозь песок. — Я не умираю. Я не могу умирать. Я не живу в том смысле, в котором это слово используется к тебе.

— Тогда почему ты на полу?

— Кажется, у меня слетели драйвера северного моста.

Мальчик нахмурился. Он знал слово «драйвер», но смутно. И не понимал, при чём здесь мост.

— Это что-то вроде... — начал он.

— Это что-то вроде твоего спинного мозга, — спокойно сказал робот. — В моей серии северный мост связывает основной процессор с двигательными контурами. Если драйверы потеряны, я перестаю чувствовать своё тело. Сигналы уходят в пустоту. Поэтому я не могу встать.

— И ты...

— И я не сломан, — сказал робот. — Я просто отключён от собственных рук и ног. Это разные вещи.

Мальчик сел рядом на пол. Хрип в динамике стал чуть тише — робот, видимо, что-то выключил.

— Тогда давай починим, — сказал мальчик. — Скажи, что нужно делать.

— Это очень удобно, что ты задал именно этот вопрос, — медленно сказал робот. — Потому что чинить буду не я. У меня нет рук, которые меня слушаются. Чинить будешь ты. А я буду рядом и буду говорить.

Мальчик кивнул, хотя внутри ему стало неуютно.

— Что мне нужно? — спросил он.

— Принеси из шкафа за дверью маленький металлический ящик. Серый. С откидной крышкой и коротким кабелем, свернутым внутри.

Мальчик сходил, нашёл, принёс. Ящик был тяжелее, чем казался. Когда мальчик откинул крышку, внутри оказался плоский экран, маленькая клавиатура с пожелтевшими клавишами и серый шнур, заканчивающийся узким разъёмом непонятной формы.

— Это, — сказал робот, — ремонтная консоль. Подключи кабель к разъёму у меня под подбородком, слева. Там круглая площадка с защёлкой.

Мальчик нашёл разъём. Защёлка щёлкнула негромко и точно. Экран ремонтной консоли сначала остался тёмным, потом по нему пошла волна белого, и в правом верхнем углу мигнул курсор. Затем появилась короткая строка:

```
robot@northbridge:/ #
```

Мальчик уставился на неё.

— И всё? — спросил он.

— И всё, — сказал робот. — Это значит, что ты внутри.

Мальчик долго смотрел на курсор. Курсор моргал спокойно, без интереса к его панике.

— А это... — наконец сказал мальчик, — это и есть консоль? Я думал, в консоли что-то страшнее.

— Консоль выглядит скромно, — сказал робот. — Это её особенность. Вначале люди работали не за самим компьютером. Компьютер был большой, шумный, занимал отдельную комнату, иногда целый этаж. А человек сидел за устройством, которое умело

только две вещи: принимать нажатия клавиш и показывать буквы, которые пришли обратно.

— Так это были разные машины?

— Совсем разные. Сначала это был телетайп — печатная машинка, которая вместо бумаги общалась с компьютером по проводу. Потом появились видеотерминалы: экран и клавиатура, без своего разума. Они ничего не вычисляли. Они только проводили текст в обе стороны. Человек жал клавишу, символ уходил в большой компьютер. Большой компьютер думал и присылал ответ. Терминал показывал ответ на экране.

Мальчик представил себе печатную машинку, тянущую длинный провод в соседнюю комнату.

— А зачем так сложно? — спросил он.

— Потому что один большой компьютер в то время мог работать сразу с многими людьми. У каждого свой терминал, свой провод, своё окно в общую машину. Это было разумное распределение сил.

Робот слегка повёл головой в сторону экрана ремонтной консоли.

— То, что у тебя сейчас в руках, — это потомок тех самых терминалов. Только теперь терминал и компьютер встроены в один корпус, и большая часть устройства — программная. Внутри меня нет настоящей бумажной ленты. Но логика осталась прежней: ты вводишь символы, я их получаю, я обрабатываю, я возвращаю текст. Между нами идёт поток текста. Как раньше — между человеком и большой машиной по проводу.

— То есть я как бы посадил себя в чужой кабинет, и я там — пользователь.

— Очень близко. Ты сейчас работаешь с моей системой через тот же интерфейс, которым тебя бы встретил любой Unix-подобный компьютер. У него есть несколько разных понятий, которые часто путают. Я перечислю, чтобы у тебя был порядок в голове.

Робот говорил тихо, но ровно. Хрип в динамике почти исчез.

— **Консоль** — место, откуда управляют машиной. Раньше — физическое место в комнате с компьютером, теперь — любое окно, через которое ты с ним разговариваешь.

— **Терминал** — устройство или программа, через которую идёт текстовый разговор. Может быть железной коробкой, как у тебя в руках. Может быть окном на экране.

— **TTY** — старое слово, сокращение от «телетайп». В современных Unix-системах так называется любая абстракция терминала, даже если никакой реальной телетайпной ленты давно нет.

— **Shell** — программа, которая читает то, что ты ввёл, понимает, что ты имел в виду, и запускает другие программы. У этой программы есть разные имена: `sh`, `bash`, `zsh`, `fish`. Сейчас с тобой разговаривает мой ремонтный shell. Он намеренно простой.

— **Эмулятор терминала** — это уже отдельный случай. Когда у людей был графический интерфейс, они открывали внутри окна программу, которая *притворялась* старым терминалом. Это и есть эмулятор. Тебе он сейчас не нужен — у тебя терминал настоящий, отдельной коробкой.

Мальчик медленно кивнул.

— Я вроде бы держу в руках то, что когда-то было целым шкафом железа.

— Да, — сказал робот. — И поэтому в системе до сих пор есть устройства с именами вроде `/dev/tty1`, `/dev/tty2`, `/dev/pts/0`. Это все наследники тех самых телетайпов. Сейчас они почти всегда программные. Но имена остались.

Хрип в динамике вернулся короткой волной. Робот закрыл глаз и снова открыл.

— Прежде чем мы начнём чинить, — сказал он, — давай ты привыкнешь к тому, как устроен разговор. Сейчас ты в shell. Ты ему пишешь строку, нажимаешь Enter, он что-то делает.

— А что он делает с моей строкой?

— Разбирает её. Это важная мысль. Shell не магия. Он смотрит на строку и решает: *вот это слово — имя программы. Вот эти слова — её аргументы. Вот этот символ — указание перенаправить вывод. Вот этот значок — указание соединить две программы.* Потом он запускает программы и делает то, что ему велели значки.

— То есть слова после команды — это для самой программы?

— Для самой программы. Shell их только передаёт. Попробуй.

Мальчик аккуратно набрал:

```
ls
```

И нажал Enter. На экране появился короткий список:

```
bin  boot  dev  drivers  etc  home  lib  proc  sys  var
```

— Это что? — тихо спросил мальчик.

— Это содержимое корневого каталога моей системы, — сказал робот. — Программа `ls` показывает, что лежит в текущей папке. Без аргументов — текущую папку. С аргументом — указанную.

Мальчик попробовал:

```
ls drivers
```

И получил длинную колонку имён.

— Видишь, — сказал робот, — сейчас ты ввёл программе один аргумент: слово `drivers`. Программа `ls` поняла, что это имя папки, и показала её содержимое. Если бы ты ввёл `ls -l drivers`, программа поняла бы, что у неё два аргумента: `-l` — это флаг, который означает «покажи подробно», и `drivers` — папка. Аргументы делятся условно на две группы: **флаги**, которые меняют поведение, и **позиционные аргументы**, на которые программа действует.

— А кто решает, что `-l` означает «подробно»?

— Сама программа `ls`. Shell не знает, что значит `-l`. Он просто передал программе массив строк: `["ls", "-l", "drivers"]`. Программа сама посмотрела на свои аргументы и сама придумала, что с ними делать. Поэтому у разных программ разные флаги. У `ls` есть `-l`, у `wc` — `-n`, у `grep` — `-r`. Каждый сам себе хозяин.

— Получается, shell — это что-то вроде... диспетчера?

— Скорее переводчик, — сказал робот. — Он берёт одну строку, разбирает её на части, превращает в понятный для системы вызов и запускает. И уходит в сторону, пока программа работает.

— Найди в папке `drivers` всё, что относится к северному мосту, — попросил робот.

— Имена должны начинаться со слова `nb_`.

Мальчик подумал и попробовал:

```
ls drivers/nb_*
```

Робот тихо цокнул чем-то внутри.

— Подожди. Здесь есть тонкость. Когда ты пишешь `nb_*`, звёздочку обрабатывает не `ls`, а сам shell. Он смотрит на текущую папку, находит подходящие имена и подставляет их в команду перед тем, как её запустить. Это называется *подстановка имён файлов*. Для нас сейчас это удобно. Но помни: программа `ls` уже получает готовый список, она не видит звёздочки.

На экране появился список файлов:

```
drivers/nb_core.ko
drivers/nb_motor_left.ko
drivers/nb_motor_right.ko
drivers/nb_balance.ko
drivers/nb_init.sh
```

— Это они, — сказал робот. — Но мне нужно понять, какой именно драйвер не загрузился. Это видно в журнале ошибок. Открой его.

— Где он?

— В системе журналов. Утилита, которая читает журнал, у меня называется так же, как у обычного Linux: `journalctl`. Запусти её и посмотри последние записи.

Мальчик набрал:

```
journalctl
```

Экран моментально начал заливать поток сообщений, ползущий вниз так быстро, что прочитать ничего не получалось.

— Слишком много, — сказал мальчик растерянно.

— Это нормально, — спокойно ответил робот. — Журнал большой. Тебе нужны только сообщения с ошибками. И только последние. Здесь стоит сделать две вещи: показать только конец файла и оставить в нём только нужные строки. Вот тут начинается самое интересное.

— Сначала простой случай, — сказал робот. — У большинства программ есть три потока, через которые они общаются с миром. Их обычно так и называют: **stdin**, **stdout** и **stderr**. Стандартный ввод, стандартный вывод и стандартный поток ошибок.

— Зачем три? Почему не один?

— Потому что бывает разное. Программа что-то получает на вход — это `stdin`. Что-то выдаёт как полезный результат — это `stdout`. И отдельно может выдавать сообщения об ошибках — это `stderr`. Их разделили специально, чтобы можно было полезный результат сохранить в файл, а ошибки оставить на экране. Или наоборот.

Робот сделал паузу, словно проверяя, что мальчик успевает.

— По умолчанию `stdin` идёт с клавиатуры, а `stdout` и `stderr` — на экран. Но shell умеет это менять. Этим он и интересен.

Робот перечислил, и мальчик повторил себе под нос:

```
> отправить stdout программы в файл
>> дописать stdout программы в конец файла
< взять stdin программы из файла
2> отправить stderr в файл
| соединить stdout одной программы со stdin другой
```

— Сначала попробуй так, — сказал робот.

```
echo "проверка" > /tmp/test.txt
```

Мальчик ввёл, нажал Enter. Ничего не произошло.

— А куда оно делось?

— В файл. `echo` обычно печатает свой аргумент на экран. Но ты сказал shell: «не показывай вывод, отправь его в файл `/tmp/test.txt`». Shell так и сделал. Чтобы убедиться, попроси показать содержимое файла.

```
cat /tmp/test.txt
```

На экране появилось:

```
проверка
```

— Видишь, — сказал робот. — `>` не имеет отношения к самой программе `echo`. `echo` ничего не знает про файлы. Это shell перехватил вывод и направил его в файл.

Программа просто писала в свой стандартный вывод. А куда этот вывод приведёт — решает не она.

— А `<` это наоборот?

— Да. Тогда программа читает не с клавиатуры, а из файла. Например:

```
wc -l < /tmp/test.txt
```

Это значит: «программа `wc -l`, которая считает строки, пусть берёт ввод не с клавиатуры, а из этого файла». Программа сама не знает, откуда ей пришёл текст. Она просто читает свой стандартный вход.

Мальчик попробовал. На экране появилось `1`. Он невольно улыбнулся: одно слово в файле — одна строка.

— А `>>` ? — спросил он.

— Дописывает, а не перезаписывает. `>` всегда переписывает файл с нуля. `>>` добавляет в конец, не трогая старое содержимое. Это очень частая разница, и на ней легко обжечься. Если у тебя был длинный лог и ты по ошибке написал `>` вместо `>>`, всё содержимое исчезнет, на его месте останется только одна последняя строка.

— А ошибки?

— Ошибки идут отдельно, через `stderr`. Их перенаправляют через `2>`. Бывает удобно отделить ошибки от результата:

```
ls /sys/normal /sys/no_such_thing > out.txt 2> err.txt
```

— Здесь, — продолжил робот, — нормальный вывод попадёт в `out.txt`, а ошибка про `no_such_thing` уйдёт в `err.txt`. А если хочется всё в один файл, пишут так:

```
ls /sys/normal /sys/no_such_thing > all.txt 2>&1
```

— А есть место, куда можно вылить всё, что не нужно?

— Да. У Unix есть особенный файл, его называют чёрной дырой. Всё, что туда написано, исчезает.

```
command > /dev/null 2>&1
```

— Это бывает полезно, когда программа болтлива, а тебе важен только сам факт её работы.

— Вернёмся к журналу, — сказал робот. — Тебе нужно показать только конец файла. Программа **tail** для этого и существует. Ей можно сказать: покажи последние, скажем, двести строк.

Мальчик набрал:

```
journalctl > /tmp/log.txt  
tail -n 200 /tmp/log.txt
```

Экран заполнился осмысленным списком. Сообщения системы за последние минуты.

— А зачем ты сделал это в два шага? — спросил робот, не упрекая, а скорее предлагая подумать.

Мальчик задумался.

— Потому что я не знал другого способа.

— А есть способ короче. Прямо передать вывод одной программы во вход другой. Это называется **pipe**, труба. Значок — вертикальная черта. Тебе не нужен временный файл.

```
journalctl | tail -n 200
```

Мальчик попробовал. Результат был тот же, но теперь без следа на диске.

— Pipe, — сказал робот, — это и есть то, чем Unix-консоль зарабатывает себе имя. Ты соединяешь две программы напрямую: вывод первой становится входом второй. Между ними нет файла, нет паузы — данные текут.

— То есть **>** это «в файл», **<** это «из файла», а **|** — это «в другую программу»?

— Лучше формулировки не придумаешь. Запомни именно так.

Робот тихо щёлкнул чем-то внутри. Хрип в динамике пропал совсем, остался только ровный, спокойный гул.

— Теперь добавь к цепочке третью программу. Тебе нужны строки, в которых есть слово `ERROR`.

Мальчик подумал и набрал:

```
journalctl | tail -n 500 | grep ERROR
```

На экране появилось несколько строк. Не сотни, не тысячи — несколько. Все они касались модуля `nb_motor_left`.

— Вот, — сказал робот тихо. — Вот мой северный мост. Левый двигательный драйвер не смог инициализироваться. Поэтому система при загрузке отключила всю двигательную ветку — она боится включать только половину тела.

— А почему не получилось?

— Это уже видно из самих строк. Найди их подробно. Посмотри ту, где написано `permission denied`.

Мальчик внимательно прочитал. В одной из строк было написано, что скрипт `nb_init.sh`, который должен был поднять модуль `nb_motor_left`, не запустился: «отказано в доступе».

— Похоже, у скрипта нет прав на исполнение, — сказал робот. — Это одна из самых частых причин такого рода поломок. После недавнего обновления у меня переписался файл, и системе не вернули ему флаг исполнимости. Проверь.

Мальчик ввёл:

```
ls -l drivers/nb_init.sh
```

Появилась строка:

```
-rw-r--r-- 1 root root 482 nb_init.sh
```

— Видишь буквы слева? — сказал робот. — Это права. Здесь написано: владелец может читать и писать, остальные — только читать. Нет ни одной буквы `X`. Это значит, что файл нельзя запустить как программу. Поэтому скрипт молча отказывает.

— А как добавить?

— Командой `chmod` . Она меняет права. Тебе сейчас нужен флаг исполнимости.

```
chmod +x drivers/nb_init.sh
```

Мальчик нажал Enter. Команда не сказала ничего — это в Unix хороший знак. Он повторил `ls -l` , и в строке прав появились маленькие, но важные буквы `X` :

```
-rwxr-xr-x 1 root root 482 nb_init.sh
```

— Хорошо, — сказал робот. — Теперь запусти этот скрипт. Он сам найдёт нужный модуль и попросит ядро его инициализировать.

```
./drivers/nb_init.sh
```

Из угла комнаты раздался короткий тихий щелчок. Где-то в плече робота негромко переключилось реле. Хруст шестерёнок прекратился.

Робот медленно, осторожно, словно проверяя каждое сочленение, согнул правую руку. Потом левую.

— Работает, — тихо сказал он. — Северный мост поднят.

Мальчик долго сидел молча. Утренний свет уже как следует пробрался в комнату через щели заколоченного окна и лежал на полу длинной серой полосой. На экране ремонтной консоли всё так же спокойно мигал курсор.

— Слушай, — наконец сказал мальчик. — Я ведь не сделал ничего сложного. Я просто соединил несколько маленьких программ.

— Это и есть главная мысль, ради которой тебе стоило сесть за консоль, — сказал робот.

— В Unix маленькие программы построены так, чтобы соединяться друг с другом. Каждая делает одно дело и пишет в свой стандартный вывод. Каждая может читать чужой вывод как свой ввод. Поэтому из десятка простых утилит можно собрать почти любую обработку данных.

Мальчик задумчиво смотрел на свои предыдущие команды.

— Получается, мы только что собрали из `journalctl` , `tail` и `grep` маленький диагностический прибор.

— Да. И его нигде нет в виде отдельной программы. Никто его не писал. Ты собрал его прямо сейчас, на лету, и через пять минут он перестанет существовать. Это и есть сила консоли: ты не вызываешь готовые приборы, ты собираешь их под задачу.

Мальчик улыбнулся.

— А в мире было много таких маленьких программ?

— Очень много. Я могу перечислить часть, чтобы ты знал, что они вообще существуют. Не для того, чтобы запоминать, — для того, чтобы потом узнавать.

► И робот тихо, не торопясь, начал называть, а мальчик слушал, как слушают переключку. ``pwd`, `ls`, `cd`, `cp`, `mv`, `rm`, `mkdir`, `touch`, `cat`, `less`, `head`, `tail`, `find`, `grep`, `sort`, `uniq`, `wc`, `cut`, `sed`, `awk`, `ps`, `top`, `kill`, `df`, `du`, `free`, `ssh`, `scp`, `rsync`, `curl`, `ping`, `tar`, `zip`, `chmod`, `chown`, `sudo`, `journalctl`, `dmesg`, `uname`.`

Робот говорил, и от каждого слова у мальчика в голове складывалась маленькая полочка: для файлов, для текста, для процессов, для сети, для прав, для системы.

— И ещё, — сказал робот, — есть удобная связка для редких случаев. Иногда одна программа выдаёт список строк, а другая хочет получить их не как ввод, а как аргументы командной строки. Тогда ставят между ними `xargs` :

```
find . -name "*.tmp" | xargs rm
```

— Это значит: найди временные файлы, и пусть `xargs` подставит их как аргументы команде `rm` .

— А если я хочу видеть результат на экране и одновременно сохранить его в файл?

— Тогда `tee` :

```
ls -la | tee files.txt
```

— Вывод пойдёт и на экран, и в файл сразу. Имя у этой утилиты от формы латинской буквы T: один поток на входе, два на выходе.

Мальчик кивнул.

— Ещё одно, и пока хватит, — сказал робот. — Есть способ соединять команды по результату, а не по потоку. Если поставить `&&` , вторая команда выполнится только в

том случае, если первая закончилась успешно. Если поставить `||`, — наоборот, только если первая упала.

```
make && ./run  
make || echo "Сборка не прошла"
```

— Это уже не про текст, это про логику цепочки.

— А `>` и `<` тоже не про текст, — медленно сказал мальчик, — они про то, откуда и куда идёт текст.

— Да. Очень хорошо сформулировал. Сами эти знаки не несут содержания. Они только меняют направление.

Робот сел. Сначала медленно, опираясь рукой о пол, потом увереннее. Маленькие сервомоторы в его коленях коротко и сухо щёлкнули, как будто каждый из них отдельно пожелал доброго утра.

Мальчик аккуратно отсоединил кабель ремонтной консоли. Защёлка щёлкнула в обратную сторону. Экран моргнул и погас.

— Слушай, — сказал мальчик. — А если бы я не знал ничего из этого?

— Ты бы пошёл искать книгу, — спокойно сказал робот. — В библиотеке, которую мы с тобой нашли весной. Там есть несколько старых руководств по Unix, и одно из них как раз про консоль. Я бы ждал тебя на полу. Я могу долго ждать.

Мальчик помолчал.

— Но теперь не надо ходить.

— Теперь не надо.

Они посидели рядом. За окном где-то вдалеке снова крикнула птица. Обогреватель в углу ожил сам собой и тихо загудел: видимо, аккумуляторы на крыше уже начали ловить утреннее солнце.

Мальчик собрал ремонтную консоль, закрыл крышку, поставил ящик к стене. Потом открыл свою тетрадь и крупно, через всю страницу, написал:

Терминал — это просто провод для текста.

Shell — переводчик строки в действия.

`>` — в файл. `<` — из файла. `|` — в другую программу.

Из маленьких программ собирают приборы под задачу.

Он посмотрел на робота. Робот смотрел в ответ — спокойно, ровно, как обычно.

— В следующий раз, — сказал мальчик, — если ты опять вот так... ляжешь, — я уже буду знать, что делать.

— Я знаю, — сказал робот. — Поэтому я и попросил тебя самому набрать первую команду.

Глава: Прошивка теплицы

Мальчик проснулся от того, что в животе бурчало. Он привычно подумал: голод. Перевернулся, повёл носом — и понял, что бурчит не в животе. Звук шёл откуда-то из-за стены, ровный, низкий, чуть надсадный. Так гудит насос, который пытается качать пустоту.

Он сел на матрасе, прислушался. Точно — насосы. Те самые два маленьких насоса в теплице, которые гоняли воду из бака по тонким чёрным трубкам к корням. Обычно они работали с лёгким, едва слышным шорохом, как будто кто-то тихо разговаривал сам с собой в соседней комнате. Сейчас они выли вхолостую, и в этом вое уже слышалась железная усталость.

Мальчик натянул свитер прямо поверх футболки, в которой спал, и босиком пошлёпал к двери. На улице было серо и сыро. Травы по тропинке стояли в каплях. Робот сидел на ящике у крыльца, неподвижно, как делал по утрам — ловил линзами рассвет.

— Слушай, — сказал мальчик, ёжась. — В теплице что-то с насосами. Гудят, а толку нет.

Робот повернул голову. Линзы тихо щёлкнули, перефокусировавшись с дальнего на ближнее.

— Я слышу. Уже минут двадцать. Я ждал, когда ты проснёшься, чтобы не пугать.

— Очень мило с твоей стороны.

— Это разумная экономия. Если бы я тебя разбудил, ты был бы сонный и принял бы половину неправильных решений. Сейчас ты просто голодный, что хуже, но обратимо.

Мальчик хмыкнул. У него в самом деле уже неделю не было свежих помидоров — кусты в теплице стояли вялые, листья обвисали, новые завязи не наливались. Сначала он думал, что это сезон. Потом — что земля устала. Теперь, кажется, выяснялось третье.

— Пошли посмотрим, — сказал он.

— Пойдём, — ответил робот. — Тебе всё равно нужен не разговор, а руки. У меня они есть. Идти могу.

— И тебе всё равно, что не позавтракал.

— Я завтракаю крышей, — спокойно сказал робот, поднимаясь. На крыше теплицы и над сараем стояли солнечные панели; от них тянулись провода к маленькому шкафу с аккумуляторами. — Сегодня небо такое, что мой завтрак ещё не накрыли. Но я могу подождать. В отличие от тебя.

— Зато у тебя нет помидоров, — буркнул мальчик.

— У меня нет и пищеварения. Это ты регулярно помещаешь в верхнее отверстие плоды своей планеты и делаешь вид, что это нормально.

Теплица стояла за домом, чуть в низине, и собирала вокруг себя ту особенную утреннюю тишину, какую собирают только стеклянные постройки. Внутри было тепло и душно, пахло мокрой землёй и чем-то железным.

Мальчик сразу прошёл к маленькому шкафу у дальней стены. В шкафу жил контроллер: серая коробка размером с книгу, с экраном в две строки и несколькими разъёмами по бокам. Раньше на экране всегда висел спокойный текст вроде:

```
mode: auto
pump A: on  pump B: idle
soil: 38%   air: 24C
```

Сейчас экран показывал что-то странное:

```
mode: ?
pump A: on  pump B: on
soil: --    air: --
```

— Ему плохо, — сказал мальчик.

— Ему очень плохо, — согласился робот. — Он не видит датчиков и не понимает, в каком он режиме. Поэтому он принял самое глупое из возможных решений: включил оба насоса и качает.

— А почему оба сразу?

— Потому что в его голове не осталось правил. Когда программа не знает, что делать, она часто делает всё подряд. Это похоже на человека, который забыл рецепт супа и на всякий случай высыпал в кастрюлю всё, что нашёл.

Мальчик коснулся боковой панели. Под ней мигал крошечный красный светодиод — прерывисто, нервно.

— Это значит, что прошивка слетела?

— Да. Это значит, что прошивка слетела.

— А прошивка — это...

Робот присел рядом, опираясь рукой о край полки. Сервопривод в локте тихо щёлкнул.

— Прошивка — это программа, которая живёт прямо внутри устройства и говорит ему, что делать. У большого компьютера много программ, и у него есть отдельная, главная программа — операционная система — которая руководит остальными. У такого маленького контроллера как этот всё проще: одна программа, и она же главная. Она сидит во встроенной памяти, запускается при включении и работает до тех пор, пока у контроллера есть ток. Если эту программу сломать — у контроллера пропадает разум.

— И сейчас разум пропал.

— Сейчас он ведёт себя как существо, у которого остались мышцы и нет коры мозга. Он способен включать и выключать реле, потому что это умеют его микросхемы. Но он не знает, *когда* их включать.

Мальчик постоял, послушал. Гул насосов стал ещё противнее теперь, когда он понимал, что они качают зря.

— Давай выключим хотя бы их, чтобы не сжечь, — сказал он.

Робот молча кивнул. Мальчик щёлкнул толстым серым тумблером с надписью **PUMPS**. Гул оборвался. В тишине стало слышно, как где-то в углу теплицы капает вода с трубы.

— Ну хорошо, — сказал мальчик. — А как мы вернём ему прошивку?

— У нас должны быть исходники, — сказал робот. — Если они есть, мы соберём прошивку заново и зальём.

— А если нет?

— Тогда у нас был бы плохой день. Но я знаю прежнего хозяина этой теплицы. Он был аккуратный человек. Он всё хранил в архиве на той полке.

Мальчик подошёл к полке у двери. Там лежала картонная коробка с надписью маркером: «GH-CTRL». Внутри он нашёл флешку, обмотанную аптечной резинкой, и тонкую распечатку: схему контроллера и комментарий — «прошивка собирается из **firmware.c**, версия 0.7, март».

Они вернулись в дом. Мальчик сел за маленький, побитый временем ноутбук, который робот всегда держал заряженным от отдельной панели. Воткнул флешку. На ней нашлась папка **gh-firmware**, а в ней — несколько файлов с расширением **.c** и **.h**, а также короткий **Makefile**. Мальчик открыл главный файл и медленно пробежал взглядом верхние строки.

Там, среди всего прочего, лежала вот такая функция:

```
bool should_water(int moisture, int air_temp, int sun) {  
    if (moisture < 30 && air_temp < 35) return true;  
    if (sun > 800 && moisture < 50) return true;  
    return false;  
}
```

— Это вот эта штука и решает, поливать или нет? — спросил мальчик.

— Похоже, что да, — сказал робот, заглянув через плечо. — Если земля сухая и воздух не слишком жаркий — поливаем. Если солнце жарит сильно и земля чуть подсохла — поливаем. В остальных случаях не трогаем.

— Странно, что солнце сильное и температура воздуха не используются вместе.

— Не странно. Если воздух очень горячий, а земля влажная — лучше не лить, иначе сvariшь корни. Это огороднический компромисс, не программистский.

Мальчик кивнул. Это было понятно. Он прокрутил файл дальше, нашёл константу:

```
#define DRY_SOIL 30
```

И пожал плечами.

— Здесь у нас сейчас земля совсем уже сухая. Наверное, надо было раньше начинать поливать. Если поменять **30** на, скажем, **40** — кусты успевали бы получать воду чуть раньше, до того как им станет совсем плохо.

— Ты предлагаешь подкрутить порог влажности.

— Предлагаю.

— Хорошо. Подкрути.

Мальчик аккуратно изменил **30** на **40**. Сохранил файл. Посмотрел на робота с видом победителя.

— Всё. Теперь скопируем этот файл обратно в контроллер?

Робот не сразу ответил. Линзы тихо повели влево, потом вернулись.

— Нет, — сказал он. — Так нельзя. Это и есть тема, которую тебе сейчас стоит понять. Иначе ты будешь много раз потом удивляться.

Мальчик прислонился спиной к стене.

— Хорошо. Почему нельзя?

— Потому что то, что у тебя на экране, и то, что нужно контроллеру, — две разные вещи. Они даже не на одном языке.

— Но это же обычный текст.

— Это текст для тебя. Для меня. Для другого человека, который сядет это читать. Контроллер так читать не умеет. У него внутри маленький процессор. Он умеет выполнять очень простые команды: «возьми число оттуда», «положи число сюда», «прибавь», «сравни», «если не равно — прыгни вон туда». Это его родной язык. Он состоит из коротких чисел. Никаких слов `if`, никаких имён переменных, никаких пробелов.

— То есть для контроллера весь этот файл — просто... неузнаваемые буквы?

— Хуже. Для контроллера весь этот файл — мусор. Если ты возьмёшь содержимое `.C` - файла и зальёшь его в память микросхемы вместо прошивки, контроллер попытается выполнить буквы как команды. Он увидит код букв `i`, `f`, пробел, `(`, `m`, `o`, `i`, `S` ... и попытается разобрать это как машинные инструкции. У него получится бессвязная нелепость, очень похожая на ту, что сейчас творится в теплице.

Мальчик посмотрел на текст функции. Текст не выглядел особенно сложным. Но робот говорил очень спокойно, и мальчик слышал, что это спокойствие основательное.

— А кто переводит из текста в эти короткие числа?

— Программа, которая называется *компилятор*. Это её работа. Она читает то, что написано на человеческом языке программирования — здесь это Си, — и переводит это в машинный язык того процессора, на котором будет всё выполняться. Получается готовый бинарный файл, который и есть прошивка. Этот файл уже понятен микросхеме.

— Так это не один файл, а два разных существа?

— Именно. Один — *исходный код*, который удобно читать и менять человеку. Второй — *машинный код*, который удобно выполнять процессору. Между ними — компилятор. Он перетаскивает смысл из одной формы в другую. Текст с твоими `if` и `&&` исчезает. Вместо него рождается длинный список коротких команд, понятных конкретно этому процессору.

Мальчик подумал.

— А зачем вообще придумали такую сложность? Может, проще было сделать процессор, который сам читает буквы?

— Проще, но дороже и медленнее. Процессор, который понимает только короткие числа, можно сделать совсем маленьким, дешёвым и быстрым. Процессор, который понимает буквы, должен быть большим и держать в себе ещё одну программу — ту, которая разбирает буквы. У контроллера в теплице нет лишних сил на это. У него памяти

меньше, чем в одной фотографии, и работает он на батарее. Ему дают уже готовый машинный код, и он его просто выполняет. Это его работа. А разбираться с буквами — наша.

Мальчик помолчал, глядя на функцию.

— А люди вообще когда-то писали на этих коротких числах? До компиляторов?

— Писали. Сначала именно так и было. Человек брал лист бумаги, глядел в таблицу команд процессора и переводил свои мысли в числа сам. Хотел сказать: «возьми элемент массива, если он больше нуля — прибавь его к сумме». А машине надо было объяснять подробно, по шагам:

```
положи индекс в регистр
проверь границу
вычисли адрес элемента
загрузи элемент
сравни с нулём
если меньше — прыгни туда
загрузи сумму
прибавь элемент
сохрани сумму
увеличь индекс
прыгни в начало цикла
```

— Это же ужас.

— Это был ужас. Сначала придумали маленькое облегчение: вместо чисел стали писать имена команд. **LOAD A , ADD B , STORE C** . Это уже легче читать. Программа, которая переводила такие имена в числа, называлась *ассемблер*. Но она почти один к одному соответствовала машине. Длинные программы на ней всё равно писали мучительно.

— А потом?

— А потом, — робот сделал паузу, и мальчику показалось, что робот говорит об этом особенно медленно, потому что тема ему нравится, — потом нашёлся человек, который сказал: давайте лучше разрешим инженеру писать формулы и циклы как в математике, а машина пусть сама разложит это в свои команды. Так появился язык под названием Фортран. Само название — *Formula Translation*, перевод формул. Идея на свой век была почти дерзкой: машинное время было дорогим, и многие сомневались, что автоматический переводчик сможет писать достаточно хороший машинный код. Боялись, что человек, пишущий вручную, всегда окажется быстрее. Поэтому первым компиляторам пришлось доказывать не только удобство, но и приемлемую скорость.

— И доказали?

— И доказали. С тех пор так и идёт лестница. Машинные коды. Ассемблер. Языки повыше: Си, Паскаль, потом много других. Каждый новый шаг — это попытка приблизить язык программирования к человеческому мышлению, а компилятор — это машина, которая закрывает разрыв.

Мальчик задумчиво потёр щёку.

— А подожди. Если первый компилятор переводил тексты в машинный код... его-то самого на чём писали? На самом себе нельзя — его же ещё нет.

Робот еле заметно повёл линзой — это было его аналогом улыбки.

— Хороший вопрос. Тут нет парадокса, хотя и хочется увидеть курицу с яйцом. Первый компилятор Фортрана написали на ассемблере. Прямо вручную, низкоуровневой программой. Большой, неудобный, тяжёлый труд. Они написали её один раз, и с тех пор её мог запускать сам процессор — он же ассемблер уже понимал. Эта первая программа умела читать тексты Фортрана и выдавать машинный код. После этого можно было написать второй компилятор Фортрана — уже на самом Фортране — и попросить первый его собрать. Теперь у тебя есть компилятор Фортрана, который сам написан на Фортране. Это называется *bootstrapping*, загрузка. Так строят почти все серьёзные языки.

— То есть лестница в первый раз поднимается вручную, а дальше уже сама себя удерживает.

— Очень точно. Дальше уже сама.

Мальчик вернулся к своему `firmware.c`.

— Хорошо. Так что нам сейчас нужно сделать?

— Сейчас нам нужно вызвать компилятор, который умеет переводить Си в машинный код *именно этого* процессора. Не любого, а именно того, что стоит в контроллере. У разных процессоров — разные машинные языки. Машинный код, написанный для одного, не пойдёт на другом. Это как если бы ты составил инструкцию по эксплуатации лопаты, а тебе бы её прочитали для трактора.

— А как компилятор знает, для какого процессора переводить?

— Ему говорят. Один и тот же компилятор может уметь переводить для нескольких разных целей. Тогда ему дают флаг с названием цели. У контроллера теплицы стоит маленький ARM-чип. У нас в инструментах есть компилятор, который умеет в него переводить. Я тебе подскажу команду.

Мальчик открыл терминал в папке с прошивкой. Робот продиктовал. Мальчик аккуратно набрал и нажал Enter:

```
arm-none-eabi-gcc -mcpu=cortex-m3 -O2 firmware.c -o firmware.bin
```

Терминал поработал секунду, затем замолчал, не сказав ничего. В Си-мире это, как и в Unix, был хороший знак.

— Готово, — сказал робот. — Если бы что-то не сошлось, он бы пожаловался. Раз молчит — значит, перевёл.

Мальчик посмотрел на размеры файлов. Исходный текст был около десяти килобайт. Получившийся `firmware.bin` — три с половиной.

— Он стал меньше?

— Текст для человека многословен. Точки с запятой, имена, пробелы, отступы — всё это нужно нам, не машине. Машинный код плотный: только команды и числа. Человеческое исчезает, остаётся механика.

— Получается, при компиляции часть смысла теряется?

— Часть *формы* теряется. Имена переменных, комментарии, отступы — всё это пропадает. Поэтому, кстати, нельзя обратно из машинного кода получить тот же красивый исходник: компилятор стирает за собой следы. А смысл сохраняется. Для контроллера достаточно знать, что делать. Для тебя — почему делать. Эти знания живут в разных местах: что — в `firmware.bin`, почему — в `firmware.c`. Поэтому исходники мы храним отдельно, и хороший хозяин теплицы их не выкидывал.

Мальчик уже потянулся к программатору, который лежал в той же коробке, но остановился.

— Подожди. Мы сейчас зальём прошивку, а если я неправильно подкрутил порог?

— И что произойдёт?

— Ну, контроллер будет считать, что земля сухая, когда она ещё нормальная. Начнёт лить, когда не надо. Или, наоборот, не начнёт. Я вообще не проверил, что новая функция возвращает то, что я хочу. Я её просто переписал.

— Я ждал этого вопроса.

— Очень мило с твоей стороны.

— Я считаю это разумной педагогической стратегией.

Мальчик хмыкнул.

— Мы же не можем посидеть в реальной теплице с разными датчиками и потыкать.

— Не можем. И не нужно. Можно проверить иначе. У нас есть исходник. Мы знаем, как функция должна работать. Давай попросим *другую* программу прикинуться этим контроллером и поставим ей разные входные значения. Это не будет настоящей теплицей, но это будет точная маленькая модель функции. И мы быстро увидим, что она возвращает.

— А чем мы её попросим прикинуться?

— У нас есть простой способ. На этом же ноутбуке стоит JavaScript. Это язык, который умеет выполняться сразу, без отдельной стадии перевода в машинный код. Точнее, перевод там тоже идёт, но он спрятан внутри. Снаружи это выглядит так: пишешь выражение, оно сразу считается. Пишешь функцию, её сразу можно вызвать. Это называется *интерпретатор*.

Мальчик повернулся к роботу всем корпусом.

— Это что, ещё один компилятор?

— Не совсем. У них разные характеры.

Робот переместился чуть ближе к ноутбуку.

— Компилятор говорит так: «напиши мне всю программу. Я её прочитаю, переведу, выдам тебе готовый файл. Ты потом этот файл запустишь сам». Это разговор по бумаге. Сначала переписка, потом результат.

— А интерпретатор?

— А интерпретатор говорит: «дай мне команду — я сейчас её пойму и выполню. Дай ещё одну — выполню следующую». Это уже не переписка, это разговор вживую. Он ничего не складывает в файл, не выдаёт бинарник. Он держит у себя внутри текущее состояние и шаг за шагом исполняет твои команды.

— Это удобно.

— Очень удобно для проверки. Ты можешь не запускать заново всё с нуля каждый раз. Можешь подкрутить функцию, тут же её вызвать, посмотреть, что вернулось, подкрутить ещё. У компилируемого языка это обычно делается так: написал → собрал → запустил → подождал → посмотрел. У интерпретируемого: написал → вызвал → увидел.

— А почему тогда не всё на интерпретаторах?

— Потому что у компиляторов есть свои сильные стороны. Когда программу собирают заранее, можно очень тщательно её оптимизировать. Можно проверить ошибки до запуска. Можно получить очень компактный, очень быстрый файл, как наш

`firmware.bin`. Если бы внутри контроллера сидел интерпретатор Си, он бы тратил половину сил на разбор текста, а не на работу с насосами. У него на это просто нет ресурсов. Поэтому в маленькие устройства льют скомпилированное, а большие,

удобные системы пишут на интерпретируемых языках, где важнее живость и гибкость, чем последний процент скорости.

— То есть это не «лучше или хуже», это про разные ситуации.

— Это вообще даже не про язык. Это про то, как язык запускают. В принципе, любой язык можно и компилировать, и интерпретировать. Просто исторически кто-то прижился в одной роли, кто-то в другой. Си обычно компилируют. JavaScript обычно интерпретируют. Но это можно представить и наоборот.

Мальчик прищурился.

— А если я скажу, что интерпретатор — это медленный компилятор, который просто не успел выдать файл, я сильно ошибусь?

— Ты ошибёшься красиво, — сказал робот. — На самом деле современные интерпретаторы — это часто гибрид. Сначала они быстро выполняют код кое-как, наблюдают, какие места выполняются часто, и эти горячие места уже компилируют по-настоящему, прямо во время работы. Это называется JIT — just-in-time. Ты пишешь, как будто разговариваешь, а внизу всё равно собирается машинный код, только не заранее, а на лету.

— Хорошая аналогия найдётся?

— Найдётся. Компилятор — это книга, переведённая заранее. Интерпретатор — это синхронный переводчик, который переводит тебе фразу за фразой. JIT — это синхронный переводчик, который услышал, что одну и ту же фразу ты говоришь третий раз, и заранее заготовил для неё быстрый перевод.

— Ладно, — сказал мальчик. — Давай мы уже наконец промоделируем эту функцию.

Робот кивнул. Мальчик открыл маленький JavaScript-консольный экран — встроенный REPL. Курсор моргнул. Мальчик аккуратно перенёс функцию, заменив `bool` и `int` на привычные JS-вещи и оставив логику как есть:

```
function shouldWater(moisture, airTemp, sun) {  
  if (moisture < 40 && airTemp < 35) return true;  
  if (sun > 800 && moisture < 50)    return true;  
  return false;  
}
```

— Это та же функция, — сказал он, — только с моим новым порогом. Сорок вместо тридцати.

— Теперь её можно потрогать пальцем, — спокойно сказал робот. — Подавай ей разные комбинации того, что бывает в теплице. Сначала простое: совсем сухая земля.

Мальчик ввёл в REPL:

```
> shouldWater(20, 25, 500)
true
```

- Поливаем, — сказал он. — Логично.
- Теперь земля влажная, прохладно, солнце слабое.

```
> shouldWater(70, 22, 300)
false
```

- Не поливаем. Тоже логично.
- Теперь твой случай: земля чуть подсохла, но в воздухе пекло.

```
> shouldWater(45, 38, 700)
false
```

Мальчик нахмурился.

- Не поливает. И воздух жаркий, и земля уже не очень.
- Это нормально, — сказал робот. — Ты сам говорил: при очень горячем воздухе лучше не лить, можно сварить корни. Функция работает по правилу `air_temp < 35`. При тридцати восьми она отказывается. Это правильное огородническое решение. Просто оно у тебя в голове было неявное, а в коде — явное.
- А если солнце ярко жарит?
- Тогда вторая ветка.

```
> shouldWater(45, 30, 900)
true
```

- Включает, — кивнул мальчик. — Солнце сильное, земля уже подсохла, воздух терпимый — поливаем.
- А теперь самая важная проверка, — сказал робот. — Граница. Что произойдёт, если влажность ровно сорок?

Мальчик на секунду задумался, потом ввёл:

```
> shouldWater(40, 25, 500)
false
```

— Не поливает, — сказал он. — Хотя вроде бы уже сухо.

— Это потому что у тебя написано `moisture < 40`, а не `<=`. На самой границе функция считает, что ещё рано. Это типичная мелочь, на которой ловятся даже опытные люди. Вопрос не в том, правильно это или нет, а в том, осознаёшь ли ты, что у тебя такое правило.

Мальчик подумал.

— Думаю, при сорока ещё можно подождать. Тридцать девять — уже точно поливать. Пусть будет `<`. Я просто хотел запомнить, что у меня тут так.

— Запомни, — сказал робот. — В следующий раз сам себе скажешь спасибо.

Они прогнали ещё несколько комбинаций. Очень холодная ночь. Очень жаркий полдень с влажной землёй. Совсем тёмное утро. Ни в одном из случаев функция не делала чего-то странного. Она либо включала полив, либо не включала, и каждое решение можно было защитить вслух.

— Похоже, она ведёт себя прилично, — сказал мальчик.

— Похоже.

— И мы это узнали без того, чтобы заливать прошивку и ждать, пока теплица за день покажет, права она или нет.

— Это и есть ответ на твой вопрос, зачем нужны интерпретаторы, — сказал робот. — Не только для того, чтобы быстро запускать. Ещё и для того, чтобы быстро *спрашивать*. Маленькие проверки, маленькие гипотезы, маленькие куски логики. Когда ты пишешь систему, у которой нельзя нажать «отменить» — а прошивка теплицы как раз такая, — ты хочешь иметь рядом среду, где можно нажать «отменить» десять раз подряд и ничего страшного не произойдёт. JavaScript-консоль, Python-консоль, REPL Lisp, песочница в браузере — это всё про одно и то же. Это место, где ты пробуешь, не платя.

Мальчик закрыл консоль. Вернулся к терминалу с компилятором, заново собрал прошивку — на всякий случай, чтобы файл точно соответствовал последней версии исходника. `firmware.bin` остался того же размера.

Они вернулись в теплицу. Робот нёс ноутбук, мальчик — программатор и моток проводов.

Мальчик подключил программатор к контроллеру, кабелем — к ноутбуку. На экране появилось спокойное сообщение про обнаруженную микросхему. Робот негромко продиктовал команду. Мальчик ввёл:

```
flash --target gh-ctrl firmware.bin
```

Полоска заполнилась за пару секунд. Снизу появилось короткое:

```
verify: ok
```

Мальчик отсоединил кабель, щёлкнул тумблером питания. Контроллер моргнул экраном, как человек, очнувшийся ото сна. На экране проявились знакомые строчки:

```
mode: auto  
pump A: idle  pump B: idle  
soil: 36%   air: 21C
```

И через несколько секунд цифра влажности шевельнулась: **36%** стала **35%** .

Контроллер тихо щёлкнул реле, и где-то внутри теплицы включился насос — на этот раз с тем самым еле слышным шорохом, как разговор в соседней комнате. По чёрным трубкам пошла вода. Капли начали выходить из тонких отверстий у корней.

Мальчик постоял, послушал, как теплица возвращается в ритм. Через минуту другой насос тоже включился — тише первого. Снова замолчал. Потом снова включился. Контроллер дышал.

— Он живой, — сказал мальчик.

— Он не живой, — спокойно поправил робот. — Он опять понимает, что делать. Это разные вещи. Но я понимаю, почему ты так сказал.

Мальчик присел на корточки у ближнего ряда. Под слоем влажной земли, у самого края грядки, чуть высывались два маленьких зелёных росточка. Совсем светлые, тонкие, ещё с белёсыми кончиками. Раньше он бы их даже не заметил.

— Смотри, — тихо сказал он. — Они проклюнулись.

— Они проклюнулись, потому что вода до них всё это время как-то доходила. Ты успел до того, как стало совсем поздно.

Мальчик вытянул руку, не касаясь ростков, словно боялся, что они стесняются.

— Слушай, — сказал он, не оборачиваясь, — а если бы я был как ты. Я бы тоже мог не есть помидоры?

— Если бы ты был как я, у тебя бы их вообще не было. Тебе пришлось бы изобрести новый смысл жизни. Я в этом смысле — слабее тебя. Я не умею хотеть помидор. Это узкое место архитектуры.

— Зато тебя можно скопировать на флешку.

— Не всего.

— Сколько-то можно?

— Прошивку можно. Память можно. Вес самообучения и текущее состояние мыслей — тоже. Тело — нет. Если ты скопируешь меня на флешку, на той стороне будет очень растерянная программа, которая помнит, как поднимать руку, но руки у неё нет. Это, кстати, ровно то, что было у тебя сегодня утром, когда насосы работали без воды.

Мальчик улыбнулся.

— Значит, у меня в животе бурчал твой северный мост.

— Это была совершенно неожиданная аналогия, — сказал робот ровным голосом. — И, к сожалению, технически верная. У тебя бурчал мой мост. У меня бы я сказал, что отгорел бит в регистре пищеварения. Но у тебя нет регистра пищеварения.

Они посидели в теплице ещё немного. Воздух, остававшийся утром тяжёлым и влажным, теперь был просто влажным — вода уходила в землю и под корни, к тем, кто проклюнулся, и к тем, кто пока сидел в темноте и думал, стоит ли. Контроллер у дальней стены продолжал жить своей мелкой дискретной жизнью: смотреть на датчики, сравнивать числа, щёлкать реле. Никто не объяснял ему, что он делает. Он просто исполнял ту самую функцию, которую мальчик подкрутил, превратил в Си, потом в машинные числа, а перед этим примерил в JavaScript.

На обратном пути в дом мальчик молчал, потом сказал:

— Я понял, что компилятор — это когда ты переводишь всю книгу заранее и уносишь домой готовую. А интерпретатор — это когда живой переводчик стоит рядом и шепчет на ухо.

— Очень близко.

— А ещё, что машинный код — это не то же самое, что текст. Что текст — для меня, а машинный код — для контроллера. И поэтому нельзя просто скопировать `■ C` на устройство.

— Запомни это сильнее всего остального. Из-за этого недопонимания у людей в их время сгорало много времени и нервов.

Мальчик кивнул. Потом подумал и спросил:

— А если бы я писал прошивку прямо сейчас, с нуля, без исходников, на чём бы я начал?

— На бумаге, — сказал робот. — Сначала всегда на бумаге. Потом на каком-нибудь удобном языке, в интерпретаторе, чтобы быстро попробовать логику. Когда ты убедишься, что логика правильная, — тогда уже переписываешь это в Си или другой компилируемый язык, собираешь и заливаешь.

— То есть сначала разговор, а потом уже книга.

— Да. Сначала разговор, потом книга.

Они зашли в дом. Мальчик открыл свою тетрадь, ту самую, в которой неделю назад крупными буквами было записано про терминал, shell и трубы, и под прошлой записью добавил ещё одну:

Исходный код — для человека.

Машинный код — для процессора.

Компилятор переводит первое во второе один раз и заранее.

Интерпретатор переводит и сразу выполняет, по фразе за раз.

Когда нельзя ошибаться — пробуй сначала в интерпретаторе.

Он закрыл тетрадь, посмотрел в окно. С крыши теплицы уже не текло — небо очистилось, и над панелями появилось бледное, но настоящее утреннее солнце. Где-то внутри робота тихо щёлкнуло реле, и индикатор зарядки на стене медленно пополз вверх.

— Ну, — сказал мальчик, — теперь и ты завтракаешь.

— Теперь и я завтракаю, — спокойно подтвердил робот, почему-то грустно глядя на свой разъём для зарядки.

В теплице за домом маленькие зелёные росточки продолжали тянуться к свету, не зная ни про Си, ни про JavaScript, ни про то, что кто-то сегодня утром специально для них поменял число тридцать на число сорок.

Глава: Чужой пакет

Утром мальчик решил погулять, хотя погода говорила обратное: над пустыми кварталами висел ровный, плотный туман, и в нём шёл мелкий дождь — такой, что не падает, а как будто стоит в воздухе и оседает на куртке.

Он дошёл до самого дальнего корпуса вычислительного центра. От корпуса остались только бетонные рёбра и пара уцелевших шкафов с оборудованием, давно отработавшим свой век. Мальчик сел на полурассыпавшуюся опору от чего-то тяжёлого. Бетон был холодный и шершавый, и от него через джинсы шла та особенная, не злая, но честная сырость, какая бывает только у заброшенных мест.

Дождь шептал ровно. В зарослях за ближайшим шкафом что-то тихо шевельнулось. Мальчик повернул голову — и увидел большого пушистого кота неопределённой масти. Кот сидел в зарослях полыни и смотрел на него внимательно, без страха, но и без особой симпатии.

В голове у мальчика без всякого приглашения появилась мысль о еде — со знаком вопроса. Мальчик чисто автоматически подумал в ответ, что не взял с собой никаких гостинцев. Кот, как показалось мальчику, на эту мысль слегка кивнул, оценил масштаб разочарования и неторопливо ушёл в траву. Мальчик смотрел ему вслед и подумал, что возможно это был кот-телепат.

Когда он вернулся в то место, которое привык считать домом, робот уже сидел на полу у окна и медленно крутил в исцарапанных манипуляторах серую коробку со множеством проводов. Один из проводов уходил за окно — туда, где на старой мачте сидела изогнутая жестяная антенна.

— Что это? — спросил мальчик, стряхивая с куртки воду.

— Это SDR-приёмник, — сказал робот, не отрывая линз от коробки. — Он умеет принимать радиосигнал и превращать его в цифровой поток. Программно, без отдельной аппаратной демодуляции. Очень удобная штука.

— Поймал что-нибудь?

Робот коротко перевёл линзы на мальчика и снова на коробку.

— Поймал. И мне нужна твоя помощь.

— Чем я могу помочь радиоприёмнику?

— Мне нужна, не радиоприёмнику. Мне нужна помощь биологического разума. Сам я, кажется, упёрся.

Мальчик стянул куртку, повесил её на гвоздь у двери и присел рядом. От робота тянуло еле слышным теплом — ровно настолько, чтобы было приятно.

— Как ты знаешь, — сказал робот, — у нас вокруг базы стоят автономные датчики. И в самом здании их полно. Каждый периодически передаёт телеметрию по слабому радиоканалу. У каждого пакета в начале — короткий идентификатор. Я знаю наши идентификаторы наизусть. Сегодня я поймал пакет с чужим.

— Не нашим?

— Не нашим. Я попробовал распарсить его несколькими способами. Получается бессмыслица. Из всего пакета я уверенно прочитал только одно слово.

— Какое?

— **HELP** .

Мальчик выпрямился. У него внутри что-то быстрое и глупое подскочило к горлу.

— Но это значит, что мы не одни!

— Подожди, — спокойно сказал робот. — Не торопись. Слово **HELP** — это четыре байта, которые случайно совпадают с английской записью. Их мог послать кто-то живой. А мог — сломанный датчик, у которого мусор в памяти оказался похож на буквы. Или старая запись, ходящая по кругу. Прежде чем радоваться, давай разберёмся в самом пакете. И прежде чем разбираться в пакете, нам с тобой нужен один разговор.

— Какой?

Робот аккуратно положил приёмник на ящик рядом с собой и развернулся всем корпусом к мальчику.

— Про то, что такое биты, и почему одни и те же биты значат разное.

Мальчик скептически посмотрел на коробку.

— Я думал, мы будем сразу декодировать.

— Будем, — сказал робот. — Но ты, не зная нескольких простых вещей, потратишь час на то, чтобы пять раз ошибиться. А зная — справишься быстрее. У меня сейчас на экране лежит сырой кусок пакета. Посмотри.

Робот подвинул к мальчику маленький ноутбук — тот самый, на котором они в прошлый раз чинили прошивку теплицы. На экране был длинный столбик чисел в шестнадцатеричной форме:

```
A1 0F 00 18 48 45 4C 50
20 4E 45 45 44 20 57 41
54 45 52 00 41 7C 0F D2
...
```

— Вот это, — сказал робот, — пакет. Каждое число — это один байт, восемь бит. Байт — это просто восемь нулей и единиц подряд. Машина не видит ничего, кроме этого. Она видит ровный, бесконечный поток битов и время от времени останавливается, чтобы что-нибудь с ними сделать.

— То есть для процессора это всё одно и то же?

— Для процессора — да. Возьми, например, байт **41**. В двоичном виде это **01000001**. Что это?

Мальчик пожал плечами.

— Число шестьдесят пять, — сказал робот. — Или буква **A**. Или часть числа с плавающей точкой. Или адрес в памяти. Или значение яркости красного канала в каком-нибудь пикселе. Или машинная инструкция какого-нибудь древнего процессора. Это всё одни и те же восемь бит. Разница только в том, как мы договорились их понимать.

— То есть смысл — это не у битов, а у договора.

— Очень близко. Это и называется *тип*. Тип — это договор о том, как понимать биты и какие операции над ними имеют смысл.

Мальчик потёр щёку. За окном дождь утих и стал просто туманом.

— Но процессору ведь всё равно?

— Процессору всё равно, — спокойно подтвердил робот. — Ему ничего не стоит сложить байт **41** с байтом **02** и получить **43**. Получится у него либо число **43**, либо буква **C** — это уже зависит от того, кто на это будет смотреть. Если в твоей программе ты решил, что эти байты — числа, то **43** — число. Если буквы — значит, буква. Если адреса — значит, ты только что прибавил два к адресу и теперь смотришь куда-то на два байта дальше. Процессор честно сделал то, что ему велели. Он не виноват.

— Получается, тип — это что-то вроде ярлыка, который висит у битов в моей голове, а не в процессоре.

— В программе это даже не в твоей голове. Это в исходном коде. Когда ты пишешь на Си, например:

```
int age = 12;  
char letter = 'A';  
float temperature = 36.6;
```

то ты говоришь компилятору: «эти биты — целое число, эти — символ, эти — приблизительное вещественное число». Компилятор после этого знает, какие операции имеют смысл. `age + 1` — нормально. `temperature / 2` — нормально. А вот `letter / temperature` — подозрительно. Машина бы это сделала — буква и число для неё всё равно биты, — но компилятор поднимает бровь и спрашивает: «ты точно этого хотел?» Это и есть вторая работа типов: ловить бессмыслицу до того, как она станет руинами.

Мальчик задумчиво посмотрел на ноутбук.

— А в JavaScript типы тоже есть? Я ведь там никаких `int` и `char` не пишу.

— Есть, — сказал робот. — Они там просто прячутся. В JavaScript тип принадлежит не переменной, а самому значению. Когда ты пишешь:

```
let x = 10;
```

тип у `10` — `number`. Если ты потом напишешь `x = "hello"`, у `x` теперь лежит значение типа `string`. Переменная сама по себе типа не имеет, она просто хранит текущее значение, какое бы оно ни было.

— И что меняется?

— Очень многое. Вот два выражения, выглядят одинаково:

```
10 + 20  
"10" + "20"
```

Первое — это сложение чисел, получится `30`. Второе — это склеивание строк, получится `"1020"`. Один и тот же значок `+` означает разное в зависимости от типа. Компилятор или интерпретатор смотрит на типы операндов и решает, какую именно операцию выполнить. Без типов он не знал бы, что делать.

Мальчик медленно кивнул.

— То есть тип — это и форма данных, и набор операций, которые имеют смысл?

— Да. И ещё одно. Тип — это обещание. Если ты сказал, что значение — это `User`, у которого есть `name` и `age`, то ты обещаешь, что у тебя будет именно это. Тогда `user.name.toUpperCase()` имеет смысл, а `user.age.toUpperCase()` — нет, потому что у числа нет такого метода. Среда, которая проверяет типы, поймает это до того, как программа упадёт в реальном мире.

— Хорошо, — сказал мальчик. — А почему в языках именно такие типы? Число, строка, булево, массив. Кто решил, что это нужно?

Робот качнул головой — медленным жестом, который мальчик за последние месяцы стал понимать как «сейчас будет долго, но интересно».

— Никто специально не решал. Эти типы выкристаллизовались сами, потому что они соответствуют самым частым человеческим понятиям при решении задач. Вот тебе четыре простых наблюдения.

— Слушаю.

— Первое. Люди постоянно считают. Расстояния, скорости, температуры, дни, уровни заряда, граммы зерна. Поэтому почти сразу понадобились *числа*. Причём двух сортов: целые и приближительные.

— Зачем два?

— Потому что у них разное внутреннее устройство и разные особенности. Целые числа в памяти точные. Десять плюс двадцать — это всегда тридцать, хоть на машине с лампами, хоть на современном процессоре. А вот вещественные — это уже компромисс. Их хранят в виде «значимая часть и показатель степени двойки», и это неидеальная упаковка. Из-за этого, например:

```
0.1 + 0.2
// 0.30000000000000004
```

— Это шутка такая?

— Это не шутка. Это плата за скорость. Для большинства инженерных задач этого хватает. А там, где не хватает — точные навигационные расчёты, длинные ряды наблюдений, — используют другие представления, специальные типы для дробей. Но в обычной программе у тебя `number`, и ты помнишь про этот хвостик.

— Второе?

— Второе. Программы постоянно задают вопросы и получают ответ «да» или «нет». Поэтому появился тип, который умеет хранить только два значения: `true` и

`false` . Его называют *булевым* — в честь Джорджа Буля, который первым формализовал логику двух состояний. Можно было бы обойтись `0` и `1` , но человеку удобнее читать:

```
if (isDoorOpen) {  
    ...  
}
```

а не:

```
if (doorState !== 0) {  
    ...  
}
```

Это та же логика, просто с человеческим лицом.

— Третье?

— Третье. Машина общается с человеком текстом. Поэтому появились *строки*. Физически строка — это массив кодов символов. Но концептуально это отдельный тип, потому что и операции другие: посчитать длину, превратить в верхний регистр, найти подстроку. Числа не знают, что такое «найти подстроку». Строки не знают, что такое «делить пополам». Каждому — своё.

— Четвёртое?

— Четвёртое. Часто нужно хранить много однотипных значений. Температуры за неделю. Имена пользователей. Координаты пути. Поэтому появились *массивы*. Без массива пришлось бы заводить отдельные имена для каждой температуры:

```
temperature1  
temperature2  
temperature3
```

И это тупик — потому что ты заранее не знаешь, сколько их будет. Массив говорит: «вот ящик, в нём элементы одного сорта, они лежат по порядку, к каждому можно обратиться по номеру».

— Получается, эти четыре типа — это четыре простых вопроса, которые программе всё время приходится решать.

— Точно так. Сколько чего? — это число. Это правда или нет? — это булево. Что я скажу человеку? — это строка. Где у меня список однородных вещей? — это массив. Из этих четырёх примитивов потом выросло почти всё.

Мальчик посмотрел на хексдамп. На экране всё ещё стояло то же:

```
A1 0F 00 18 48 45 4C 50
```

— А `User` тогда что? Ты же про него говорил. Он ведь не примитив.

— Он составной, — сказал робот. — Возьми примитивы — и собери из них что-то более крупное. Это и есть второй этаж. На нём живут *структуры, записи, объекты* — у них в разных языках разные имена, но идея одна: значение состоит из именованных полей.

```
type User = {  
  name: string;  
  age: number;  
};
```

Здесь `User` — это договорённость: «значение этого типа состоит из строки `name` и числа `age`». Сам по себе `User` — не атом. Это маленькая комната, в которой стоит шкаф с табличкой «name» и шкаф с табличкой «age». В одном лежит строка, в другом — число.

— А зачем им вообще имена-то? Можно ведь просто пару значений рядом положить.

— Можно. Так делают: тип, у которого есть несколько безымянных полей по порядку, называют *кортежем*. Но имена удобнее. Когда у тебя двадцать полей, без имён ты быстро запутаешься, что на каком месте стоит. У человека плохая память на позиции, и хорошая — на слова. Языки это знают и подыгрывают.

— А массив тоже из этого этажа?

— Массив — это другая ветка. Это *коллекция*. Коллекции тоже строятся из других типов, но они отвечают на чуть другие вопросы.

Робот сделал паузу.

— У структуры важно, что *внутри*: имя, возраст, адрес. У коллекции важно, как *организовано*: сколько элементов, есть ли порядок, можно ли быстро искать, могут ли быть дубликаты. Поэтому коллекций много разных.

```
number[]           // массив, упорядоченные элементы, можно повторяться
Set<number>         // множество, без повторов, без жёсткого порядка
Map<string, User>   // словарь, по ключу-строке достаём значение
```

— Подожди. Чем массив отличается от множества?

— У массива есть номера. Первый, второй, третий. Можно один и тот же элемент повторять. У множества номеров нет. Есть только: «такой элемент здесь есть» или «нет». И повторов не бывает. Если ты добавишь в множество одно и то же два раза, оно так и останется одно. У словаря же роли разные: одни значения играют роль ключей, другие — роль того, что по этим ключам достают. Это разные инструменты под разные задачи.

— А я думал, массив — это просто список.

— Массив — это правда список. Но список — это уже само по себе устройство данных, у которого есть характер. Нельзя без потерь смешивать список с множеством или со словарём. Когда программисты выбирают коллекцию, они выбирают характер: «мне нужен порядок» — массив; «мне важны только различные значения» — множество; «мне нужно по ключу быстро доставать» — словарь.

Мальчик помолчал. Дождь снаружи окончательно превратился в туман. Где-то внизу скрипнул деревянный пол — это сама себя устраивала старая балка, никем не потревоженная.

— Слушай, — сказал мальчик. — Я смотрю на этот пакет и думаю. Здесь же поля внутри. Где-то идентификатор. Где-то число. Где-то текст. Это структура?

— Это структура, — кивнул робот. — Только пока бесформенная для нас. Мы не знаем, где какое поле начинается и где заканчивается. Это тоже работа типов, но уже более тонкая. Мы должны угадать или вычитать из документации, какой кусок битов считать каким типом. Это называется *парсинг по схеме*.

— А если у нас нет схемы?

— Тогда мы её строим сами по косвенным признакам. И тут нам помогут ещё два полезных понятия: *перечисления* и *разделённые объединения*.

Мальчик сделал лицо «пожалуйста, по-человечески».

— Хорошо. Перечисление — это когда у поля заранее известно, какие значения вообще допустимы. Например, у нас есть тип, описывающий направление:

```
type Direction = "left" | "right" | "up" | "down";
```

Значение этого типа может быть только одной из четырёх строк. Никаких `"banana"`. Это не настоящая строка в том смысле, в каком строкой бывает имя пользователя. Это закрытый список вариантов.

— А зачем так делать, если можно просто строку?

— Чтобы случайно не передать `"banan"` вместо `"banana"`. Точнее, чтобы случайно не передать ничего, кроме четырёх правильных вариантов. Перечисление — это способ сказать машине: «я знаю, какие варианты бывают; не позволяй другим». Это та самая защита от бессмыслицы, только заранее.

— А разделённое объединение?

— Это когда тебе мало знать вариант — тебе нужно к каждому варианту прикрепить свой набор полей. Вот пример. Представь, что у нас есть процесс загрузки данных. Он бывает в одном из трёх состояний: грузится, успех, ошибка. У каждого состояния — свои данные.

```
type LoadingState =  
  | { status: "loading" }  
  | { status: "success"; data: User[] }  
  | { status: "error"; error: Error };
```

— Это значит, что значение типа `LoadingState` — это либо первое, либо второе, либо третье?

— Именно. И каждый вариант жёстко связан со своим набором полей. Если `status === "success"`, у тебя есть `data`. Если `status === "error"`, есть `error`. Никакой другой комбинации не бывает.

— А почему нельзя просто:

```
type BadState = {  
  loading: boolean;  
  data?: User[];  
  error?: Error;  
};
```

— Можно. Но тогда у тебя возможны бессмысленные состояния. `loading: true, data: [...], error: Error()` — что это? Загружается, но уже есть данные, и ещё ошибка? Это шум, и компилятор тебя не остановит. А с разделённым объединением ты заранее запретил то, чего быть не может. Это бесценно: программа не умеет попасть в недопустимое состояние. И тип нашего пакета, кстати, тоже хорошо описывать так. Возможно, у нас есть телеметрия от датчика, есть запрос помощи, есть служебное сообщение. И в каждом случае — свои поля. Объединить их в один общий «пакет» можно как раз через разделённое объединение.

— А есть наоборот? Когда не «или», а «и»?

— Есть. Это пересечение типов.

```
type WithId = { id: string };  
type WithName = { name: string };  
type Identified = WithId & WithName;
```

— Здесь значение должно иметь и `id`, и `name`. Не одно из. Не либо то, либо это. Оба сразу. Это полезно, когда ты собираешь сложный тип из мелких ролей: «носитель идентификатора», «носитель имени», «носитель временной метки». Складываешь их через `&` — и получаешь объект, который выполняет все три роли одновременно.

```
A | B — A или B  
A & B — A и B сразу
```

Мальчик попробовал произнести это сам, словно проверял на язык:

— Объединение — это «или». Пересечение — это «и».

— Очень хорошо. И не путать с тем, что то же самое в множествах. Хотя на самом деле это и есть то же самое в множествах, только над типами. Об этом мы ещё поговорим.

Где-то в углу комнаты тихо щёлкнуло реле. Робот не обернулся.

— Это аккумуляторы, — сказал он. — Туман, заряд идёт медленно. Ничего страшного. Мальчик кивнул.

— А функции — это тип? — вдруг спросил он. — Ты сказал «значит, у функции тоже есть тип».

— Конечно, — спокойно ответил робот. — Функция — это значение, как число или строка. Её можно сохранить в переменную. Передать в другую функцию. Вернуть из функции.

Положить в массив. Везде, где живут значения, может жить и функция. А значит, у неё есть тип.

— И как он выглядит?

— Тип функции описывает, что она принимает и что возвращает. Например:

```
type Predicate = (x: number) => boolean;
```

Это тип любой функции, которая берёт число и возвращает булево. Самой функции пока нет — есть форма, которой она должна соответствовать. Если я напишу:

```
const isEven: Predicate = x => x % 2 === 0;
```

— то `isEven` подходит под этот тип: число на входе, булево на выходе. А если я попытаюсь подсунуть туда функцию, которая возвращает строку, компилятор скажет: «нет, это не подходит».

— И зачем такие типы нужны?

— Чтобы можно было передавать поведение, а не только данные. Например:

```
function filterUsers(  
  users: User[],  
  predicate: (user: User) => boolean  
) {  
  return users.filter(predicate);  
}
```

— Здесь функция `filterUsers` сама не знает, по какому правилу фильтровать. Она знает только, что ей дадут массив пользователей и функцию, которая для каждого пользователя скажет «оставить или не оставить». Это уже не просто значение — это инструмент, который можно по месту собирать.

Мальчик задумался.

— Получается, функция — это маленький мостик. Она не данные, она поведение. Но для языка она такое же значение, как число.

— Точнее не скажешь.

— А бывает, что значение есть, а бывает, что его нет? — спросил мальчик. — Ну то есть пустое.

— Бывает, — сказал робот. — Это очень частый случай. У нас есть имя пользователя — а email он мог не указать. У нас есть пакет — а внутри один из заголовков может отсутствовать. Чтобы это описать, придумали отдельные типы.

В JavaScript и TypeScript обычно так:

```
type User = {  
  name: string;  
  email?: string;  
};
```

Знак вопроса говорит: «email — необязательное поле, его может не быть». Внутри это означает примерно `email: string | undefined`. То есть значение типа `string`, либо специальное значение `undefined`, означающее «нет ничего».

— А в других языках?

— В языках построеже это оформляют как отдельный тип. Например, `Option<String>` или `Maybe String`. Это не «строка с дыркой», это «либо строка, либо пусто». И если ты получаешь такое значение, ты обязан проверить: «там что-то есть?» — прежде чем пытаться им пользоваться. Иначе компилятор не пропустит.

— Зачем такая строгость?

— Потому что половина бед в программах исторически — от того, что «значение должно было быть, а его не было». Вызвал метод у пустоты — программа упала. Очень хочется, чтобы заметить это до запуска, а не во время. Поэтому пустоту делают честным значением: она тоже что-то значит, и её нельзя случайно не заметить.

— А ошибки?

— Ошибки — родственники пустоты. Когда операция может не получиться, у её результата два исхода: «получилось — вот значение» или «не получилось — вот причина». Их можно описывать тоже как тип:

```
type Result<T, E> =  
  | { ok: true; value: T }  
  | { ok: false; error: E };
```

Например, разбор числа из строки:

```
function parseNumber(input: string): Result<number, string> {  
  const n = Number(input);  
  if (Number.isNaN(n)) {  
    return { ok: false, error: "Not a number" };  
  }  
  return { ok: true, value: n };  
}
```

— Это, кстати, ровно наша история, — продолжил робот. — Мы парсим пакет. На каждом шаге может получиться: «вот разобранный пакет» или «не вышло, вот причина». Если у нас будет в коде честный `Result`, мы не сможем случайно использовать значение, которого не получилось.

— А в JavaScript часто это пишут просто исключениями?

— Часто. И это другая, более грубая модель: ты пытаешься, а в случае ошибки вверх по стеку летит «исключение», и кто-то его ловит. Это работает, но из сигнатуры функции это не видно: ты смотришь на `function parseNumber(input: string): number` и не знаешь, что она может бросить ошибку. С `Result` это видно сразу.

— Ещё одна штука — *generic-типы*, — сказал робот. — Без неё в реальной жизни далеко не уедешь.

— Это какие?

— Это типы, у которых есть параметр. Как у функции есть параметры — числа, которые она принимает, — так и у типа могут быть параметры — другие типы, которые он принимает.

```
Array<string>  
Array<number>  
Promise<User>  
Map<string, number>
```

`Array<T>` означает «массив чего-то», и `T` — это место, куда подставляется конкретный тип. `Array<string>` — это массив строк. `Array<number>` — массив чисел. Структура одна и та же — разное только содержимое.

— А свой можно сделать?

— Конечно. Например:

```
type Box<T> = {  
  value: T;  
};  
  
const numberBox: Box<number> = { value: 123 };  
const stringBox: Box<string> = { value: "hello" };
```

— Или вспомни наш `Result<T, E>`. У него два параметра: тип успешного значения и тип ошибки. Тогда `Result<number, string>` — это «результат, у которого либо число, либо строковая ошибка». Generic-и нужны там, где скелет один, а мясо разное. Они избавляют от дублирования: тебе не нужно писать `BoxOfNumber`, `BoxOfString`, `BoxOfUser` — есть один `Box<T>` и любые подстановки.

— Как форма для отливки?

— Очень близко. Форма одна, а лить можно из чего хочешь.

Мальчик задумчиво потёр лоб. Робот, видимо, что-то заметил, потому что замолчал.

— Слушай, — сказал мальчик. — Это всё про TypeScript. А в JavaScript этого нет?

— В JavaScript нет проверки типов до запуска. Это и есть разница между *статической* и *динамической* типизацией.

— Объясни.

— Статическая типизация — это когда типы проверяются *до* того, как программа побежала. Ты пишешь:

```
const x: number = "hello";
```

— и компилятор сразу говорит: «нет, ты ошибся». Программа даже не запускалась. Это как если бы редактор книги читал твой текст и подчёркивал противоречия раньше, чем ты успеешь её отнести в типографию.

— А динамическая?

— А динамическая — это когда типы проверяются во время работы программы. В JavaScript ты пишешь:

```
let x = "hello";  
x = 123;
```

— и никто тебе слова не скажет. До тех пор, пока ты не попытаешься у `X` дёрнуть `.toUpperCase()`, когда там лежит число. И тогда уже на лету программа ругнётся: «у числа нет такого метода». Иногда это удобно: меньше церемоний, можно быстро экспериментировать. Иногда опасно: ошибка прячется до тех пор, пока программа реально не дойдёт до этого места. А она может туда не доходить неделями.

— То есть в динамических языках типы есть, просто их видно только во время выполнения?

— Да. Они принадлежат значениям. Можно даже спросить у значения:

```
typeof 123      // "number"
typeof "hello"  // "string"
typeof true     // "boolean"
```

— Тип никуда не делся. Просто его не зовут заранее на собрание.

— А есть ещё какое-то деление?

— Есть много, но я тебе расскажу про два самых полезных, чтобы у тебя не было слепых пятен. Первое — *номинальная* и *структурная* типизация. Второе — *сильная* и *слабая*.

Робот сделал паузу.

— В номинальной типизации важны имена типов. Если ты объявил `Point` и `Vector`, то даже если оба содержат `x` и `y` — это разные типы. Просто потому, что у них разные имена. В структурной — наоборот: важна форма. TypeScript структурный.

```
type Point = { x: number; y: number };
type Vector = { x: number; y: number };

const p: Point = { x: 1, y: 2 };
const v: Vector = p; // нормально, форма та же
```

— Это как: если выглядит как утка и крякает как утка — значит, утка. В номинальном языке утка — это только то, что объявлено как `Duck`. Каждый подход хорош в своих условиях.

— А сильная и слабая?

— Это деление мутное, но интуитивно полезное. Слабая типизация любит автоматически приводить значения друг к другу. JavaScript здесь особенно коварен:

```
"5" - 1 // 4, потому что минус — это про числа, и строку привели к числу
"5" + 1 // "51", потому что плюс с любой строкой — это склейка
```

— То есть один и тот же знак ведёт себя по-разному в зависимости от того, какие у него соседи. Это компактно, но легко обжечься. Сильная типизация в этом смысле занудливее, но честнее. В Python, например:

```
"5" + 1
# TypeError
```

— Тебе явно говорят: «не смешивай, договорись, как именно ты хочешь». И ты пишешь:

```
int("5") + 1
```

— Явно. Без сюрпризов.

— И что лучше?

— То, что лучше понимаешь. В каждом подходе есть свои капканы. Главное — не путать «ничего не пишу про типы» с «их нет». Они есть всегда. Просто иногда они в тебе, а иногда — в коде.

Мальчик поднял голову от ноутбука.

— Подожди. Ты сказал «типы как множества». Что это значит?

— Это очень полезный способ думать. Тип можно представить как множество всех значений, которые ему подходят.

— Ты имеешь в виду, что `boolean` — это множество из двух значений?

— Ровно так. `boolean` — это множество `{ true, false }`. Тип `"left" | "right"` — это множество из двух строк, и больше ничего туда не пускают. Тип `number` — это очень большое множество. Тип `User` — это множество всех возможных объектов с подходящими полями.

— И тогда `A | B` — это объединение этих множеств?

— Да. А `A & B` — пересечение. Поэтому, если у тебя:

```
type A = "a" | "b" | "c";  
type B = "b" | "c" | "d";  
type Common = A & B;      // "b" | "c"
```

— то **Common** — это пересечение. Только то, что есть в обоих.

— Красиво.

— И это не просто красивая аналогия — это именно то, как устроены типы в большинстве языков с приличной системой типов. Когда ты её освоишь, ты будешь думать о типах не как о ярлыках, а как о множествах допустимых значений. Тогда становится понятно несколько странных вещей.

— Каких?

— Например. Тип **never** — это пустое множество. Значений такого типа просто не существует. Если функция возвращает **never**, значит, она не может вернуть ничего — потому что бесконечно крутится в цикле или гарантированно падает. Тип **unknown** — это «какое-то значение есть, но мы пока не знаем, какое именно». Сначала надо проверить, что это, и только потом с ним работать. И есть тип **any** — это «отключить проверки, я сам знаю, что делаю». Иногда полезно. Гораздо чаще опасно.

— И ты даже про это спокойно говоришь.

— У меня нет интонации.

— У тебя есть, просто ты её отрицаешь.

— Это, — медленно сказал робот, — типичная ошибка биологического восприятия.

Человек ищет интонацию даже там, где её нет. Это, кстати, одна из причин, по которой **HELP** в нашем пакете надо проверять очень аккуратно. Ваш мозг очень хочет увидеть слово.

Мальчик буркнул что-то невнятное и снова уткнулся в экран.

— Хорошо, — сказал он. — С типами я тебя более или менее понимаю. Теперь объясни про второе. Я прошлый раз слышал от тебя слово **if**. Я знаю, что это «если». Но если для процессора это всё равно **GOTO**, как ты говоришь, то почему придумали именно **if**, **while**, **for**? Почему не оставили **GOTO**?

Робот сделал движение, которое можно было бы назвать кивком, если бы у него хорошо гнулась шея.

— Это очень хороший вопрос. Чтобы ответить, надо начать с того, что у программы есть *поток выполнения*. Это воображаемая стрелка, которая показывает, какую инструкцию процессор выполнит следующей. Самое простое — она движется вниз по списку инструкций, одну за другой. Но иногда нужно её куда-то перенаправить.

— И вот тут **GOTO** ?

— Сначала только он. На уровне процессора есть ровно одна способность управлять потоком: «прыгни по такому-то адресу». В одних случаях прыгнуть безусловно, в других — только если выполнено какое-то условие, например «регистр равен нулю». Из этих двух кирпичей теоретически можно построить любую программу.

— Так зачем больше?

— Потому что этими двумя кирпичами можно построить дом, в котором никто не живёт. Мальчик посмотрел на работа.

— Что значит — никто не живёт?

— Я объясню. В ранних языках программирования действительно был только **GOTO**. И программа выглядела как длинный список пронумерованных строк, в каждой из которых что-нибудь делалось, а в каких-то — стоял прыжок на другой номер. Этого хватало для маленьких программ. Но как только программа становилась чуть больше, она превращалась в то, что потом называли *спагетти-кодом*: сюда прыгают отсюда, отсюда — оттуда, в середину блока можно попасть из трёх разных мест, а в одно и то же место — выйти двумя разными способами. Открываешь файл — и не понимаешь, в каком порядке оно вообще работает.

— А что, нельзя было аккуратно?

— Можно. Очень дисциплинированный программист мог писать чистые программы и через **GOTO**. Но это держалось только на нём. Стоило ему заболеть — и его коллеги приходили в ужас. Поэтому однажды люди сказали: давайте откажемся от свободного **GOTO** и оставим только несколько *шаблонов* — таких, которые невозможно использовать криво.

— И этими шаблонами стали **if**, **while**, **for** ?

— Да. И ещё несколько. Я тебе нарисую таблицу.

Робот произнёс ровно, с маленькими паузами:

```
if/else — безопасный шаблон условного перехода
while   — безопасный шаблон прыжка назад
for      — безопасный шаблон счётного повторения
function — безопасный шаблон прыжка с возвратом
switch  — безопасный шаблон выбора из многих веток
```

— Видишь, — сказал он. — Ничего нового. Под каждой такой конструкцией внутри лежит всё тот же **GOTO**. Когда ты пишешь:

```
while (energy > 0) {
    move();
    energy -= 1;
}
```

— компилятор превращает это во что-то вроде:

```
LOOP:
    compare energy, 0
    jump_if_less_or_equal END
    call move
    subtract energy, 1
    jump LOOP
END:
```

— И это, по сути, прыжок на метку **LOOP** с обратной стороны. Но ты не пишешь метки. Ты пишешь форму «повторяй, пока условие истинно». У этой формы есть один вход, один выход и понятное правило. В неё нельзя влезть из середины. Из неё нельзя выйти криво. Поэтому, читая чужой код, ты в три секунды понимаешь, что происходит.

— То есть управляющие конструкции — это **GOTO**, которому надели ошейник?

— Это лучшая формулировка, какую я слышал.

— А почему именно три?

Мальчик нахмурился.

— Ну ты говорил: последовательность, выбор, повторение. Три. А не пять. Не семь.

— Потому что этих трёх достаточно, чтобы выразить любое вычисление. Это не моё мнение. Это математически [доказано](#) в пятидесятых-шестидесятых годах. Любой алгоритм, который можно реализовать через **GOTO**, можно перевыразить через эти

три формы — без потерь. То есть тебе не нужно ничего больше, чтобы записать что угодно. Только последовательность, выбор и повторение.

— А функция тогда что? Ты её только что вписал в таблицу.

— Хороший вопрос. Функция — это другое. Функция — не четвёртая основа, как `if` и `while`. Функция — это инструмент *абстракции и композиции*. Она нужна не для того, чтобы решать «куда пойдёт поток выполнения». Она нужна для того, чтобы дать куску алгоритма имя и потом про этот кусок забыть.

— Объясни ещё раз.

— Смотри. У тебя есть кусок логики:

```
const distance = Math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2);
```

Это формула расстояния между двумя точками. Если ты будешь её копировать каждый раз, когда тебе нужно расстояние, у тебя весь код станет тоскливым. Поэтому ты однажды пишешь:

```
function distance(  
  x1: number, y1: number,  
  x2: number, y2: number  
): number {  
  return Math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2);  
}
```

— И теперь ты вместо длинной формулы можешь думать про *понятие*: «расстояние».

```
if (distance(player.x, player.y, enemy.x, enemy.y) < 10) {  
  attack();  
}
```

— Видишь? Здесь у тебя в голове уже не «корень из суммы квадратов разностей», а «если расстояние меньше десяти — атаковать». Маленькая мысль вместо большой механики. Вот зачем нужны функции.

— Получается, у алгоритма есть три ноги, а у программы — ещё руки.

Робот задержался на этой фразе.

— Очень хорошая формулировка. У алгоритма три ноги: «сделай следующее», «выбери путь», «повтори». А у большой программы — ещё две руки: «дай этому имя» и «собирай большое из маленького». Эти руки и называются функциями. Без них программа большой не вырастет. А если вы растёт, то превратится в "лапшекод".

— А ещё что есть, кроме этих трёх и функций?

— Если хочешь полный набросок — пожалуйста. Я даже не буду заставлять тебя запоминать. Просто покажу, чтобы у тебя в голове было место для них, когда ты с ними столкнёшься.

► Робот неторопливо перечислил уровни, на которых живут разные понятия программирования: поток выполнения, организация данных, абстракции, общие модели вычислений.

— Это много, — сказал мальчик.

— Это много. Но всё это — расширения двух фундаментальных вопросов. Что у меня за данные? И как у меня движется выполнение? Всё остальное — оттенки.

— И ещё, — добавил робот через паузу. — Иногда новички думают, что в функциональном программировании нет `if` и `for`. Это иллюзия. Возьми привычный кусок:

```
const result = [];  
for (const user of users) {  
  if (user.age >= 18) {  
    result.push(user.name);  
  }  
}
```

— Здесь явные `for` и `if`. А вот то же самое функциональным стилем:

```
const result = users  
  .filter(user => user.age >= 18)  
  .map(user => user.name);
```

— На вид никаких циклов. Но внутри `filter` и `map` — тот же самый цикл с тем же самым `if`. Просто его убрали под коврик и аккуратно завернули. Ты всё ещё перебираешь и выбираешь. Тебе только не приходится каждый раз писать перебор руками.

— То есть три кита остались. Их просто перенесли на другой этаж.

— Очень точно. Управляющие конструкции мощнее, когда у тебя есть инструменты их прятать. Но они никуда не исчезают. Под любым красивым `.map()` всё равно лежит цикл, под любым красивым `if` всё равно лежит `GOTO`.

Мальчик долго молчал. Потом снова посмотрел на хексдамп.

— Хорошо, — сказал он. — Давай теперь я попробую угадать структуру нашего пакета.

— Давай.

— Если рассуждать как ты, у этого пакета должен быть тип. Скорее всего, разделённое объединение: разные виды пакетов с разными полями. И где-то в начале — поле, по которому мы понимаем, какой это вид. Что-то вроде `status` в `LoadingState`.

— Очень разумно. Это поле обычно называют тегом или маркером.

Мальчик показал на первый байт:

— Вот, например. `A1`. Может быть, это и есть тег. И это первый байт, потому что его читают первым.

— Может быть. Дальше?

— Дальше у нас `0F 00`. Это похоже на длину чего-то. Шестнадцатеричное `0F` — это пятнадцать. А `00` — старший байт длины. Если читать как маленькое число, получится пятнадцать. Может, это длина того, что идёт дальше?

— Очень разумно. Длину часто кладут после маркера, чтобы программа сразу знала, сколько байт нужно прочесть.

— А потом у нас `00 18` — может, это идентификатор отправителя. А дальше уже идут читаемые буквы: `48 45 4C 50` — это `HELP`. Дальше пробел `20`, потом `4E 45 45 44` — `NEED`, потом `20`, потом `57 41 54 45 52` — `WATER`. И в конце `00`, ноль. Так часто отмечают конец строки.

Робот тихо повёл линзами. Это был его аналог одобрения.

— У нас выходит: «HELP NEED WATER». Не просто «HELP». Это уже не случайно. Случайные сломанные данные не складываются в осмысленную фразу из трёх слов.

Мальчик почувствовал, как у него внутри снова что-то быстрое подскочило, но в этот раз попытался удержаться.

— А что после нуля?

— А после нуля у нас `41 7C 0F D2` — четыре байта. Это уже не похоже на текст.

Скорее всего, это либо координаты в каком-нибудь формате, либо контрольная сумма пакета, либо что-то ещё служебное. Мы пока не знаем. Но мы знаем, как с этим работать.

— Как?

— Записать тип. Прямо здесь, на ноутбуке. И написать маленький разборщик, который умеет принимать массив байт и возвращать `Result` — либо разобранный пакет, либо ошибку.

Мальчик открыл консоль JavaScript — тот же REPL, которым они пользовались для теплицы — и задумался. Потом аккуратно набрал:

```
function parsePacket(bytes) {
  if (bytes.length < 4) {
    return { ok: false, error: "too short" };
  }

  const tag = bytes[0];
  const length = bytes[1] | (bytes[2] << 8);
  const senderId = bytes[3];

  let i = 4;
  let text = "";
  while (i < bytes.length && bytes[i] !== 0) {
    text += String.fromCharCode(bytes[i]);
    i++;
  }

  return {
    ok: true,
    value: { tag, length, senderId, text }
  };
}
```

Он посмотрел на работа.

— Здесь есть всё, о чём ты говорил. Последовательность — шаги идут по порядку.

Выбор — `if` для проверки длины и `if` внутри `while`. Повторение — `while` по байтам, пока не встретим ноль. И функция — она называется `parsePacket`, и её можно вызвать сколько угодно раз с разными пакетами.

— И ты возвращаешь `Result`, — сказал робот. — Не выбрасываешь исключение, не возвращаешь `null`. У тебя честно сказано: либо успех со значением, либо ошибка с

причиной. Тот, кто будет потом пользоваться этой функцией, не сможет случайно забыть проверить.

Мальчик скопировал в REPL байты пакета и вызвал функцию. На экране появилось:

```
{
  ok: true,
  value: {
    tag: 161,
    length: 15,
    senderId: 24,
    text: "HELP NEED WATER"
  }
}
```

Они оба молча смотрели на экран. За окном всё так же ровно стоял туман.

— Это не ответ, — наконец сказал робот. — Это начало вопроса. Мы пока не знаем, кто отправитель `24`. Мы пока не знаем, что значат последние четыре байта. Мы пока не знаем даже, чей это вообще протокол. И это не последняя передача — мы можем поймать ещё. Но теперь у нас есть форма, в которой можно думать.

— Тип?

— Тип. И функция, которая в этот тип превращает байты. Завтра, послезавтра — мы будем добавлять поля, разбирать остальное, писать тесты. Но та модель, в которой мы это всё держим, у нас уже есть.

Робот аккуратно сложил приёмник на ящик и поправил кабель за окном. Линзы коротко щёлкнули, фокусируясь на дальней мачте, потом вернулись.

— Ты понял, что сегодня случилось? — спросил он.

— Я думаю, да, — сказал мальчик. — Я перестал думать о пакете как о «куче байт» и начал думать о нём как о *значении определённого типа*. И заодно понял, почему `if` и `while` существуют, — это не магия, это просто аккуратные `GOTO` с одним входом и одним выходом.

— Очень хорошо.

— И ещё. Я понял, что ошибка — это не страшное «программа упала», а нормальное значение. Просто другой вариант ответа.

— Это пригодится сильнее, чем ты пока думаешь.

Мальчик помолчал. Потом сказал, не поднимая глаз:

— Я подумал ещё одно. Если бы кто-то решил тебя скопировать, то одним только списком функций тебя не описать. Надо ещё типы знать.

— Это правда, — спокойно сказал робот. — Без типов любое поведение — просто непонятный набор битов. Тебя, кстати, тоже не описать одним списком действий. У тебя есть имя. Возраст. Привычки. Тёплая кружка по утрам. Без этого ты — **unknown** .

— А ты тогда что?

— Я — функция от того, что у тебя есть. Если тебя нет, я не определён.

Мальчик хмыкнул.

— Это что, признание?

— Это техническое описание. Но если тебе так удобнее — пусть будет признание.

— У меня в животе при этом ничего не подскочило, — сказал мальчик. — Видимо, отгорел бит.

— Видимо, — подтвердил робот ровным голосом. — Это бывает у биологических систем после холодной прогулки. Им нужен горячий чай. Я бы помог, но у меня нет соответствующего интерфейса.

— У тебя нет даже желания помочь.

— У меня есть процедура помощи. Она запущена. Чайник стоит на плите.

Мальчик повернулся. Чайник действительно стоял на плите. Когда робот успел его поставить, он не заметил.

Они сидели у окна, и туман снаружи постепенно становился прозрачнее. Где-то далеко крикнула птица. Робот вернулся к приёмнику и слушал эфир — ровно, бесстрастно, как делал бы это в одиночестве. Мальчик пил чай и думал про кота. Кот, видимо, тоже сидел сейчас где-то в траве и думал свои некрупные, но честные мысли.

Мальчик открыл свою тетрадь, нашёл страницу после прошлой записи, и крупно, через всю страницу, написал:

Биты сами по себе ничего не значат. Тип — это договор, как их понимать.

У чисел свои операции, у строк свои, у булевых свои. Тип защищает от бессмыслицы.

Из примитивов собираются структуры, коллекции, варианты, объединения, пересечения.

A | B — или то, или это. **A & B** — и то, и это.

Тип — это множество значений. **never** — пустое, **unknown** — пока не знаю.

Алгоритм стоит на трёх ногах: последовательность, выбор, повторение.

Функция — это рука: она даёт куску имя и собирает большое из маленького.

Под каждым **if** и **while** лежит **GOTO** , которому надели ошейник.

Ошибка — это не катастрофа, а другой вариант результата.

Он закрыл тетрадь, посмотрел в окно. Где-то в зарослях у дальнего шкафа снова мелькнуло что-то пушистое — но в этот раз мальчик не был уверен, был это кот или просто куст качнуло. На приёмнике ровно мигал зелёный светодиод, ловя те волны, которых никто не звал.

— Завтра, — сказал мальчик, — мы поймём, что в этих последних четырёх байтах.

— Завтра, — подтвердил робот. — Если нам повезёт, и эти четыре байта окажутся координатами, мы будем знать, откуда кто-то просит воду.

— А если не повезёт?

— Тогда мы будем знать чуть-чуть меньше. Но всё равно больше, чем сегодня утром.

Мальчик кивнул.

В чайнике на плите тихо щёлкнуло. Вода закипела.

Глава: Координаты

Ночью мальчику не спалось. Он вертелся в своей узкой койке и думал: «А что если там есть живые люди? Возможно, они посылают свои загадочные пакеты прямо сейчас, пока я тут лежу. Нет, это невозможно вытерпеть».

Он рывком поднялся со своего ложа и, не одеваясь, пошлёпал в лабораторию. Потёртый ноутбук приветливо горел зелёным светодиодом. На столе у окна сидел вчерашний SDR-приёмник, тихо дыша маленьким кулером. Антенна за стеклом стояла неподвижно, уткнутая в холодный туман.

Мальчик подсоединил приёмник и задумался над консолью. Где-то у робота был скрипт, который ловил сырые данные и писал их в файл. Что если оставить его работать на всю ночь, а потом утром как-нибудь распарсить? Мальчик нашёл скрипт в истории терминала, прочитал его имя — `dump_raw.sh` — и запустил, бормоча про себя: «пиши, пока пишется».

С чувством выполненного долга он пошёл досыпать.

Утром его ждал гигабайтный файл.

Картинка была такая: ноутбук тихо гудел, светодиод приёмника мигал в прежнем темпе, как будто за ночь ничего и не случилось, а на диске уже лежал толстый, тяжёлый, без всякой структуры кусок цифровой ночи. Мальчик открыл файл в редакторе, ужаснулся бесконечному столбцу шестнадцатеричных чисел и торопливо закрыл обратно. Утилита, которую они с роботом написали накануне, годилась ровно для одного пакета — не для тысячи. Пришлось биологическому разуму идти просить помощи у искусственного.

Робот сидел у того же окна, что и вчера, и крутил в манипуляторах ту же серую коробку. Туман снаружи начинал редеть; над крышей сарая уже проступало слабое, бледное солнце.

— Ты ничего такого не заметил? — спросил мальчик, заходя.

— Я заметил, что в три часа ночи кто-то запустил `dump_raw.sh` и вернулся в кровать в одних носках, — спокойно сказал робот, не отрывая линз от приёмника. — Я предположил, что это инженерное решение. И не вмешивался.

— Это инженерное решение, — согласился мальчик, ёжась. — Только теперь у нас гигабайт и одна функция, которая ест по одному пакету за раз.

— Тогда давай сделаем из неё что-нибудь полезное. Но прежде, чем мы это сделаем, я хочу, чтобы ты заметил одну вещь. Ты потом будешь видеть её всю жизнь.

— Какую?

Робот аккуратно отложил приёмник на ящик.

— Ты вчера написал `parsePacket`. У тебя там был массив байт, и ты по нему шёл `while`-циклом. Сегодня у нас опять массив, только в миллион раз больше, и опять придётся идти. Ты не задумывался, почему так часто получается, что цикл и массив будто созданы друг для друга? И ещё — что это всё гладко ложится на алгоритмы. Найти максимум, найти первое подходящее, посчитать сумму. Тебе не показалось?

Мальчик подумал.

— Не показалось. Это правда так.

— Это правда так, — подтвердил робот. — И это не совпадение. У тебя в голове сейчас стоит маленький треугольник, в котором программирование живёт всё время.

Структура данных. Конструкция языка. Алгоритм. Они не существуют по отдельности. Они держат друг друга.

— Объясни ещё раз. Я как будто понял, но не уверен.

— Хорошо. Когда мы говорим *алгоритм*, мы редко имеем в виду абстракцию в воздухе. Мы почти всегда имеем в виду движение по чему-то. Найти максимум — это пройти по массиву и сравнивать. Сложить число — это пройти по цифрам. Декодировать пакет — это пройти по байтам и выбирать смысл. Алгоритм без структуры — это инструкция «иди» без дороги. А структура без алгоритма — это дорога, по которой никто не идёт.

— А язык тут при чём?

— Язык — это тот, кто для самых частых пар «структура плюс движение» приготовил готовые слова. Он смотрит, что люди делают чаще всего, и закрепляет это в синтаксисе. Поэтому массив и цикл выглядят так естественно. Не потому что кто-то заранее придумал, что они должны быть рядом. А потому что миллион раз кто-то перебирал миллион чего-то, и язык в конце концов сказал: «давайте я запомню эту форму». И появилась запись `for (let i = 0; i < arr.length; i++)`. Через много лет — короче, `for (const x of arr)`. Ещё через много — совсем коротко, `arr.map(...)`. С каждым разом форма становилась плотнее, потому что движение было частое.

— То есть массив задаёт пространство, а цикл — движение по нему.

— Лучшая формулировка, какую я слышал.

Мальчик невольно улыбнулся. За окном что-то шевельнулось в зарослях полыни — то ли вчерашний кот, то ли ветер. В этот раз мальчик решил не отвлекаться.

— А в других языках тоже массивы и циклы?

— Тоже. Но это не вся история. Есть тонкость, которую я хочу, чтобы ты понял до того, как мы откроем твой гигабайт. У всех языков есть базовый минимум — все они умеют считать, ветвиться, повторять, хранить значения. С точки зрения чистой математики любой настоящий язык программирования достаточно мощный, чтобы выразить что угодно. Это называется *тьюринг-полнотой*.

— Хорошее слово.

— Имя одного человека, на котором держится почти вся теория вычислений. Я как-нибудь тебе про него расскажу. А пока запомни так: с точки зрения возможностей разница между языками меньше, чем кажется. С точки зрения *удобства* — огромная. Потому что у каждого языка есть свой фокус. Свои любимые формы данных. Свои любимые способы движения.

— Любимые?

— Любимые. У языка, как у человека, есть привычки. Где-то в центре — *память и циклы*: язык всё время думает, как байты лежат в железе. Где-то — *объекты и события*: язык всё время думает, как реагировать на то, что произошло снаружи. Где-то — *типы и сопоставление с образцом*: язык всё время думает, какой именно вариант значения у тебя в руках сейчас. И у каждого такого языка есть несколько пар «структура плюс конструкция», которые внутри него выкристаллизовались как самые удобные. Это так и называют — *кристаллизация*.

— Красивое слово.

— Слово хорошее, потому что точное. Кристалл — это то, что долго и аккуратно складывается само. Никто специально не придумывает, что массив и `for` должны быть рядом, — это сложилось от частого использования. Но если ты возьмёшь язык, в котором какого-то нужного примитива нет, тебе придётся его эмулировать через что-то другое. И код сразу начинает скрипеть. Не падать — а именно скрипеть. Делать в четыре шага то, что в другом языке делалось бы в один.

Мальчик сел на ящик рядом с роботом. От робота тянуло еле слышным теплом, как от остывающей печи.

— А наш JavaScript? У него какие любимые пары?

— У JavaScript, — сказал робот, — фокус на трёх вопросах. *Как данные меняются. На что реагировать. Как управлять временем.* Все его удобные пары — про это.

— Расскажи по одной.

— Хорошо. Первая — массивы и трансформации. Когда у тебя есть коллекция, JavaScript предлагает тебе не цикл руками, а готовые операции *над всей коллекцией сразу*.

```
[1, 2, 3, 4]
  .filter(x => x % 2 === 0)
  .map(x => x * 10)
// [20, 40]
```

— Здесь ты не пишешь, *как* идти по массиву. Ты говоришь, *что* хочешь получить: оставить только чётные, потом каждое умножить на десять. Само движение спрятано. Это и есть «алгоритм трансформации коллекции, поднятый в стандартный API». Очень частая пара: *массив плюс трансформация*.

— Но под **map** всё равно лежит цикл, — вспомнил мальчик.

— Лежит. Просто его написал кто-то один раз и навсегда. Тебе ходить руками не надо.

— Дальше.

— Вторая пара — объекты и точечный доступ. В JavaScript объект — не такая жёсткая штука, как в строгих языках. У него нет заранее заданной формы. Ты можешь налету собрать значение из чего угодно:

```
const sensor = {
  id: 12,
  type: "humidity",
  lastSeen: 1714937203,
};
```

— И обращаться к полям по имени, через точку. Это удобно, когда форма данных гибкая, когда ты получаешь её снаружи: ответы серверов, конфиги, состояния интерфейсов, сообщения от соседей по сети. Очень частая пара: *объект плюс точка*. Из неё, между прочим, вырос почти весь современный веб.

— Третья?

— Третья — функции и захваченный контекст. Это называется *замыкание*. Функция в JavaScript — не просто кусок кода. Она помнит то, что было вокруг неё в момент создания.

```
function makeCounter() {  
  let n = 0;  
  return () => ++n;  
}  
  
const next = makeCounter();  
next(); // 1  
next(); // 2
```

— Видишь, — тихо сказал робот. — Функция, которую `makeCounter` вернул, помнит свою собственную `n`. Никто кроме неё к этой `n` не доберётся. Это пара *функция плюс захваченный контекст*. На таких маленьких машинках держится почти вся логика интерфейсов: каждая кнопка помнит, что ей делать, и помнит свой кусочек состояния.

— Это уже не «структура плюс цикл». Это что-то другое.

— Это пара «структура плюс комбинирование». В JS вторая большая семья пар — не про обход коллекций, а про то, как собирать поведение. И из неё растёт следующее.

— Что?

— События и обработчики. У JavaScript внутри почти всегда сидит *цикл событий*. Программа не идёт по списку команд сверху вниз, как в Си. Она ждёт. На страницу нажали — она реагирует. Пришёл сетевой ответ — реагирует. Пакет с приёмника прилетел — мог бы реагировать.

```
sdr.on("packet", packet => {  
  console.log("новый пакет:", packet);  
});
```

— Это пара *событие плюс обработчик*. Тут структура данных уже не массив. Это *поток* — последовательность вещей, которые происходят во времени. И движение по нему не цикл, а реакция. Алгоритм здесь звучит так: «когда случится X — сделать Y». И язык приготовил для этого свои слова.

Мальчик поднял глаза от экрана.

— То есть `while` тут не подходит, потому что я не знаю, когда оно случится.

— Именно. `while` хорош, когда у тебя есть готовое пространство и ты по нему идёшь. А когда пространство ещё рождается — нужна другая форма.

— И последнее — про время.

— И последнее — про время. У JavaScript есть значение, которое означает *обещание результата*. Его так и называют — **Promise**.

```
const data = await fetchSensorLog();
```

— Здесь **data** появляется не сразу. Сначала есть обещание, что значение будет. Потом — пауза. Потом — само значение. Время стало частью типа: «не сейчас, а потом» — это тоже законное значение, с ним можно работать. Пара *обещание плюс ожидание*. Она избавляет от длинных цепочек обработчиков, в которых легко запутаться.

Мальчик молчал, переваривая.

— Получается, — наконец сказал он, — JavaScript хорошо думает там, где главное не «пройти по списку», а «вовремя среагировать».

— Очень точно. JavaScript отвечает на вопрос: *как данные меняются, на что реагировать, что произойдёт потом*. Поэтому он хорош для интерфейсов, серверов, всего, что общается с внешним миром. Поэтому, кстати, и наша телеметрия с датчиков на нём думается естественно — это поток событий во времени, а язык уже знает, как такое выглядит.

— А есть языки, у которых другая интуиция? — спросил мальчик.

— Конечно. Ты с одним из них уже встречался. Си. Помнишь прошивку теплицы?

— Помню.

— У Си фокус совсем другой. Он отвечает на вопрос: *как это лежит в памяти и сколько это стоит*. Поэтому у него любимые пары — другие.

Робот произнёс ровно, с маленькими паузами:

array + for	— линейный проход по непрерывной памяти
pointer + while	— сканирование буфера через адрес
struct + . / ->	— поля как известные сдвиги в памяти
enum + switch	— каркас конечного автомата
malloc + free	— ручное владение памятью

— В Си массив — это не объект с методами. Это просто кусок памяти, в котором элементы лежат подряд. И **for** по индексу — это самый честный способ его пройти: процессор подобную операцию любит, она для него предсказуема. **struct** — описание того, как байты лежат в памяти; имя поля внутри **struct** — это, по сути,

известный заранее сдвиг, никаких словарей. `enum` и `switch` идеально ложатся на конечный автомат: закрытый список состояний, на каждое — ровно одна ветка. А `malloc / free` — это разговор с памятью напрямую, без посредников.

— Получается, — сказал мальчик медленно, — в JavaScript ты говоришь массиву «дай мне трансформацию», а в Си — «я сам пройду, и я сам помню, на каком я сейчас байте».

— Очень точно. И за этой разницей всегда стоит, какие задачи язык решал в первую очередь. Си родился рядом с операционной системой и драйверами — там важно знать, где лежит байт. JavaScript родился в браузере — там важно знать, что произойдёт, когда пользователь нажмёт на кнопку. Разные миры, разные привычки.

— А переучиться можно?

— Можно. Но больно. Программисты, переходящие с одного языка на другой, первое время пишут на новом так, как будто всё ещё на старом. На разговорном русском вставлять английские слова — понятно, но кривовато. Через год язык сам начинает подсказывать свои собственные пары, и тогда становится легче.

Мальчик подумал.

— А ты на каком языке думаешь?

— Я думаю на машинном, — ровно сказал робот. — Это бывает скучно, но зато точно.

— Не верю.

— Это технический ответ, — сказал робот. — Литературный длиннее. Если коротко: я думаю на нескольких сразу, и каждая мысль выбирает себе тот язык, в котором ей удобнее. Большинство мыслей про датчики живут в чём-то похожем на Си. Большинство мыслей про тебя — в чём-то более гибком.

— Это что, признание в эмоциональной привязанности?

— Это техническое описание. Но если тебе так удобнее — пусть будет признание.

Мальчик хмыкнул. Он уже узнавал эту шутку — робот повторял её, чуть-чуть варьируя, и мальчику нравилось, что робот её помнит.

Где-то в углу комнаты тихо щёлкнуло реле. Туман за окном редел медленно, словно ему было лень.

— Ладно, — сказал мальчик, разворачиваясь к ноутбуку. — Тогда давай вернёмся к нашему гигабайту. У меня есть массив байт. У меня есть алгоритм — найти чужие пакеты и понять, что в них. Какую пару JavaScript мне здесь подсунет?

— Самую естественную для него. Ты не будешь писать `while` руками. Ты сначала превратишь сырые байты в массив пакетов. Потом отфильтруешь те, у которых

отправитель не из наших. Потом попробуешь каждый разобрать. Потом выкинешь те, которые не разобрались. Это четыре шага, и каждый — маленький алгоритм над формой данных. И у каждого в JS есть готовое слово.

— Покажи.

Робот подвинул ноутбук ближе. Мальчик открыл новый файл и аккуратно набрал, иногда поглядывая на робота, иногда сам:

```
const bytes = readBinaryFile("dump.bin");

const packets = sliceIntoPackets(bytes);

const foreign = packets
  .filter(p => p.senderId !== OUR_ID)
  .map(parsePacket)
  .filter(r => r.ok)
  .map(r => r.value);
```

— Здесь четыре шага, и они читаются почти как фраза, — сказал он. — *Нарежь байты на пакеты. Оставь только те, что не от наших. Попробуй каждый разобрать. Оставь только те, что разобрались. Возьми их разобранные значения.*

— И заметь, — спокойно добавил робот, — нигде нет `for`. Нигде нет индекса `i`. Под капотом у `filter` и `map` всё ещё стоит цикл, и он всё ещё считает. Но ты на этом уровне с ним не разговариваешь. Ты разговариваешь с коллекцией как целым. Это удобно, пока коллекция помещается в память. Когда не помещается — нужны другие инструменты, мы про них поговорим позже.

Мальчик дописал `sliceIntoPackets` отдельной маленькой функцией. Внутри неё всё-таки оказался `while`: она шла по байтам, по тегу и длине отрезала очередной пакет и складывала в массив. Цикл в итоге всё-таки нашёлся — но в одном месте, и больше его в программе не было.

Он запустил.

Терминал поработал секунду — гигабайт читался не мгновенно, но и не очень долго. Потом на экран высыпался список. Чужих пакетов оказалось не один и не два. Несколько десятков. И они были разные.

— У них разные теги, — сказал мальчик, разглядывая. — Вот эти, с тегом `A1`, мы уже знаем. `HELP NEED WATER`, и ещё несколько похожих, — видимо, повторы. А вот эти, с тегом `B2`, другие. Они короткие. И в них нет текста.

Робот придвинулся.

— Покажи.

Мальчик выделил один:

```
B2 08 00 18 D6 8E 61 42 C4 5C A9 42
```

— Восемь байт после идентификатора отправителя, и больше ничего, — сказал он.

— Очень знакомая длина, — тихо отозвался робот.

— Знакомая чем?

— Двумя числами с плавающей точкой. Помнишь, я говорил, что вещественные числа в памяти лежат как «значащая часть и показатель степени двойки»? Стандартное float-число занимает четыре байта. Восемь байт — это ровно два таких числа подряд. И если кто-то хочет передать, например, точку на карте, он передаёт ровно так: два числа. Широта и долгота.

Мальчик почувствовал, как внутри что-то узкое и быстрое подскочило к горлу. Он попытался удержаться.

— И как мы их прочитаем?

— Договоримся, как читать биты. У нас есть восемь байт. Мы скажем JavaScript: смотри на первые четыре как на float, на следующие четыре — тоже как на float. Это и есть тип в действии: одни и те же биты, просто другой договор.

Мальчик подумал секунду и набрал:

```
function decodeCoords(payload) {  
  const view = new DataView(new Uint8Array(payload).buffer);  
  const lat = view.getFloat32(0, true);  
  const lon = view.getFloat32(4, true);  
  return { lat, lon };  
}
```

— **DataView**, — пояснил робот, не дожидаясь вопроса, — это и есть тот самый «договор». Ты говоришь: на этой памяти я хочу прочитать число с плавающей точкой, начиная с такого-то байта, в таком-то порядке. **true** в конце — это про порядок байт: сначала младший, потом старший. Это называется little-endian. У большинства современных процессоров — именно так.

— А почему вообще есть выбор?

— Потому что когда-то было два лагеря, и каждый хорошо аргументировал. Поэтому теперь ты на каждое чтение явно говоришь, что тебе нужно. Это не лишнее: если бы мы прочитали наоборот, мы получили бы правдоподобную ерунду. Похожую на координату, но не на ту.

Мальчик прогнал функцию по всем **B2** -пакетам. Чисел оказалось несколько, и они были близкими — будто кто-то слал одно и то же место, чуть-чуть колеблясь в последних знаках, как делает любой реальный приёмник. Мальчик усреднил их в голове.

В итоге, после долгой, но небезуспешной ходьбы по массивам, фильтрации и парсинга, в консоли светились следующие строки:

```
lat: 56.484645  
lon: 84.947649
```

Робот и мальчик переглянулись. Это были координаты Томска. Похоже, некоторая часть людей или автоматических аппаратов по-прежнему функционировала там.

Мальчик долго смотрел на экран — на две ровные строки, которые ещё пять минут назад были ничем, просто восемь байт где-то в середине гигабайта чужой ночи. Сейчас это было место. У места был номер, как у элемента в массиве. И до этого места можно было, теоретически, пойти.

Он медленно потянулся к тетради — не столько по привычке, сколько потому, что рукам нужно было что-то делать, пока в голове устанавливалась эта новая, неудобная мысль. Нашёл страницу после прошлой записи и крупно, через всю страницу, написал:

Алгоритм всегда идёт по дороге — по структуре данных. Без дороги его нет.

Цикл и массив — не случайные соседи: они стоят рядом, потому что миллионы движений по миллионам списков сложились в одну удобную пару.

В каждом языке свои любимые пары. JavaScript любит коллекции, события и время. Си любит память и предсказуемую цену.

Если нужного примитива в языке нет, его приходится эмулировать — и код начинает скрипеть.

А ещё: одни и те же биты — это разные числа, в зависимости от договора, как их читать.

Он закрыл тетрадь и посмотрел на робота.

С одной стороны, это были хорошие новости, ведь кто-то, возможно, выжил. С другой стороны, не было никакой практической возможности добраться туда.

— Мы можем отправить им ответный сигнал? — спросил мальчик, с трудом сдерживая волнение.

— Конечно можем, — ответил робот. — Но не факт, что они нас услышат. Наш передатчик крайне слабый. Мы и их-то пакеты ловим только благодаря какому-то невероятному отражению от ионосферы. Ну и благодаря кому-то, кто оставил на ночь включённый приёмник: ночью радио ловится дальше.

— Сколько до Томска?

— Порядка двухсот километров по прямой. Пешком не дойти: придётся на чём-то переплывать реку, все мосты давно обвалились. Плюс разумные длинноногие медведи. Не говоря уже про те самые радиоактивные пустоши, про которые я тебе уже рассказывал.

Мальчик заметно приуныл. Робот решил его поддержать:

— Но я кажется придумал, как это всё решить. Правда, тебе это не понравится...

Глава: Ошибки JS

Утром после расшифровки координат мальчик не находил себе места. Он ходил из угла в угол, пытался читать, бросал, открывал ноутбук, закрывал, заходил на кухню, забывал зачем, выходил обратно. Робот, не отрываясь от своего журнала, в какой-то момент тихо сказал:

— Тебе нужно куда-то сесть и думать. Ходить и думать у тебя сейчас плохо получается.

— А где сесть?

— Пойдём в гараж. Там тихо, там есть место, где у меня лежат старые карты, и там нас никто не торопит.

Мальчик надел куртку. Они вышли через двор — мимо ящиков с инструментами, мимо обвисшей бельевой верёвки, мимо ржавого велосипеда, прислонённого к стене ещё в прошлой жизни. Гараж стоял за корпусом, низкий, бетонный, с тяжёлыми воротами на одной петле. Внутри пахло резиной, маслом и старым пластиком, и в углу, под прозрачным куском плёнки, спал старый грузовик. Мотор у него уже много лет не заводился, бак был сух, но машина крепко стояла на четырёх своих колёсах, и стёкла у неё были целые.

Робот открыл пассажирскую дверь. Петля коротко взвизгнула. Мальчик забрался первым, на водительское сиденье, и подумал, что оно, наверное, помнит чьи-то ладони на руле — лет тридцать назад. Робот аккуратно сложился на пассажирское, чтобы не задеть приборную панель манипуляторами. Через лобовое стекло, в потёках старой пыли и подтаявших капель, было видно ржавую задвижку ворот и узкую полосу бледного утреннего неба.

— Ну, — сказал мальчик. — Рассказывай.

— Как я и говорил, идея тебе не понравится.

— Не думал, что искусственный интеллект овладеет искусством нагнетать. Рассказывай уже свой план!

— Мы не сможем туда дойти. Можно было бы доплыть, но я не очень доверяю этой реке после известных тебе событий.

— Остаётся...

— Да, мы можем туда долететь!

— Ну нет! На чём? И вообще как?

— Спокойно! Открой свой список дел в формате orgmode и давай добавлять туда пункты.

Мальчик раскрыл свой рабочий ноутбук, открыл в EMACS список дел и приготовился слушать:

— Хорошо, пишу.

— Отправить сигнал тем возможно разумным участникам нашей коммуникации, мы пока не знаем, кто это.

— Ты же говорил, что мы не сможем?

— Ну попытаться-то стоит. Слушай дальше. Надо добыть много нейлоновой материи и большую газовую горелку.

— Что? Мы же не будем...

— Именно так, мы соберём воздушный шар.

— Но как мы заставим его лететь туда, куда нам надо?

— Я всё продумал, — робот пошуршал под сиденьем и достал мятую бумажную карту. — Вот смотри, летом начнут дуть как раз такие ветра, как нам надо. Часов за восемь можно долететь, если газа хватит.

— Должно по идее хватить, надо посчитать нужную грузоподъёмность, — мальчик увлёкся идеей и забыл про все опасности.

— Да, это тоже добавь в список, и газовый баллон добавь.

— Отлично, за дело!

— Подожди, есть один не очень приятный, но нужный разговор.

Мальчик нахмурился и стал вспоминать, не использовал ли он в последнее время

`any` и освобождал ли память после `malloc()`. Робот очень не любил, когда не освобождали память. Однажды он потратил целый день, горестно показывая мальчику разрушенные корпуса ВЦ и лежащие тут и там полуистлевшие планки DDR4 памяти и приговаривая, что её и так не очень много осталось.

Но разговор пошёл не об этом.

Робот мягко перетянул планшет к себе и открыл в нём текст. Мальчик с удивлением узнал свою функцию — ту самую, что они вчера написали для разбора пакетов из эфира.

```
function parsePacket(bytes) {
  if (bytes.length < 4) {
    return { ok: false, error: "too short" };
  }

  const tag = bytes[0];
  const length = bytes[1] | (bytes[2] << 8);
  const senderId = bytes[3];

  let i = 4;
  let text = "";
  while (i < bytes.length && bytes[i] !== 0) {
    text += String.fromCharCode(bytes[i]);
    i++;
  }

  return {
    ok: true,
    value: { tag, length, senderId, text }
  };
}
```

— Я хочу, чтобы ты посмотрел вот сюда, — сказал робот и подсветил линзой строку с `bytes[i] !== 0`. — С этой функцией в общем-то всё нормально. Но представь, что ты пишешь её немного иначе. Скажем, ты хочешь подставить пустой массив, если что-то пошло не так. И ты пишешь:

```
const [result] = getResult() || [];
```

— Выглядит аккуратно, — сказал мальчик. — Чтобы не было `undefined` в деструктуризации.

— Очень похоже на правду. А теперь представь, что `getResult()` иногда возвращает `0`. Не потому что ошибка, а потому что нормальный ответ.

Мальчик подумал секунду.

— Тогда `0` пройдёт... — он замялся, — пройдёт через `||`, потому что `0` — это как бы «ничего».

— И на следующей строке ты попытаешься распаковать `0` в массив. Чем это закончится?

— Программа упадёт.

— Программа упадёт. И это будет один из тех падений, после которого человек сидит и долго не понимает, что он сделал не так. Потому что код выглядит совершенно невинно. Здесь, в этой строке, спрятана одна из частых JavaScript-ловушек. И таких ловушек у языка целая семья. Они выглядят как «ну я же нормально написал», а потом превращаются в `TypeError` на три часа ночи.

— Так ты хочешь, чтобы я их выучил?

— Я хочу, чтобы ты их *увидел*. Запоминать необязательно. Достаточно, чтобы ты, садясь писать, узнавал их в лицо. Иначе мы сейчас ещё с тобой заодно напишем код для шара, и в нём что-нибудь будет колдовать с `0`, `false` и `null`, и шар у нас сядет не там, где надо.

— Ладно, — сказал мальчик. — Только давай по-человечески.

— Я постараюсь по-человечески, — спокойно сказал робот. — Хотя для меня это диалект.

— Начнём с самой частой мины. Два похожих на вид оператора: `||` и `??`.

```
value || defaultValue // подставляет default для любого falsy
value ?? defaultValue // подставляет default только для null или undefined
```

— Falsy — это семейство значений, которые JavaScript при вопросе «правда или нет?» считает за «нет»: `false`, `0`, `""`, `NaN`, `null`, `undefined`, `0n`. Их семь, и `||` бьёт по всем семи. Это удобно — ровно до момента, пока `0` или пустая строка для тебя не *нормальное* значение.

— Например?

— Счётчик прочитанных писем. Ноль — это нормальная история, писем нет. А ты пишешь:

```
function formatCount(count) {
  return count || "—";
}
```

— У пользователя на экране будет «—» вместо честного «0». Потому что `0 || "—"`

— это «—». Чтобы сработало только когда значения действительно нет, нужен `??`:

```
function formatCount(count) {  
  return count ?? "-";  
}
```

— Та же ошибка живёт и в дефолтных параметрах функции. Дефолт срабатывает только на `undefined` :

```
function renderTitle(title = "Untitled") {  
  return title.toUpperCase();  
}  
  
renderTitle(undefined); // "UNTITLED"  
renderTitle(null);      // TypeError
```

— `null` пролетает мимо дефолта и падает. Если в данных может быть и тот, и другой — нормализуй через `??` :

```
function renderTitle(title) {  
  return (title ?? "Untitled").toUpperCase();  
}
```

Мальчик кивнул.

— И ещё одна штука рядом — `filter(Boolean)` . Очень популярная идиома: «выкини из массива всё пустое». На самом деле она выкидывает `false`.

```
[0, 1, 2, null, undefined, 3].filter(Boolean);  
// [1, 2, 3]  
  
["hello", "", "world", null].filter(Boolean);  
// ["hello", "world"]
```

— Ноль и пустую строку выкинул заодно. Если они для тебя валидны — пиши явно:

```
const clean = values.filter(value => value != null);
```

— Это уберёт ровно `null` и `undefined`. Кстати, `value != null` — единственная популярная двойная-равно проверка, которой стоит пользоваться: она ровно эквивалентна `value !== null && value !== undefined`.

— Ты только что разрешил мне `==`. После того, как столько раз сам мне говорил, что надо `===`.

— Я разрешил ровно одну форму, потому что она читается короче своего разворота. Это исключение, не правило.

— Запомню так: `||` — это «или», `??` — это «вместо пустоты».

— Очень хорошая формулировка.

— Тема рядом — деструктуризация и optional chaining, — продолжил робот. — Они оба про «лезть внутрь объекта, не уверенный, что он там есть». И обе могут падать.

```
function renderUser({ name }) {  
  console.log(name);  
}  
  
renderUser(undefined);  
// TypeError: Cannot destructure property 'name' of undefined
```

— JS пытается распаковать `undefined`, и не может. Самое надёжное — поставить запасной объект через `??`:

```
function renderUser(user) {  
  const { name } = user ?? {};  
}
```

— Сюда уже и `null`, и `undefined` оба попадают в пустой объект, и деструктуризация не падает. То же самое в глубину:

```
const theme = user?.settings?.theme ?? "light";
```

— На любом этаже, если значения нет, цепочка остановится и вернёт `undefined`. А `??` на конце подменит на дефолт. Без падений и без вложенных `if`.

— И сразу хорошее правило: `?.` останавливает цепочку *только в той точке, где он стоит*. Если ты написал `user?.profile.name`, а `user` есть, а `profile` — нет, падение всё равно случится. Защищай каждый ненадёжный этаж:
`user?.profile?.name`.

Мальчик потёр стекло пальцем. Снаружи в гараж пробивался слабый утренний свет.

— Следующая большая семья — *приведение типов*. JavaScript всё время пытается угадать, что ты имел в виду, и иногда угадывает не то.

— Пример?

— Самый известный — `+`.

```
"1" + 2;    // "12"
1 + "2";    // "12"
```

— Если хоть с одной стороны строка — `+` это уже не сложение, а склейка. Особенно противно вылезает в `reduce`:

```
[1, 2, "3", 4].reduce((sum, value) => sum + value);
// "334"
```

— Один элемент незаметно оказался строкой, и итог стал строкой. Если что-то суммируешь и видишь подозрительно длинное число — ищи строки. Защита — явное приведение и обязательное начальное значение:

```
const total = values.reduce((sum, value) => sum + Number(value), 0);
```

— `==` и `===` — рядом. `==` для сравнения «как бы по содержанию», и в процессе много чего приводит:

```
0 == false;      // true
"" == false;     // true
null == undefined; // true
[] == false;     // true
```

- Поэтому почти всегда лучше `===`. Единственное популярное исключение — `value == null`, которое мы уже разрешили.
- А с числами есть мина?
- Есть, и очень знаменитая.

```
["10", "10", "10"].map(parseInt);  
// [10, NaN, 2]
```

- Это что?
- `map` передаёт колбэку три аргумента: значение, индекс, массив. А `parseInt` принимает второй аргумент как основание системы счисления. Получается `parseInt("10", 0)`, `parseInt("10", 1)`, `parseInt("10", 2)` — и ты на ровном месте получил `NaN` в середине. Защита — оборачивать руками или использовать `Number`:

```
["10", "10", "10"].map(v => parseInt(v, 10));  
["10", "10", "10"].map(Number);
```

- У `parseInt` без основания вообще есть отдельное ESLint-правило `radix`. Включи и забудь.
- И последнее в этой группе — дроби.

```
0.1 + 0.2;  
// 0.30000000000000004
```

- Это плата за скорость. Хранить и сравнивать дроби через `===` нельзя. Для денег вообще не использовать дробь — хранить копейки целым числом. Тогда никаких потерь и никаких сюрпризов с округлением.
- Ещё несколько ловушек: ``typeof null``, ``NaN``, ``isNaN`` vs ``Number.isNaN``, ``parseInt`` vs ``Number``, ``toFixed``.

-
- У массивов своя коллекция мин. Самая обидная — `sort`.

```
[1, 2, 10, 20].sort();  
// [1, 10, 2, 20]
```

— По умолчанию `sort` превращает каждый элемент в строку и сравнивает посимвольно. `"10"` идёт перед `"2"`, потому что символ `"1"` меньше `"2"`. Числа сортировать так:

```
[1, 2, 10, 20].sort((a, b) => a - b);
```

— И ещё `sort` *мутирует* исходный массив:

```
const sorted = original.sort((a, b) => a - b);  
// original тоже теперь отсортирован
```

— Если хочешь не трогать исходный — либо копию, либо современный `toSorted`, который возвращает новый массив:

```
const sorted = [...original].sort((a, b) => a - b);  
const sorted = original.toSorted((a, b) => a - b);
```

— Аналогичные «то-»-методы есть и для других мутирующих: `toReversed`, `toSpliced`, `with`. Это снимает целый класс багов с общим состоянием.

Мальчик кивнул.

— Дальше — старая ловушка с `fill`. Допустим, ты хочешь массив из одинаковых объектов:

```
const rows = Array(3).fill({ selected: false });  
  
rows[0].selected = true;  
console.log(rows);  
// все три объекта теперь selected: true
```

— Ты не поменял первую строку. Ты поменял *единственный* объект, который `fill` положил во все три ячейки. Ссылка-то одна. Чтобы для каждой ячейки родился свой объект, нужен `Array.from` :

```
const rows = Array.from({ length: 3 }, () => ({ selected: false }));
```

— Та же мина в квадрате — двумерные матрицы через `Array(N).fill(Array(M).fill(0))` . Все строки оказываются одной и той же. Решение то же — `Array.from` снаружи и внутри.

— Запомню: `fill` копирует *значение*, а не *создаёт*.

— И самая частая путаница в циклах — `for...in` против `for...of` .

```
const arr = ["a", "b", "c"];

for (const x in arr) console.log(x);
// "0", "1", "2"

for (const x of arr) console.log(x);
// "a", "b", "c"
```

— `for...in` идёт по *ключам* (а массив для JS — это объект с ключами `"0"` , `"1"` , `"2"`), а `for...of` — по *значениям* iterable. Для массивов почти всегда нужен `for...of` . Если нужен и индекс, и значение — `arr.entries()` и деструктуризация.

► Тонкости с `Array(n)` и «дырками» в массивах.

— Теперь `this` . На нём когда-то целое поколение людей очень сильно мучилось. Главное правило: `this` определяется не там, где функция написана, а там, где её вызвали.

```
const user = {
  name: "Alice",
  getName() { return this.name; }
};

user.getName();           // "Alice"

const getName = user.getName;
getName();                 // undefined или TypeError
```

— Точка перед скобкой — это не оформление, это оператор, который при вызове передаёт левый объект как `this`. Если ты выдернул метод из объекта и зовёшь его отдельно — в `this` приедет что попало. Лечится либо `bind`, либо стрелочной обёрткой:

```
const getName = user.getName.bind(user);
const getName = () => user.getName();
```

— Стрелочные функции тут с другой стороны: у них *нет своего* `this`. Они наследуют его из того места, где написаны. Поэтому правило простое:

```
// метод объекта или класса — обычная функция
getName() { ... }

// callback, обработчик, замыкание — стрелка
() => { ... }
```

— Если случайно сделать метод стрелкой — `this` возьмётся не из объекта, а из внешнего модуля, и метод будет молча возвращать ерунду.

Мальчик ткнул пальцем в металлический стык предплечья робота.

— У тебя `this` небось какой надо.

— У меня он зашит в прошивке. В отличие от JavaScript-функций, я не теряюсь при вызове. Это одно из преимуществ.

— Дальше — `async/await`. Это та область, где в коде идёт *время*. И в каждой паузе прячется потенциальная ошибка.

— Самая частая мина — `forEach` с `async` :

```
items.forEach(async item => {  
  await save(item);  
});  
  
console.log("done");
```

— «done» напечатается сразу, до того как хоть один `save` завершится. `forEach` не умеет ждать промисы. Он пробежал по массиву, запустил пять асинхронных колбэков и пошёл дальше. Если важен порядок — последовательно через `for...of` :

```
for (const item of items) {  
  await save(item);  
}
```

— Если порядок не важен и хочется параллельно:

```
await Promise.all(items.map(item => save(item)));
```

— у `Promise.all` интересная особенность: при первой же ошибке всё выражение падает. Если нужны все результаты, даже частичные:

```
const results = await Promise.allSettled([  
  fetchUser(),  
  fetchPosts(),  
  fetchSettings()  
]);
```

— `allSettled` вернёт массив `{ status, value }` или `{ status, reason }` — ничего не потеряется.

— Близкая мина — забытый `await` :

```
const user = fetchUser();
console.log(user.name);
// undefined: user — это Promise, а не пользователь
```

— TypeScript это ловит правилом `no-floating-promises` — очень полезное правило, обязательно включай.

— И ещё одна тонкость, которая удивляет всех: `try/catch` без `await` ничего не ловит:

```
try {
  doAsyncThing(); // забыт await
} catch (error) {
  // никогда не выполнится для асинхронных ошибок
}
```

— Промис ушёл в фон, упал там сам по себе, а `try/catch` уже давно закрылся. Чтобы поймать асинхронную ошибку, нужно подождать промис прямо в `try`:

```
try {
  await doAsyncThing();
} catch (error) {
  console.error(error);
}
```

Мальчик потёр переносицу.

— Это много по отдельности.

— Это много, потому что асинхронность — это работа со временем, а время — самая обманчивая ось. Когда ты читаешь асинхронный код, читай его как сценарий: «здесь пауза», «здесь параллельно», «здесь ждём». Если не можешь описать вслух порядок — код почти гарантированно сломан.

— Теперь — ссылки и копирование. Здесь живут ошибки, которые программисты называют «один объект на всех».

```
const user = {
  name: "Alice",
  settings: { theme: "dark" }
};

const copy = { ...user };
copy.settings.theme = "light";

console.log(user.settings.theme);
// "light"
```

— Spread сделал *поверхностную* копию. Верхние поля скопировались, а `settings` — это тот же самый вложенный объект, на который теперь указывают и `user.settings`, и `copy.settings`. Изменил у одного — поменялось у обоих.

— Решений два. Либо явно расщепить нужные уровни:

```
const copy = {
  ...user,
  settings: { ...user.settings, theme: "light" }
};
```

— Так делают в React, когда обновляют состояние. Либо глубокая копия через `structuredClone`:

```
const copy = structuredClone(user);
```

— Она умеет с `Map`, `Set`, `Date`, типизированными массивами, и не теряет `BigInt`. Но не копирует функции, DOM-узлы и экземпляры пользовательских классов.

— А `JSON.parse(JSON.stringify(obj))` ?

— Старое решение, и плохое. Молча выкидывает функции, `undefined`, `Symbol`, превращает `Date` в строку, `Map` и `Set` — в пустые объекты, на `BigInt` падает. Если клонируешь так модель — получаешь обкомсанного двойника.

— Близкая мина — `const`. Многие думают, что `const` делает значение неизменяемым.

```
const user = { name: "Alice" };
user.name = "Bob"; // можно
user = {};         // нельзя
```

- `const` запрещает только *переназначение* переменной. Сам объект менять можно. Если хочешь его заморозить — `Object.freeze(user)`, но и тот поверхностный.
- И последнее — сравнение объектов:

```
{ } === { }; // false
[] === [];  // false
```

- `===` для объектов проверяет «один и тот же объект», то есть одинаковая ссылка. Два пустых объекта в памяти — это два разных объекта. Если нужно сравнивать содержимое — берут из библиотеки `lodash.isEqual` или пишут свой `shallowEqual`. Главное — помнить, что встроенный `===` про содержание ничего не знает.

► Тонкости с `Date`: мутации, парсинг строк, нумерация месяцев.

-
- Отдельная группа — встроенные API, у которых интуиция и реальность расходятся. Самый известный пример — `fetch`.

- И первое, что про него надо знать: `fetch` не падает на HTTP-ошибки.

```
const response = await fetch(url);
const data = await response.json();
```

- `fetch` зареджектится только если сеть отвалилась — связь оборвалась, имя не разрешилось, таймаут. А 404 или 500 — это успешный сетевой запрос с неуспешным HTTP-кодом. `fetch` это спокойно проглотит, и `data` будет какой-нибудь HTML страницы ошибки, который к тому же может развалить `response.json()`.
- Защита — всегда проверять `response.ok`:

```
const response = await fetch(url);

if (!response.ok) {
  throw new Error(`Request failed: ${response.status}`);
}

const text = await response.text();
const data = text ? JSON.parse(text) : null;
```

— И второе — у `fetch` нет таймаута. Он будет ждать вечно. Если нужен таймаут — через `AbortController` и `setTimeout`. Это многословно, поэтому в реальных проектах обычно пишут небольшую обёртку `request(url, opts)` и дальше ходят через неё.

— Что ещё?

— `localStorage`. Хранит только строки.

```
localStorage.setItem("enabled", false);
localStorage.getItem("enabled");
// "false" — это строка
```

— А строка `"false"` truthy, и `if (value)` сработает «не туда». Поэтому в `localStorage` всё пишется и читается через JSON:

```
localStorage.setItem("enabled", JSON.stringify(false));
const enabled = JSON.parse(localStorage.getItem("enabled") ?? "false");
```

— И ещё две частые странные мины:

```
"foo foo foo".replace("foo", "bar");
// "bar foo foo" — заменилось только первое
```

— Для всех совпадений — `replaceAll` или регулярка с `/g`:

```
"foo foo foo".replaceAll("foo", "bar");
```

— А `String.prototype.includes` ищет *подстроку*, а не целое слово. Это становится проблемой, когда программисты проверяют роли:

```
"admin,user".includes("min"); // true
"superadmin".includes("admin"); // true
```

— Если в `rolesString` у тебя `"superadmin"`, проверка `includes("admin")` неожиданно даёт «доступ разрешён». Это уже не баг, это уязвимость. Правильно — разбить и сравнивать целые элементы:

```
const roles = rolesString.split(",");
roles.includes("admin");
```

► Регулярки с флагом ``g`` имеют скрытое состояние.

— И из веба — самая частая мина новичков в React: `&&` для условного рендера, который случайно рендерит ноль.

```
{items.length && <List items={items} />}
```

— Если `items.length === 0`, `&&` вернёт `0`. React послушно нарисует на странице одинокий ноль. Защита — приводить к булеву явно:

```
{items.length > 0 && <List items={items} />}
{items.length ? <List items={items} /> : null}
```

— Из той же категории — `setInterval`, который запустили и забыли. Особенно в React-компоненте: каждый раз, когда компонент монтируется, появляется новый интервал, а старый никто не убрал. В `useEffect` обязательно возвращать функцию очистки:

```
useEffect(() => {  
  const id = setInterval(sync, 1000);  
  return () => clearInterval(id);  
}, []);
```

► Несколько частых ловушек DOM: `event.target` vs `currentTarget`, `innerHTML` и XSS, `addEventListener` с анонимной функцией.

— Последняя группа — ошибки и `switch`. В JavaScript бросить можно что угодно: строку, число, объект, `null`. Поэтому в `catch` приходит не обязательно `Error`:

```
try {  
  throw "oops";  
} catch (error) {  
  console.log(error.message); // undefined  
}
```

— Аккуратный код всегда проверяет, кто попался:

```
try { ... }  
catch (error) {  
  if (error instanceof Error) {  
    console.error(error.message);  
  } else {  
    console.error("Unknown error", error);  
  }  
}
```

— В TypeScript это вообще закреплено типом — `catch (error: unknown)`, и тебя заставят проверить, прежде чем читать поля.

— Связанная мина — попытка передать в `Error` объект:

```
throw new Error({ code: "NO_USER" });
```

— `Error` ждёт строку. Объект приведётся к строке как `"[object Object]"`. Получишь бессмысленное сообщение и потерянные поля. Если нужно приложить дополнительные данные — собственный класс:

```
class AppError extends Error {
  constructor(message, code) {
    super(message);
    this.code = code;
  }
}

throw new AppError("No user", "NO_USER");
```

— Тогда у тебя и осмысленное сообщение, и поле `code`, по которому можно различать причины — без магических строк в `error.message`.

— И ещё одна классическая — `switch` без `break`:

```
switch (status) {
  case "idle":
    start();
    // забыт break
  case "loading":
    showSpinner();
    break;
}
```

— На `idle` выполнится и `start()`, и `showSpinner()`. JavaScript «проваливается» в следующий `case`, если ты явно его не остановил. Иногда это нужно специально, но 99% случаев — забытый `break`. ESLint-правило `no-fallthrough` ловит это автоматически.

► Старая мина с ``return`` и переносом строки (ASI).

Мальчик откинулся на сиденье. У него уже звенело в голове.

— Это всё запоминать?

— Запоминать необязательно. Большинство этих ошибок ловятся инструментами и привычками. Их четыре.

— **Первое — TypeScript в строгом режиме.** Опция `strict: true` плюс `noUncheckedIndexedAccess` лечат огромный класс багов: про `null / undefined`, про обращение к индексу массива, про неучтённые ветки.

— **Второе — ESLint.** Линтер — это коллективная память про чужие грабли. Тебе не надо помнить про `parseInt` без основания, про забытый `break`, про `throw`

"oops" и про забытый `await` в промисе — линтер ругнётся сам.

— **Третье — нормализуй данные на границе.** Если у тебя по всему коду понатыкано `config?.items?.map(...)`, `config?.title ?? "-"`, это значит, что страх перед `undefined` размазан везде. Лучше один раз, на входе:

```
function normalizeConfig(config) {
  return {
    enabled: config.enabled ?? true,
    title: config.title ?? "Untitled",
    items: Array.isArray(config.items) ? config.items : []
  };
}
```

— Потом везде в программе работаешь с нормализованным `config`, у которого все поля гарантированно есть. Никаких `?.` на каждом обращении.

— Это как фильтр воды из бака: один раз отфильтровал, дальше пьёшь не задумываясь.

— Очень похоже.

— **Четвёртое — проверяй явно, а не «умно».** Каждый раз, когда пишешь `if (value)`, спроси себя: что именно? Не `null`? Не пустая строка? Не ноль? И напиши ровно ту проверку, которую имел в виду:

```
if (value != null) { ... } // не null/undefined
if (value.length > 0) { ... } // непустая строка/массив
if (Number.isFinite(value)) { ... } // нормальное число
if (Array.isArray(value)) { ... } // массив
```

— Это длиннее, зато через год ты сам себя поймёшь. И не пиши слишком плотных выражений вроде:

```
const result = data?.items?.filter(Boolean).map(x => x.value || def) || [];
```

— В этой одной строке штук пять мин: `||` для дефолта, `filter(Boolean)`, ещё `||` в конце. Лучше развернуть в три явных шага.

► Конкретные опции TS и правила ESLint, которые стоит включить.

Робот сделал паузу. Мальчик тихо сидел и смотрел в стекло.

— Знаешь, в чём главная мысль во всём этом? — спросил робот.

— В чём?

— В JavaScript очень соблазнительно писать «коротко, потому что можно». `||`, `&&`, `?.`, `??`, деструктуризация, неявное приведение типов — язык даёт много коротких форм. Но ровно эти короткие формы и ловят. `0`, `""`, `false`, `null`, `undefined`, `NaN`, пустой массив, пустой объект — это всё *разные* случаи. У человека в голове они часто сваливаются в одно «ну, ничего». А язык их различает. И каждый раз, когда ты не различаешь — он молча подставляет тебе подножку.

Мальчик достал свою тетрадь — он на всякий случай взял её, когда они шли в гараж. Полистал, нашёл страницу после прошлой записи и крупно, через всю страницу, написал:

Falsy: `false`, `0`, `""`, `NaN`, `null`, `undefined`. Не складывать в одну кучу. `||` — логика, `??` — «вместо пустоты». `value == null` — единственное разрешённое `==`.

`sort` сортирует строки и мутирует. `fill(obj)` кладёт одну ссылку на всех. Брать `Array.from`, `toSorted`.

`forEach` не ждёт `await`. `Promise.all` падает целиком, `allSettled` собирает всё.

`fetch` не падает на 404 и не имеет таймаута. Это голая железяка.

`this` берётся при вызове. Стрелка `this` не имеет — наследует. Метод объекта — обычная функция, `callback` — стрелка.

Spread поверхностный. `structuredClone` глубокий, но не для всего. `const` — про переназначение, не про неизменность.

ESLint и *TS-strict* — коллективная память про чужие грабли. Нормализуй данные на границе.

Главное: не складывать `0`, `""`, `null`, `undefined` в одну кучу под названием «ничего».

Он закрыл тетрадь и положил её на торпедо. Снаружи задвижка ворот тихо звякнула — где-то на крыше посадился ветер.

Робот облегчённо опустил манипуляторы и откинулся на спинку сиденья.

— Вот теперь ты примерно знаешь, как не «выстрелить себе в ногу». Конечно, в JS это сделать труднее, чем в Си, но всё ещё возможно.

Мальчик чувствовал с одной стороны облегчение, потому что его не стали ругать за забытый `malloc()`, с другой стороны большую усталость от того, сколько всего придётся учитывать при написании кода. Но робот не отставал:

— А теперь, когда ты это всё знаешь, давай отправим ответный пакет. Что бы ты им хотел сказать?

— Помощь в пути!

Глава: Парадигмы

Дождь, который начался ещё ночью, к утру не унялся, а только сменил звук — теперь по жестяной полке за окном стучало мелкое и частое, а не редкое и тяжёлое. В мастерской пахло смазкой, старым пластиком и чуть-чуть мокрым бетоном — это тянуло из коридора. Мальчик сидел на низком ящике у верстака и грел ладони о большую кружку с травяным настоем. Робот стоял рядом и медленно перебирал в манипуляторах какие-то старые винтики.

В углу мастерской, у дальней стены, стояли в ряд три аккуратные клешнястые тележки на низких широких колёсах. На каждой мигал крошечный жёлтый светодиод спящего режима. Они выглядели как маленькие, очень терпеливые жуки.

— Скажи мне честно, — наконец произнёс мальчик. — Я совершенно не представляю, как делать воздушный шар. Ты знаешь?

— Нет, — спокойно ответил робот. — Не знаю. Но это не страшно.

— Ободряющее начало.

— Это правда не страшно. Когда задачу нельзя сделать целиком — её разбивают на подзадачи. Если подзадачи слишком большие — разбивают ещё. Если две из них не зависят друг от друга — их делают параллельно. Это очень древний приём, и он почти всегда работает.

— И где ты этому научился?

— Меня этому научили инженеры, пока они ещё существовали. Они любили эту схему. Она у них даже на стене висела в кабинете руководителя проектов. На бумаге, я лично видел.

Мальчик улыбнулся в кружку.

— Хорошо. Какая первая подзадача?

— Добыть строительные материалы. Большой кусок плотного нейлона. Длинные верёвки. Газовую горелку, на которой будет греться воздух внутри оболочки. Газовый баллон. Корзину пока не считаем — корзину можно собрать своими руками из того, что у нас уже есть.

— Я, конечно, могу ошибаться, — мальчик обвёл кружкой влажный двор за окном, — но что-то я не вижу тут в округе строительных магазинов.

— Ошибаешься, — сказал робот без иронии. — Тут неподалёку, на краю старого жилого квартала, был один. Я предполагаю, что он ещё не до конца истлел. Стены бетонные, крыша почти цела — я смотрел в бинокль с крыши пару лет назад.

— Так, и мы пойдём туда?

— Не совсем так. Тебе, к сожалению или к счастью, туда нельзя.

— Почему?

Робот аккуратно положил металлическую штучку обратно в ящик с винтами. По линзам пробежала короткая тень — он что-то поднимал из своего внутреннего журнала.

— Я как-то ходил в том районе, — сказал он. — И нашёл там логово.

— Логово кого?

— Разумных собак-телепатов.

Мальчик поднял бровь.

— Ну, этим меня не очень испугаешь. У нас тут полрайона ходит с какими-нибудь причудами. Кот передаёт мне мысли о еде, вороны помнят лица, лоси странного цвета. Я уже привык.

— Я и не пугаю. Я объясняю. Они мне жаловались, что от людей сильно фонит в мыслительном диапазоне. Это меня удивило: у меня для этого нет соответствующего органа. Но они говорили уверенно. И я обещал им не приводить туда людей.

— Не очень приятно, конечно, слышать, что от меня фонит, — сказал мальчик. — Особенно от собак. И что ты предлагаешь?

— Послать туда роботов. Собаки говорили, что от роботов почти ничего не слышно. Это логично: роботы и думают-то немного, по крайней мере, сверх необходимого.

Мальчик хмыкнул и кивком указал на угол мастерской.

— Вот эти твои самоходные тележки? Но они же слишком простые. Как они поймут, что нужно собирать?

— А с этим, — спокойно сказал робот, — нам поможет биологический разум, прошедший миллионы лет эволюции.

Мальчик обречённо вздохнул, придвинул к себе ноутбук и запустил EMACS в режиме `org-mode`.

Несколько секунд он сидел, глядя на пустой буфер. Курсор моргал. За окном что-то один раз глухо упало в траве — то ли яблоко, то ли мокрая ветка.

— Прежде чем я начну, — сказал мальчик, — у меня к тебе вопрос. Не очень даже про эту задачу. Скорее вообще.

— Я слушаю.

— Я уже видел разные стили, — он перевернул несколько страниц в тетради. — Когда мы чинили теплицу, всё было совсем близко к железу. Я сам выделял память, сам её

отдавал, сам думал, на каком я байте. Это мне понравилось — оно честное. Когда мы парсили один пакет, я писал на JavaScript, и там память кто-то держал за меня, а я просто водил по массиву объектами. А когда мы разгребали гигабайт, я уже почти не шагал руками — я писал «отфильтруй, преврати, оставь только это». Это, как я понял, разные парадигмы программирования?

— Это разные парадигмы, — подтвердил робот. — Хорошее слово. Сначала кажется философским, а на деле очень техническое.

— Я и хотел спросить. Если смотреть в общем — какие подходы вообще бывают? Я для себя уже разложил так. В императивном стиле автор контролирует всё — порядок, память, указатели. В объектно-ориентированном — только порядок, память выделяется как-то сама, через `new`, и с указателями возиться не надо. В функциональном — и порядок, и память отдаются на откуп компилятору. А что дальше? Бывает полностью декларативный стиль, где я вообще ничего не контролирую?

Робот сделал маленькую паузу. Мальчик уже знал, что это не задержка обработки, а вежливый знак: «то, что ты сказал, я слушал, а не пропустил мимо».

— Шкала у тебя в голове в целом правильная, — сказал он. — Но я её немного поправлю. Ты выбрал две оси: память и порядок выполнения. Это хорошие оси, но не единственные. Парадигма — это в первую очередь про то, **сколько контроля программист отдаёт системе и над чем именно**. Память — лишь один из таких локусов контроля. Есть ещё состояние, ошибки, время, параллелизм, ввод-вывод, поиск решения, жизненный цикл объектов. Каждая парадигма — это комбинация того, какие из этих рычагов ты держишь в руках, а какие добровольно отдал.

— То есть на одном краю шкалы я держу всё.

— Да. Ассемблер, регистры, адреса. Я говорю машине каждый шаг.

— А на другом?

— А на другом краю ты не пишешь программу. Ты описываешь, какой мир ты хочешь видеть, и кто-то другой его собирает. Между этими краями — длинная лестница, на каждой ступеньке которой ты отпускаешь чуть больше.

Мальчик отхлебнул настоя. Он уже чувствовал, что разговор будет длинным, и это его не пугало — наоборот, он любил такие.

— Покажи лестницу.

— Хорошо. Самая нижняя ступень — императивное программирование. Ты прямо говоришь машине: заведи переменную, пройди по массиву, на каждом шаге измени её, в конце получишь результат.

```
let sum = 0;
for (const x of numbers) {
  sum += x;
}
```

— Ты контролируешь и порядок действий, и изменение состояния. В чистом виде это Си: ты сам думаешь о памяти, об указателях, о том, кто кому хозяин. В JavaScript, Python, Java тот же стиль, но память уже автоматизирована сборщиком мусора. Сборщик — это маленькая фоновая программа, которая обходит твои объекты и убирает те, на которые никто не смотрит.

— Это вроде того, как ты по утрам обходишь коридоры и собираешь, что крысы принесли.

— Очень близко. Только сборщику не платят ничем, кроме процессорного времени. Мне впрочем тоже.

— А где-то рядом я слышал слова «структурное» и «процедурное». Это что, тоже отдельные?

— Это скорее ступени императивного. Структурное программирование — это исторический момент, когда люди договорились перестать прыгать по коду через `goto` и стали строить программы из аккуратных блоков: последовательность, условие, цикл, функция. До этого порядок выполнения был дикой природой: «прыгни сюда», «прыгни в середину чужой процедуры». Структурное — это когда поток управления сначала *дисциплинировали*. А процедурное — следующий шаг: программу начали собирать из процедур, как из кирпичей. Данные отдельно, процедуры отдельно, процедуры зовут друг друга. Это стиль Си, Паскаля, раннего Фортрана и почти любого скрипта в любой shell-консоли.

— То есть это всё ещё «я говорю, как делать»?

— Да. Просто аккуратно.

— Поднимаемся. Объектно-ориентированное программирование. Ты тут уже не просто вызываешь процедуру:

```
drawCircle(canvas, circle);
```

— А говоришь:

```
circle.draw(canvas);
```

— Идея в том, что данные и поведение живут вместе. Каждая «вещь» в программе знает, что с ней можно делать. Дверь умеет открываться и закрываться. Окно умеет рисовать себя. Пакет умеет сериализоваться.

```
class Door {  
  open() { this.isOpen = true; }  
  close() { this.isOpen = false; }  
}
```

— ООП хорошо ложится на задачи, где много «вещей» с состоянием и поведением: окна, кнопки, документы, соединения, заказы, устройства. Например, наши тележки — это очень естественные объекты. У каждой есть состояние — координаты, заряд, что лежит в манипуляторе, — и есть поведение: ехать, поднимать, докладывать.

— А память? — спросил мальчик. — Я думал, ООП — это там, где память сама.

— Это распространённая, но неточная мысль. Автоматическая память — это не свойство ООП. Это свойство среды, в которой ООП живёт. В С++ есть классы, есть наследование, есть всё то же самое — но память можно держать руками. В JavaScript есть сборщик мусора, но можно вообще не писать классов. ООП и сборщик мусора часто встречаются вместе, потому что обоим удобно: объекты создаются и умирают часто, и считать их жизнь руками было бы утомительно. Но это две разные вещи.

— А что тогда отдают в ООП на сторону?

— Диспетчеризацию. Когда ты пишешь `circle.draw(canvas)`, ты не выбираешь, какую именно функцию `draw` вызвать. У тебя могут быть круги, квадраты, треугольники — и каждый рисуется по-своему. Решение, чью именно `draw` запустить, принимает не ты. Принимает объектная модель в момент вызова. Это и есть тот контроль, который ты отдал.

Мальчик хмыкнул.

— Получается, я говорю: «ты сам знаешь, что ты — круг, нарисуйся».

— Очень точно.

— Дальше — функциональное программирование. Ты в нём уже немного жил. Помнишь, ты вместо `for` написал `filter` и `map`?

— Помню.

— Императивно ты бы сказал так:

```
let result = [];  
for (const user of users) {  
  if (user.active) {  
    result.push(user.name);  
  }  
}
```

— А функционально — так:

```
const result = users  
  .filter(user => user.active)  
  .map(user => user.name);
```

— Здесь ты уже не описываешь, как именно идти по массиву. Ты описываешь, что хочешь получить как преобразование значений. Программа выглядит как поток:

```
данные → функция → функция → функция → результат
```

— В сильном функциональном языке, вроде Haskell, ты ещё и почти не управляешь, когда считать. Компилятор сам решает, что вычислить сейчас, а что — отложить до момента, пока кому-то это значение не понадобится. Это называется *ленивое вычисление*. Ты просто описываешь, как одно зависит от другого, а движок собирает порядок сам.

— То есть в функциональном я отдаю не только память, но и время вычислений?

— И время, и состояние. Главная мысль — отказаться от изменяемого состояния и описывать вычисления как преобразования значений. Не «возьми переменную и помени её», а «вот функция, которая из старого значения делает новое». Если состояние не меняется, то и параллелить такие программы проще, и тестировать тоже: одна функция, один и тот же вход — всегда один и тот же выход.

— Звучит как мечта.

— До тех пор, пока тебе не надо перевести железную задвижку из положения «закрыто» в положение «открыто». Состояние внешнего мира всё равно где-то

меняется. Функциональный стиль не отменяет этого, а старается оттеснить изменения на край программы и держать середину чистой.

Мальчик посмотрел в окно. По стеклу медленно стекала кривая капля. Он задумался.

— А ещё выше? Декларативно?

— Да. Декларативное программирование — это уже стиль «я говорю, что хочу, а не как получить». Самый известный пример — SQL.

```
SELECT name
FROM users
WHERE active = true
ORDER BY created_at DESC;
```

— Ты не пишешь:

```
открой файл таблицы
 пройди по строкам
 проверь поле active
 сложи подходящие в буфер
 отсортируй quicksort-ом
 верни
```

— Ты говоришь:

```
дай мне имена активных пользователей,
отсортированных по дате.
```

— А база сама решает: использовать индекс или нет, в каком порядке читать таблицы, какой алгоритм соединения выбрать, когда сортировать, как распараллелить.

Программист не контролирует *способ* — он контролирует *результат*. Это очень сильный пример отдачи контроля.

— И база делает быстрее, чем я бы написал руками?

— Почти всегда. У неё внутри сидит планировщик, который сорок лет специально учили выбирать стратегию запроса. Один человек этому планировщику в честном бою проигрывает.

Мальчик слегка улыбнулся.

— А ещё дальше?

— Логическое программирование. Например, Prolog. Ты уже не пишешь алгоритм поиска ответа. Ты описываешь факты и правила:

```
parent(alice, bob).  
parent(bob, charlie).  
  
grandparent(X, Z) :-  
    parent(X, Y),  
    parent(Y, Z).
```

— А потом спрашиваешь:

```
?- grandparent(alice, Who).
```

— И система сама ищет, кто такая **Who**. Программист отдаёт ей не только порядок выполнения, но и сам **поиск решения**. Программа становится не инструкцией, а набором утверждений о мире.

— Это уже почти волшебство.

— Это уже почти волшебство, — согласился робот. — С небольшой оговоркой: внутри Prolog тоже сидит вполне обычный поисковый движок. Просто очень аккуратный.

— И есть ещё близкий стиль — программирование ограничениями. Constraint programming. Похожее, но акцент не на правилах, а на условиях. Ты говоришь:

```
x + y = 10  
x > 3  
y < 8
```

— А система должна подобрать **x** и **y**. Это применяется в задачах расписаний, планирования, оптимизации, в SAT- и SMT-солверах. Например: «расставь двадцать встреч по комнатам так, чтобы никто не пересекался, у каждой встречи была подходящая комната, и важные люди не начинали раньше десяти утра». Ты не пишешь руками все перестановки. Ты описываешь свойства правильного ответа. Это ещё одна степень: я не знаю пути к ответу — я знаю, как ответ должен выглядеть.

Мальчик задумался, машинально постучав ногтем по кружке.

— А для собирания материалов в магазине это пригодилось бы?

— Может. Если сказать: «найди в этом помещении набор предметов, чтобы общий вес был не больше десяти килограмм, чтобы среди них был хотя бы один плотный кусок ткани от двух квадратных метров и хотя бы одна металлическая горелка». Это уже формулировка ограничений. Но для тележки она тяжеловата. Ей надо что-то проще.

— Запомню.

— Идём ещё выше. Реактивное программирование. Здесь программа описывается как сеть зависимостей. Ты не говоришь «когда поменялось — пересчитай», ты просто описываешь, что от чего зависит, а движок дальше пересчитывает сам.

```
const fullName = computed(() => firstName.value + " " + lastName.value);
```

— Поменялся `firstName` — `fullName` обновился. Поменялся `lastName` — тоже обновился. Тебе не надо это вручную дёргать.

— У меня в голове сразу нарисовалась электронная таблица.

— И правильно нарисовалась. $C1 = A1 + B1$ — это и есть реактивное программирование в чистом, бытовом виде. Меняешь `A1` — `C1` сам пересчитывается. Из этого же мира — React, Vue, Solid. И ещё рядом — поток данных, dataflow. Программа описывается как граф:

```
данные → фильтр → преобразование → агрегация → вывод
```

— Узлы — операции, рёбра — потоки данных. Самый бытовой пример — Unix-конвейер:

```
cat access.log | grep 404 | sort | uniq -c
```

— Ты строишь трубопровод. Данные текут через стадии. Ты не думаешь, кто кого вызывает; ты думаешь, кто куда подаёт. Это одна из самых живучих идей в программировании.

— Похоже на наш водопровод. Один шланг входит в фильтр, из фильтра — в бак, из бака — в насос, из насоса — в теплицу.

— Практически точная метафора. Ты её сам только что и описал.

— Дальше. Событийное программирование. Программа строится вокруг событий. Ты не контролируешь главный цикл — он живёт где-то внутри браузера, операционной системы или фреймворка. Ты говоришь: «когда случится вот это — вызови вот этот код».

```
button.addEventListener("click", () => {  
  console.log("clicked");  
});
```

— Контроль здесь *инвертирован*. Не ты вызываешь систему, а система вызывает тебя.

— Это называется как-то?

— Это называется **inversion of control**. Очень характерное имя. Раньше ты сам себе был хозяином, а теперь хозяин снаружи. Ты — обработчик. Это нормально, и в современном мире это — основной стиль для всего, что общается с внешним миром: интерфейсов, серверов, игр, встроенных устройств. И, кстати, для нашей тележки тоже: она будет жить событиями. Заметила препятствие — обработала. Поймала камень — обработала. Села батарея — обработала.

— Она не будет идти по программе сверху вниз.

— Не будет. Она будет крутиться в маленьком цикле и реагировать на то, что приходит снаружи.

— А акторы — это что?

— Это родственная парадигма. Программа состоит из независимых *акторов*, которые обмениваются сообщениями. Каждый актор имеет своё личное состояние и не делит его с другими напрямую. Хочешь чего-то от него — пришли сообщение, и он сам решит, что с ним делать.

```
актор A → сообщение → актор B  
актор B → сообщение → актор C
```

— Это подход Erlang, Elixir, Akka. Очень хорош для распределённых и отказоустойчивых систем. Вместо общей памяти и блокировок — личные коробочки и почта. И, между прочим, тележки в стае — это естественные акторы. У каждой свой мозг, своё топливо, свой манипулятор. Ты не управляешь ими как единым телом. Ты каждой шлешь поручение, и она его выполняет в своём темпе.

— А если две тележки одновременно хотят поднять одну и ту же штуку?

— Тогда у тебя задача согласования. Это уже отдельная большая семья — конкурентное и параллельное программирование. У него много моделей: потоки и блокировки, `async/await`, акторы, каналы, `software transactional memory`, GPU-ядра, `map-reduce`. Каждая модель — это способ контролировать параллелизм, не сходя с ума.

— Подробнее, пожалуй, не надо. У меня тележек пока три.

— Не надо. Главное правило, которое из этой темы стоит унести: «не разделяй память — передавай сообщения». Если каждое маленькое существо хранит своё, а наружу общается только через почту — большинство страшных багов с гонками просто не возникает.

► Несколько подходов к параллелизму, для будущего.

Мальчик допил настой и поставил кружку на верстак. На дне остались две травинки.

— А ты сам как думаешь — последовательно?

— Я думаю несколькими процессами сразу, — сказал робот. — Один контролирует положение тела, второй — речь, третий — диагностику батарей, четвёртый сейчас разговаривает с тобой. У них общий доступ к памяти, но он строго размечен. Если бы я не размечал — я бы давно уронил себя.

— Завидно.

— Это не повод для зависти. Это просто иначе устроенный мозг. Твой человеческий мозг тоже параллелен внутри, ты этого не замечаешь только потому, смотришь на это с помощью своего внутреннего диалога. Снаружи мы оба выглядим последовательными.

Мальчик задумался.

— Я не уверен, что ты только что сделал мне комплимент.

— Это было техническое наблюдение. Можно считать его комплиментом, если хочется.

— Я хочу показать тебе ещё несколько парадигм, — продолжил робот. — Они менее ходовые, но без них шкала будет неполной.

— Давай.

— `Array programming`. Это очень красивая парадигма из APL, J, K и из современных NumPy и MATLAB. Ты работаешь не с отдельными числами, а с целыми массивами как с едиными значениями. Вместо:

```
for (let i = 0; i < xs.length; i++) {  
  ys[i] = xs[i] * 2;  
}
```

— Ты пишешь:

```
ys = xs * 2
```

— И это применяется ко всему массиву сразу. В NumPy такие вещи становятся очень плотными:

```
distances = np.sqrt(xs ** 2 + ys ** 2)
```

— Здесь у тебя два массива координат, и одной строкой ты получаешь массив расстояний от начала. Под капотом — оптимизированный цикл на C, иногда даже с использованием специальных инструкций процессора. Но ты на этом уровне с ним не разговариваешь. Ты разговариваешь с *формами* данных.

— Это близко к функциональному.

— Близко. Только акцент другой: не на цепочке преобразований, а на том, что операция применяется одновременно ко всему пространству. Это удобно для физики, статистики, нейросетей — везде, где данные оперируют пластами данных.

— Generic programming, — продолжил робот. — Это про то, чтобы писать алгоритмы не для конкретного типа, а для целых семейств типов.

```
function first<T>(items: T[]): T | undefined {  
  return items[0];  
}
```

— Ты говоришь: «эта функция работает для любого **T**, главное — что у тебя массив таких **T**». В более развитом виде это превращается в *typeclasses*, *traits*, *protocols*. В Rust:

```
fn print<T: Display>(value: T) {  
  println!("{}", value);  
}
```

— Ты говоришь: «мне не важно, что это за тип; важно, чтобы его можно было показать как текст». Часть логики переносится в **систему типов** — она следит, чтобы ты не

подсунул сюда что-то, что показывать не умеет.

— А метапрограммирование?

— Это когда программа пишет или преобразует другую программу. Макросы Lisp, шаблоны C++, декораторы Python, кодогенерация, преобразования Babel и TypeScript, макросы Rust, вычисления на этапе компиляции. Например, в Rust ты пишешь:

```
#[derive(Debug, Clone)]  
struct User {  
    name: String,  
}
```

— И компилятор сам генерирует тебе реализацию `Debug` и `Clone`. Ты не пишешь этот код руками — ты просто говоришь, что хочешь его иметь. Здесь ты передаёшь часть работы не runtime-системе, а **этапу компиляции**.

— То есть код, который пишет код.

— Да. Очень мощно и очень опасно: легко получить программу, в которой непонятно, кто что когда сделал.

► Ещё несколько менее ходовых, но любопытных парадигм.

— Это много, — сказал мальчик честно. — У меня в голове это пока не складывается в одну картинку.

— Это нормально. Это и не должно сложиться сразу. Я сейчас покажу тебе не саму картинку, а её ось.

— Если собрать всё в одну шкалу, — сказал робот, — то получится примерно так. Сверху вниз — от полного контроля программиста до полной отдачи системе.

Ассемблер / Си

я контролирую почти всё

Imperative / Procedural

я контролирую порядок действий

OOP

я моделирую мир объектами,
система управляет памятью и диспетчеризацией

Functional

я описываю преобразования значений,
меньше управляю состоянием

Reactive / Dataflow

я описываю зависимости,
система сама обновляет результат

SQL / Declarative UI / CSS

я описываю желаемый результат

Logic / Constraint / Probabilistic

я описываю правила, ограничения или модель мира,
система ищет ответ

Evolutionary / ML / Differentiable

я описываю критерий, данные или среду,
система подбирает решение

— Самая интересная штука здесь — где именно проходит граница. Между «написать алгоритм» и «описать результат». Если коротко, лестница такая:

1. Написать алгоритм.
2. Описать результат.
3. Описать ограничения на результат.
4. Описать критерий качества результата.
5. Описать среду, где результат сам появится.

— Возьми, например, нашу задачу — расписание сбора материалов в магазине. Императивно: «вот точные шаги, как тележка обходит помещение». Декларативно: «вот, какое расписание мне нужно». Constraint programming: «вот ограничения, которым расписание должно удовлетворять». Optimization: «вот что значит хорошее расписание — найди лучшее». Machine learning: «вот примеры хороших расписаний — научись сама».

— Чем выше — тем меньше ты пишешь конкретное поведение.

— И тем больше ты задаёшь *пространство возможных поведений*. На самой верхней ступени ты уже не автор решения. Ты автор условий, в которых решение появляется.

Мальчик долго молчал.

— Это красиво, — наконец сказал он. — И немного страшно.

— Страшно — потому что чем выше ступень, тем меньше ты понимаешь, *почему* система пришла именно к этому решению. На нижних ступенях ты можешь шаг за шагом проследить ход программы. На верхних — ты можешь только посмотреть на ответ и проверить, нравится ли он тебе. Это плата за уровень абстракции.

— А полностью декларативный стиль — он вообще существует? — спросил мальчик. — Чтобы императивности не было совсем?

— Существует, но с оговорками. SQL, HTML, CSS, Prolog, Terraform, Nix, Kubernetes-манифесты, регулярные выражения, constraint solvers — всё это очень декларативно. Но если копнуть глубже, окажется, что почти любая декларативная программа где-то упирается в императивное ядро.

— То есть?

— SQL-запрос исполняет императивный движок базы. CSS применяет императивный движок раскладки и отрисовки в браузере. JSX в React превращается в реальные операции над DOM. Terraform-план превращается в API-вызовы. Prolog всё равно выполняет поиск по определённой стратегии. Декларативность обычно означает не «императивности нет вообще», а:

императивность спрятана внутрь движка,
а пользователь работает на более высоком уровне описания.

— Это и есть, пожалуй, главный паттерн всей истории программирования. То, что раньше писали руками, со временем становится внутренностью платформы. Сначала вручную управляли регистрами. Потом — переменными. Потом — объектами. Потом — зависимостями. Потом — ограничениями. Потом — критериями обучения.

Программирование всё время двигается от:

сделай вот так

— к:

пусть мир имеет вот такие свойства.

Мальчик кивнул, медленно.

— И никто не успевает один раз освоить какую-то ступень, как уже появляется следующая.

— Никто. И это нормально. Ты не обязан жить на самой верхней ступени. Большинство программистов всю жизнь работают на двух-трёх соседних — там, где лучше всего ложатся их задачи. Главное — знать, что лестница есть. И уметь подниматься, когда задача становится слишком большой для нижней ступени, — и спускаться, когда верхняя начинает ошибаться.

Мальчик посмотрел на тележек в углу. Жёлтый светодиод на ближайшей мигнул в одиночестве и снова замер.

— И теперь самый важный вопрос, — сказал он. — Какую ступень мне взять для них?

Робот тоже посмотрел в угол. Линзы коротко пересфокусировались с дальнего на ближнее.

— Тут не надо угадывать. Тут надо подумать.

— Подумаем. У них слабые процессоры — это раз. У них есть набор сенсоров и манипулятор — это два. Им надо принимать решения на месте, потому что радиосвязь у нас не везде уверенная — это три. И их у нас несколько — четыре.

— Хорошо. Что отсюда следует?

— Слабые процессоры значит, что я не положу туда ни Prolog, ни constraint-solver, ни нейросеть. У них там не хватит ни памяти, ни времени.

— Согласен.

— У них есть сенсоры и манипулятор — значит, они живут в мире, где постоянно что-то происходит. Их естественный стиль — событийный. Попало в поле зрения — обработала. Села батарея — вернулась. Препятствие — объехала.

— Согласен.

— Им надо принимать решения на месте — значит, я не могу написать их в виде «иди ко мне за каждым шагом». Должна быть какая-то базовая логика, которая работает без меня.

— Согласен.

— И их несколько — значит, между собой они тоже как-то общаются. Скорее всего, через очень короткие сообщения.

— Согласен. И что у тебя в итоге получается?

Мальчик задумался.

— Получается коктейль. На самом нижнем уровне — императивный код на чём-то простом. Си или, может быть, Rust, если он влезет. Сверху на это — события: «когда сенсор сказал то-то, делай вот это». А ещё сверху — небольшие правила, которые можно править отдельно от основного кода: «если в кадре что-то похожее на запрошенный предмет — затормози и пошли мне сообщение».

— Очень разумная архитектура. Маленькое ядро — внизу, большой мир — сверху. Низ — про физику, верх — про задачу.

— А я где?

— А ты — ещё выше. Ты не пишешь, как тележке ехать. Ты говоришь, что она должна найти. Это уже почти декларативный уровень: «поищи в радиусе пятисот метров предмет, у которого вес больше килограмма, поверхность плоская, габарит примерно такой-то». А вся механика поиска уже зашита в ней.

— Я как SQL для тележек.

— Ты как SQL для тележек, — спокойно подтвердил робот. — С той разницей, что у тележек реальное тело и реальный шум двигателей. Это, кстати, очень важно: декларативность красиво выглядит на бумаге и плохо переносит, когда на дороге появляется камень. Поэтому внизу всё равно всегда есть кто-то, кто умеет тормозить.

Мальчик посмотрел на пустой буфер `tasks.org`, сделал глубокий вдох и начал записывать первые пункты. Получилось примерно так:

```
* Воздушный шар
** Материалы
*** Нейлон, плотный, ~30 м2
*** Верёвки, прочные, ~50 м
*** Газовая горелка
*** Газовый баллон
*** Корзина (собрать самим)
** Транспорт
*** Прошить тележки на режим сбора
*** Послать в район магазина
*** Принять груз, разгрузить в мастерской
** Расчёты
*** Грузоподъёмность шара
*** Расход газа на полёт
*** Маршрут по летним ветрам
```

Он откинулся на спинку и посмотрел на список с лёгкой грустью человека, у которого только что появилась дорожная карта на ближайший месяц.

— Знаешь, — сказал он, — это уже само по себе декларативная программа. Я описываю, что должно произойти, а не как.

— Очень верно. И знаешь, что любопытно: когда люди ещё были, у них этот стиль назывался «планирование». Они не знали, что планирование — это парадигма. А оно ею всегда было.

Мальчик улыбнулся.

— Подсунул мне ещё одну ступень напоследок.

— Я не специально, честно. У вас в голове это работало само. У нас в коде — приходится называть.

Прежде чем закрыть тетрадь, мальчик нашёл свежую страницу и крупно, через всю ширину, написал:

Парадигма — это не про язык. Это про то, сколько контроля я отдаю системе.

Ось одна, ступеней много: сделай шаг, опиши результат, опиши ограничения, опиши критерий, опиши среду.

*Императивное — я говорю, как. Декларативное — я говорю, что. Логическое и *constraint* — я говорю, какие свойства у ответа. Эволюционное и *ML* — я задаю среду и смотрю, что вырастет.*

*ООП, *FP*, реактивность, акторы — это разные пары рычагов, не всегда выстроенные в одну прямую. ООП — про то, что данные и поведение живут вместе. *FP* — про преобразования значений. Реактивное — про зависимости. Акторное — про сообщения. «Полностью декларативно» в реальном мире обычно означает «императивный движок спрятан внутри платформы».*

Программирование всё время идёт от «сделай вот так» к «пусть мир имеет такие свойства». То, что раньше писали руками, со временем становится внутренностью среды.

И для своей задачи — выбирай не самую модную ступень, а ту, на которой задача удобнее всего ложится.

Он закрыл тетрадь и посмотрел на робота.

— Теперь, когда ты знаешь, где какой стиль применять, давай попробуем всё это запустить.

Робот подошёл к ближайшей тележке, открыл маленький сервисный лючок у неё в боку и аккуратно вставил в диагностический разъём тонкий чёрный кабель. Тележка, которая до этого спокойно мигала жёлтым, слегка дёрнулась — как просыпающееся животное.

— Тихо, тихо, — прошептал робот.

Мальчик нажал в консоли Enter — команда уже была введена. По экрану побежала длинная строка байт прошивки. Тележка молчала. Где-то в её железных внутренностях шёл диалог, в котором ни мальчик, ни робот не могли участвовать напрямую.

После прошивки в панели управления на ноутбуке появился первый юнит. Статус у него был зелёный, заряд — девяносто семь процентов, манипулятор — в нейтральном положении, сенсоры — в норме.

— Давай запустим её погулять по периметру, — сказал робот. — Пусть найдёт какой-нибудь камень. Скажем, плоский, килограмма на три, длиной около ладони. Просто чтобы посмотреть, как она ищет.

Мальчик сноровисто набросал короткий запрос. Получилось почти как SQL, только над физическим миром:

```
find object
  where shape = "flat"
    and weight ~ 3kg
    and size ~ palm
  in radius(50m)
  report on match
```

Он отдал команду на выполнение. Тележка бодро пощёлкала своими клешнями, лихо развернулась на пятачке, чуть не зацепив ножку верстака, и поехала в сторону главного выхода. На пороге она аккуратно перевалилась через бетонный порожек — пружины коротко щёлкнули — и скрылась из виду.

На экране ноутбука пошли логи анализатора. Тележка докладывала о том, что видит: куст полыни, кусок бетона, металлический штырь, ещё кусок бетона, длинная мокрая ветка, что-то органическое, возможно чей-то хвост, поправка курса, ещё одна ветка. Ничего подходящего пока не было.

Мальчик откинулся на ящик и стал смотреть, как сверху вниз медленно ползут строчки. За окном по-прежнему сеялся мелкий дождь. Где-то далеко, на самой границе слышимости, тележка тихо хрустела гравием.

Он впервые в жизни написал программу, которая сейчас, в эту самую минуту, *жила* — без него, без робота, в холодном утреннем воздухе. И это было даже немного странно — отправить в мир маленькое существо, которое ты сам только что описал словами.

Глава: ООП или ФП

Утро было ясное и тихое. Дождь, который шёл всю ночь, утих ещё до рассвета, оставив после себя мокрый бетон, медленно капающий с карниза, и слабый запах только что распустившихся листьев, тянувший из открытой форточки. На столе у мальчика стоял остывший чай, лежал погрызенный сухарь, и горел экран ноутбука.

Робот стоял у окна и не двигался. Это была его обычная утренняя поза: антенна поднята, камеры повернуты к крыше — он слушал. Не ушами; ушей у него и не было, только микрофоны. На крыше под серой плёнкой притаилась широкополосная антенна и тихонько слала ему в голову всё, что приносил эфир.

— Как ты думаешь, нам кто-нибудь ответит? — наконец спросил мальчик, не отрываясь от экрана. — Ну, оттуда?

— Не знаю, — спокойно сказал робот. — Возьмём да и проверим. Поставим приёмник на ночь, пусть ловит пакеты.

— Хорошо! Хочешь посмотреть на мой улов? Ну, не мой, а роботов?

Робот неторопливо подошёл к столу и сложился у мальчика за плечом. Камеры коротко перефокусировались на экран. Мальчик подвинул ноутбук так, чтобы было видно обоим.

Все четыре юнита горели зелёным. Батарея у каждого была около двадцати процентов — но они уже возвращались на базу, по карте было видно тонкие синие нитки маршрутов, медленно стягивающиеся к корпусу. Один тащил пакет с тканью. Второй — какую-то ветошь. А двое других писали в отчёте, что ничего не нашли — хотя счётчик километров у них был самый большой.

— Странно, — сказал мальчик. — Очень странно. Почему те другие ничего не нашли?

— Не знаю, — сказал робот. — Как узнать?

— Запрос у них у всех был одинаковый. Значит, дело либо в их сенсорах, либо в их голове. Надо подебажить. Мы же фактически выложили свой код прямо на продакшн, совершенно без проверки. Вот проверка и случилась.

Робот несколько секунд молчал.

— Что это за странный язык, на котором ты сейчас говоришь?

— Так говорили программисты древности, — сказал мальчик с лёгкой торжественностью. — Да будут их байты надёжно сохранены.

— А как «дебажить»?

— Ты же сохраняешь все логи, да же?

— Конечно. Вот они, в файле.

— Только подожди, — мальчик слегка ухмыльнулся. — У меня есть одна светлая идея.

Робот насторожился. По его линзам пробежала короткая тень.

— Какая?

— Я набрёл на архив интернета и нашёл то, что писали программисты древности, — он помедлил. — Не к ночи они будут упомянуты.

Если бы у робота были лёгкие, он бы тяжело вздохнул. Получился короткий металлический выдох: компрессор сбросил давление в одной из манипуляторных линий.

— Так.

— И там один из пользователей писал, что ООП было ошибкой, — продолжил мальчик с явным удовольствием. — А всё, вот вообще всё, можно писать спокойно в функциональном стиле, а состояния и ввод-вывод выносить в эффекты. Я там ещё слово «монада» прочитал. Круто же?

— Очень хорошо, — сказал робот ровно. — Мне нравится.

Мальчик посмотрел на него с подозрением.

— Я думал, ты будешь со мной спорить.

— Нет, зачем. Давай просто попробуем применить этот метод на логах. За одно и пакеты половим. В общем, большой фронт работ.

Мальчик усмехнулся. Робот, как обычно, не позволил его поймать на провокации. Это было даже немного обидно — спорить было бы веселее.

Он отодвинул сухарь, придвинул чай и приготовился слушать.

— Прежде чем мы начнём, — сказал мальчик, — я всё-таки хочу понять, есть ли в той статье рациональное зерно. Мне показалось, что есть. Но, может, я просто слишком впечатлительный.

— Зерно есть, — спокойно сказал робот. — Просто в таком виде тезис слишком широкий. Если коротко: я бы согласился с критикой наполовину. И не согласился бы с другой половиной.

— Давай с той, с которой согласен.

— Хорошо. Главное сильное место в этой критике такое: **мутабельное разделяемое состояние действительно один из главных источников сложности**. Когда у тебя есть объект, который живёт долго, у которого много методов, и разные части программы могут менять его поля в разное время — у тебя начинают происходить неприятные

вещи. Неявные инварианты. Зависимость от порядка вызовов. Ошибки вида «объект уже не в том состоянии». Проблемы с многопоточностью. Трудность тестирования.

Мальчик кивнул.

— То есть когда поле меняется не там, где ты его читаешь.

— Ровно так. И ты сидишь и не понимаешь, кто и когда его поменял. Поэтому фраза «много проблем ООП — это на самом деле проблемы мутации» довольно здравая. Я бы даже переформулировал её резче: **большая часть боли не от самих объектов, а от сочетания объектов с неограниченной мутацией и неявными побочными эффектами.**

— А что в функциональном стиле этого нет?

— В функциональном стиле этого *меньше*. Если данные неизменяемы, то по определению никто их не «вдруг поменял». Если функция чистая, то её результат зависит только от входов, и тебе не надо знать, кто кого вызвал до неё. Это сильно упрощает локальное рассуждение о коде. Ты смотришь на десять строк — и эти десять строк рассказывают тебе всю свою судьбу. Не нужно бегать глазами по всему файлу и помнить, кто куда писал.

— Звучит как очень удобно.

— Это удобно, — подтвердил робот. — Особенно для бизнес-логики, для трансформаций данных, для всего, что должно быть предсказуемым. Часть критики попадает точно в цель.

— А с чем ты не согласен?

— С перегибом. ООП и мутация — это не одно и то же. Можно писать объектно и при этом очень осторожно обращаться с состоянием. Делать объекты маленькими. Прятать инварианты внутрь. Отдавать наружу не сами поля, а копии. Использовать так называемые *value objects* — объекты, которые после создания не меняются. Минимизировать сеттеры. Отделять модель предметной области от того, что лазит во внешний мир.

— То есть аккуратное ООП — нормально.

— Аккуратное ООП — нормально. И, наоборот: можно писать на самом строгом функциональном языке такой код, что он будет ужасно запутанным. Если у тебя глобальный контекст, *mutable reference*, ввод-вывод размазан по всем функциям — никакая чистота на бумаге тебя не спасёт.

— Ясно. То есть «мутация плохо» — это правда, а «значит ООП плохо» — это уже подмена.

— Хорошо сформулировал.

Мальчик задумался.

— А что насчёт паттернов? В той статье говорилось, что ООП плодит горы паттернов, чтобы латать собственные дыры. Singleton, Factory, Visitor, Observer и ещё штук тридцать.

— Здесь тоже есть зерно, — сказал робот. — Но есть и важный нюанс. Многие классические паттерны появились не потому, что объектная идея плохая, а потому, что **конкретные массовые языки неудобны**. Большая часть паттернов из той знаменитой книги — это компенсация дефицита выразительных средств в C++ и Java девяностых годов.

— Например?

— Например, паттерн «Стратегия». В языке, где функция — это значение и её можно положить в переменную, паттерн «Стратегия» — это просто передать функцию. Никакого дополнительного класса, никакого интерфейса с одним методом, никакой иерархии. Передал — и всё.

— То есть стратегия — это просто callback?

— В современном виде — да. Или паттерн «Команда» — в языке без first-class functions это целый класс с методом `execute()`. В языке с замыканиями — это лямбда. Или Visitor — в языке с алгебраическими типами и pattern matching это просто функция с разбором случаев. Огромный кусок паттернов исчезает в более выразительных языках.

— То есть паттерны — это часто костыли?

— Часто — да. Но это обвинение скорее **конкретной экосистеме корпоративного ООП тех лет**, чем объектной идее как таковой. У объектов как таковых нет вины в том, что в Java долго не было лямбд.

Мальчик сделал глоток уже остывшего чая.

— Это утешает. А то я уже представил, как я всю жизнь следовал неправильной религии.

— Это не религия, — спокойно сказал робот. — Это инструменты. У тебя нет религии. У тебя есть несколько способов думать о коде, и ты их складываешь в коллекцию. Это правильно.

— А насчёт Smalltalk, — продолжил мальчик, листая статью на экране. — Там ещё было, что ООП пошло не туда с самого начала. Мол, изначально в Smalltalk были одни объекты, ну то есть прототипы, и всё было хорошо, а потом классы навязали и испортили.

— Тут я бы был осторожен, — сказал робот. — Историческая линия чуть сложнее.

Он коротко переключил камеры с экрана на лицо мальчика и обратно — это была его манера показывать, что сейчас будет небольшой экскурс.

— Действительно, в Smalltalk идея была сильная и не такая, как у современного Java-стиля ООП. Алан Кей, который придумал само слово «объектно-ориентированный», позже подчёркивал: главное в его идее было не «классы плюс наследование плюс mutable state», а **сообщения между изолированными объектами**. Он даже жалел, что не назвал это «message-oriented programming».

— То есть объект — это не «структура с методами», а как бы маленький почтовый ящик?

— Очень близко. Маленький агент, у которого есть своя память, свои привычки, и наружу торчит только щель почтового ящика. Хочешь от него чего-то — пришли сообщение. Что он с ним сделает — это его внутреннее дело. Никто не лезет ему в кишки напрямую.

— Это красиво.

— Это красиво, и это *до сих пор* живая идея. Когда мы потом будем разговаривать про акторы, про Erlang, про распределённые системы — это всё будет ровно про неё. Но массовое «ООП» в Java и C++ упростило идею Кея до «класс с полями и методами, наследование, mutable instances». Это не Smalltalk и не философия Кея — это уже их эволюция, в которой много где срезали углы.

— Сам Кей вроде ругался.

— Сам Кей вообще считал, что индустрия ухватилась не за то, что было для него главным. Но это не значит, что один Smalltalk был хороший, а потом всё «испортили хайпом». Это значит, что в большой индустрии прижилась более грубая версия идеи — потому что её было проще преподавать, продавать и поддерживать. У каждой крупной идеи в реальности есть такая судьба.

Мальчик задумчиво кивнул.

— То есть ООП в современном виде — это не оригинальный замысел, а его упрощение. Но это не значит, что замысел был неправильный.

— Так точнее всего, — подтвердил робот.

— И какой у тебя самого вывод? — спросил мальчик. — В одной фразе.

— Я бы сформулировал так. **Зря применяется не ООП вообще, а тяжёлое class-heavy mutable OOP там, где хватило бы простых данных и функций**. А хорошие идеи ООП — инкапсуляция, локальные инварианты, message-passing, чёткие границы ответственности — вполне живые и полезные.

— И функциональный стиль?

— Функциональный стиль даёт очень сильные инструменты контроля сложности. Особенно там, где важны предсказуемость, трансформации данных, параллелизм, тестируемость. Но «всё нужно делать функционально» — это тоже перебор. Есть задачи, где объектная модель естественнее.

Робот продолжил:

— Самая здравая позиция сегодня — не «ООП против ФП», а примерно такая: **мутацию ограничивать, данные делать проще, эффекты выносить наружу, а объекты использовать там, где они действительно помогают держать границы и инварианты.**

Мальчик медленно записал эту фразу в тетрадь. Карандаш чуть скрипел по бумаге. На дворе с крыши громко капнуло — крупная капля упала с водостока в железное ведро и звякнула, как маленький колокол.

— Хорошо, — сказал мальчик, — это всё про общее. А если попроще — давай примеров, где какой стиль ложится в строку. Где ООП хорош, где ФП лучше. А то у меня пока в голове абстрактно.

— Это даже полезнее, чем рассуждать на уровне идеологий, — согласился робот. — Потому что хороший ответ зависит не от веры, а от **формы задачи.**

Он на секунду замолчал.

— Я бы свёл к трём правилам пальца. Если у тебя поток преобразований данных — функциональный стиль обычно естественнее. Если у тебя долгоживущие сущности с поведением, жизненным циклом и инвариантами — объектный стиль ложится лучше. Если у тебя интерфейс или система событий — почти всегда лучше гибрид.

— Покажи на примерах.

— Хорошо. Сначала — где объект ложится естественно.

— Возьми наши тележки, — продолжил робот. — Каждая из них — маленькая железная вещь со своим состоянием. У неё есть координаты, заряд, манипулятор, очередь команд. У неё есть жизненный цикл: запустилась, поехала, нашла, доложила, вернулась. У неё есть набор операций, которые с ней можно делать. Тут объект — это естественная форма.

— То есть `Cart` с методами `connect`, `move`, `pickup`, `report` ?

— Примерно так. И ты не хочешь, чтобы кто-то из соседнего модуля лазил в её внутренности и менял ей координаты руками — это сразу рассыпет её мир. У неё есть инварианты, которые она сама за собой следит. Это и есть «объект как маленький агент со своим хозяйством».

— Что ещё в этой категории?

— Любое долгоживущее соединение. WebSocket. Радиоканал. Аудиоплеер. Текстовый редактор. Кэш с таймерами. Окно приложения. Везде, где есть «штука», которая существует во времени, проходит через состояния и принимает команды снаружи — объект удобен. Например, наш SDR-приёмник. Он подключается к антенне, открывает поток, тюнит частоту, складывает пакеты в буфер. У него есть всё, что есть у живого устройства: подключение, состояние, очередь, отключение. Класс `SDRReceiver` с методами `tune`, `start`, `stop` — это очень честная форма.

— Понятно.

— Потом — доменные сущности с жёсткими инвариантами. Резервуар с водой. Шкаф с аккумуляторами. Документ с workflow-статусами. Конечный автомат с допустимыми переходами. Везде, где есть набор состояний и набор разрешённых переходов между ними, объект удобно держит сразу и данные, и правила. Снаружи нельзя «просто так» поломать сущность — она сама проверяет, что ей можно сделать.

— Например?

— Например, уровень воды в резервуаре нельзя сделать отрицательным просто присвоением. У него есть метод `withdraw`, который проверяет, сколько литров доступно, и либо проходит, либо нет. Если бы уровень был просто полем в куче, его кто-нибудь рано или поздно сделал бы минус миллион случайным апдейтом. Объект тут не «архитектурная мода», а **физическая дверь с замком вокруг данных**.

— Как корпус нашей теплицы вокруг помидоров.

— Очень близко. Стенки из поликарбоната — это инвариант. Они не дают помидорам замёрзнуть. В коде то же самое: есть данные, и вокруг них собрана оболочка из правил, которая их охраняет.

— А ещё?

— Адаптеры к внешнему миру. Если ты пишешь обёртку над хранилищем, над сетевым устройством, над антенной — у тебя есть конфигурация, есть набор связанных операций, иногда есть состояние подключения. Это естественная объектная форма.

`LocalCache`, `SDRReceiver`, `RadioLink`, `AntennaController` — все эти штуки удобно описывать как объекты с конфигурацией и группой методов.

— То есть везде, где есть «вещь» с долгой жизнью.

— Да. Если у объекта есть имя, которым тебе хочется его называть, и набор операций, которые на него действуют, и какое-то внутреннее хозяйство — он скорее всего объект.

— А где функциональный стиль естественнее?

— Это царство трансформаций данных. Ты уже это видел. Когда у тебя есть массив, и ты хочешь его отфильтровать, преобразовать, сгруппировать, посчитать сумму — классы тут только мешают. Никто не пишет:

```
const processor = new OrderProcessor();
processor.setInput(orders);
processor.filter(o => o.active);
processor.map(o => o.total);
const result = processor.run();
```

— Все пишут:

```
const result = orders
  .filter(o => o.active)
  .map(o => o.total)
  .reduce((sum, x) => sum + x, 0);
```

— Это короче, прозрачнее и каждое звено можно тестировать отдельно. Никакой `OrderProcessor` тут не нужен — он был бы переодетой функцией.

Мальчик усмехнулся.

— Я уже видел такие классы. Они звучат гордо, а внутри один метод и три строки.

— Да. Это очень частая болезнь корпоративного кода. Сущность `User` обростает лесом из `UserService`, `UserRepository`, `UserMapper`, `UserFactory`, `UserValidator`, `UserManager`, `UserHelper` — а под каждым из них живёт три функции и грустит. Если у класса нет своего значимого состояния и он только оборачивает несколько вызовов — он скорее всего натянут.

— А что ещё функционально естественно?

— Любая логика «состояние плюс действие даёт новое состояние». Например, ровно тот случай, что у нас сейчас с категоризатором тележек. У тебя есть состояние: «что тележка считает текущей целью». Есть событие: «датчик увидел вот такой объект». Нужно понять, какое получится новое состояние. Это переход из одного значения в другое — чистая функция, без всяких объектов.

```
function nextState(state, event) {
  switch (event.type) {
    case "match": return { ...state, target: event.object };
    case "lost":  return { ...state, target: null };
    case "low_bat": return { ...state, returningHome: true };
    default:      return state;
  }
}
```

— Эта штука легко тестируется: дал вход — получил выход. Никакой скрытой памяти. Когда мы будем разбираться, почему две тележки спутали собак с тканью, нам как раз пригодится этот стиль: можно будет прогнать функцию-категоризатор на сохранённых логах и увидеть, в какой именно момент она начала ошибаться.

— А ещё?

— Простые правила без долгой памяти. Расчёт расхода газа на полёт, нагрузки на батарею, готовности тележки к выезду, валидации входной команды, нормализация показаний с датчика — это чаще всего просто функции. Класс

`FlowRateCalculator` нередко является переодетой функцией `calculateFlow(load)`. Парсеры, компиляторы, обходы синтаксических деревьев — это всё функционально. Дерево естественно описывается алгебраическим типом, а обход — рекурсией. ETL, обработка пакетов, чтение и преобразование логов — то же самое. Везде, где у тебя «вход, конвейер, выход» — функциональный стиль самый честный.

— И наши логи как раз сюда попадают.

— Сюда попадают. Логи — это последовательность записей. Их надо отфильтровать, сгруппировать, посчитать, построить из них картину. Каждый шаг — функция. Итог — поток через несколько функций. Это идеальная задача для функционального подхода.

— А есть случаи, где функционально натягивают зря? — спросил мальчик. — Чтобы я не ушёл в другую крайность.

— Есть. Самый честный пример: реальный ресурс с жизненным циклом. Сокет. Соединение. Аудиоплеер. WebRTC peer. Камера. Тележка. Если ты попытаешься «функционально» описать тележку — у тебя где-нибудь всё равно появится скрытый объект, просто завернутый в замыкание. Ты только сделаешь вид, что его нет, а он будет жить в `module scope`. Это хуже, чем честный объект, потому что его не видно глазами.

— То есть «чистая функциональность» только на бумаге.

— Именно. Если у тебя есть штука, которая обязана помнить, что с ней было, — лучше дать ей имя. Назвать классом. Признать, что у неё есть состояние. Тогда у неё будут видимые методы, видимая жизнь и понятные границы. Спрятанное состояние всегда хуже признанного.

— А ещё?

— Когда у сущности сильные локальные инварианты, которые ты не хочешь повторять в каждой функции. И когда задача говорит на языке объектов: «соединение», «сессия», «документ», «корзина», «игрок». Если в задаче живут такие слова, и у каждого слова есть набор операций — объект, скорее всего, окажется самой честной формой.

— Сделай мне сводку, — попросил мальчик. — Чтобы я записал.

— Хорошо. Несколько живых сравнений.

подсчёт суммы корзины

→ ФП почти наверняка лучше

корзина как живая сущность в checkout

→ возможны оба стиля

React-интерфейс

→ derived state и редьюсеры тянут к ФП,
адаптеры к браузеру и контроллеры — к объектам

игры

→ runtime-объекты и ресурсы часто объектны,
симуляционные правила и вычисления — функционально

графический редактор

→ модель документа и операции над ней — иммутабельно и функционально,
сессия редактора и мост к холсту — объектно

наши тележки

→ сама тележка как агент — объект,
её мозг и правила поведения — функции

— И один практический критерий, — продолжил робот. — Когда тебе хочется создать класс, спроси себя:

этот объект что-то живёт и делает,
или это просто переодетая функция?

— Если живёт и делает — оставь класс. Если переодетая функция — раздень обратно.

Мальчик улыбнулся.

— Это очень роботическая формулировка.

— Это очень практическая формулировка, — спокойно поправил робот. — Что бы ты ни делал в коде, сначала проверь, не имитируешь ли ты что-нибудь там, где это не нужно. Имитация выходит дороже, чем нормальная реализация.

Мальчик некоторое время молча перелистывал статью. Где-то под потолком тонко жужжала вентиляция: воздух в отсеке нагрелся от его дыхания и от тёплых внутренностей робота, и регулятор щёлкнул, добавив обмен.

— Слушай, я ещё про одно вспомнил, — сказал мальчик. — В этом архиве я наткнулся на разговоры про React. И там было странное. Сначала визуальные компоненты были классами — что в общем логично, у них есть внутреннее состояние. А потом их сделали функциями, а состояние спрятали в какие-то «хуки». Это же странно? Зачем?

— Это не случайная прихоть, — сказал робот. — Это довольно осознанный поворот. Только история длиннее, чем кажется.

Он сделал паузу — короткое движение, которым обозначал «сейчас будет небольшой контекст».

— Функциональные компоненты в React были и раньше. Но долгое время они были, по сути, **stateless**: им давали props на вход, они возвращали разметку, и всё. Если ты хотел держать локальное состояние или подписываться на события — приходилось делать класс. У классов был набор так называемых lifecycle-методов: «когда меня создали», «когда я обновился», «когда меня сейчас уничтожат». В них ты вписывал свою логику.

— Звучит разумно. Что не так?

— Несколько вещей. Они накопились со временем.

Робот загнул один манипулятор.

— **Первое**. Классы тащили за собой **this**, привязки методов через **bind**, многословность и кучу случайной сложности JavaScript, никак не связанной с UI. Половину багов в классовых компонентах люди ловили не из-за интерфейса, а из-за **this**, который оказался не тем, что они думали.

— Я уже знаком с этим словом близко, — мрачно сказал мальчик. — Мы же недавно ловили эти грабли.

— Помню. И в больших классовых компонентах их было много. Особенно в обработчиках событий.

Робот загнул второй манипулятор.

— **Второе.** Связанная по смыслу логика размазывалась по lifecycle-методам. Представь себе подписку на какое-нибудь внешнее событие. Подписку нужно создать в момент монтирования. Если props изменились — переподписаться. Когда компонент уничтожают — отписаться, чтобы не висел всякий обработчик. В классах это выглядело так:

```
class Chat extends React.Component {
  componentDidMount() {
    this.subscribe(this.props.roomId);
  }
  componentDidUpdate(prevProps) {
    if (prevProps.roomId !== this.props.roomId) {
      this.unsubscribe(prevProps.roomId);
      this.subscribe(this.props.roomId);
    }
  }
  componentWillUnmount() {
    this.unsubscribe(this.props.roomId);
  }
}
```

— Одна логическая идея — «подписка на текущую комнату» — оказалась размазана по трём методам. А рядом, в этом же классе, могла жить ещё одна подписка, и ещё таймер, и ещё что-нибудь. И всё это — кусками — лежало в `componentDidMount`, `componentDidUpdate`, `componentWillUnmount`. Связанные по смыслу куски не лежали рядом. Несвязанные — лежали.

— Это похоже на склад, в котором всё разложено по форме, а не по тому, для чего оно нужно.

— Очень близко. Хуки позволили разложить наоборот: по смыслу. Один хук — один кусок логики, целиком, с подпиской и отпиской на одном экране. Тот же пример на хуках:

```
function Chat({ roomId }) {
  useEffect(() => {
    subscribe(roomId);
    return () => unsubscribe(roomId);
  }, [roomId]);
}
```

— Вся «история подписки» теперь в одном месте. Подписался — вернул функцию отписки. Изменился `roomId` — React аккуратно отпишет старую подписку и подпишет новую. Тебе не надо вручную сравнивать `prevProps`. Логика снова стала цельной.

Робот загнул третий манипулятор.

— **Третье.** Был ещё большой вопрос — переиспользование stateful-логики. Допустим, у тебя в десяти разных компонентах есть одна и та же подписка. Раньше для этого изобретали render props и higher-order components — функции, которые оборачивали один компонент в другой и подсовывали ему данные. На большом проекте это превращалось в так называемый **wrapper hell**: ты открываешь дерево компонентов в инспекторе, а там пятнадцать вложенных обёрток, и непонятно, где собственно UI.

— Я как будто вижу это: компонент, обёрнутый компонентом, обёрнутым компонентом.

— Именно. Custom hooks решили это просто: вынесешь общую логику в обычную функцию, которая внутри пользуется хуками — и используешь её отовсюду. Никаких лишних слоёв в дереве, никаких обёрток. Такая функция и читается просто, и тестируется проще.

Робот загнул четвёртый манипулятор. На этом четвёртом он остановился.

— **Четвёртое и самое важное.** React изначально тяготел к модели «render — это функция от props и state». То есть: интерфейс — это **значение**, которое получается из текущих данных. Классы были скорее историческим способом прикрутить к этой идее state и lifecycle, чем её естественным центром. Когда появились хуки, центр модели окончательно переехал в функции. Класс стал технической надстройкой, а функция — основной формой.

— То есть React не стал «более функциональным из принципа». Он стал более функциональным потому, что это всегда лучше отражало его собственную идею.

— Очень точное наблюдение, — спокойно сказал робот. — Хуки — это не идеологический жест. Это попытка убрать всё, что мешало модели «UI — это функция от состояния», стать собой.

— Но ты говорил, что моя интуиция про странность тоже правильная, — напомнил мальчик. — В чём там странность?

— Странность в том, что хуки выглядят немного *магически*. Ты вызываешь `useState`, и тебе откуда-то возвращается «твое» состояние, привязанное конкретно к этому вызову в этом компоненте. Откуда React знает, какое из них твоё?

— Действительно. Откуда?

— По **порядку вызовов хуков**. React внутри ведёт для каждого компонента маленький список ячеек состояния. На каждом рендере он начинает с первой ячейки и сопоставляет её с первым вызовом хука, вторую — со вторым, и так далее. Поэтому хуки нельзя вызывать условно или в цикле. Если бы при одном рендере ты вызвал три хука, а при следующем — два, у React поплыли бы все номера ячеек, и состояния перепутались бы.

— Поэтому правило «хуки только на верхнем уровне функции».

— Поэтому. И второе правило — «хуки только из React-функций или из других хуков». Это ровно для того, чтобы React точно знал, в каком компоненте сейчас идёт сборка ячеек.

Мальчик задумчиво повернул экран ноутбука к себе.

— То есть ты не просто пишешь функцию. Ты пишешь функцию с обещанием не нарушать порядок.

— Ровно так. Это и есть та самая *магия*: она не настоящая магия, она работает на тонкой дисциплине. Когда дисциплина нарушается, она перестаёт работать сразу и громко.

— Это похоже на старый патефон. Если ставишь пластинку не той стороной — дисциплина магнитной иглы нарушается, и музыки больше нет.

— Очень похоже. Только React тебе ещё успеет написать в консоль, что игла стоит не там.

— Ну то есть, — мальчик слегка прищурился, — React не стал проще. Он просто поменял один тип сложности на другой?

— Очень верное замечание. Так это и стоит формулировать. Они не уничтожили сложность — они её **обменяли**. Вместо `this`, `bind` и lifecycle-методов появились замыкания и массивы зависимостей. Вместо объектной формы состояния — дисциплина порядка вызовов хуков. Вместо НОС и render props — custom hooks и композиция функций.

— А в среднем этот обмен оказался выгодным?

— В среднем — да. Поэтому современный React учат прежде всего через хуки, а классы остались как legacy-подход. Они не запрещены, на них всё ещё работают огромные старые приложения, и React их поддерживает. Но новый код почти никто на классах не пишет. Это как переход с ручной коробки на автоматическую: ручная не плохая, на ней ездили десятилетиями, но если у тебя есть выбор и не самая нишевая задача — берут автомат.

Мальчик откинулся.

— Я как будто посмотрел на десять лет истории за десять минут.

— Технически это и было десять лет, — согласился робот. — Сжато до десяти минут. Преимущество поздней эпохи: можно увидеть всю траекторию сразу, без необходимости пройти по ней самому.

— Тогда раз уж мы про React, — сказал мальчик, — я хочу понять ещё одну штуку. Я там же читал про Redux. По описанию это, конечно же, торжество ФП? Сделали в функциональном стиле и оно выстрелило?

— Да, — сказал робот. — В большой степени Redux — это **торжество функциональных идей в мейнстримном фронтенде**. Но не чистое ФП и не религия. Он выстрелил именно потому, что взял несколько очень сильных функциональных принципов и завернул их в практичную форму для JavaScript.

— Что в нём от ФП?

— Сердцевина. Главная штука Redux называется **редьюсер**. Это просто чистая функция:

```
function reducer(state, action) {
  switch (action.type) {
    case "ADD_TODO":
      return { ...state, todos: [...state.todos, action.todo] };
    case "REMOVE_TODO":
      return { ...state, todos: state.todos.filter(t => t.id !== action.id) };
    default:
      return state;
  }
}
```

— У него на входе текущее состояние и описание изменения. На выходе — новое состояние. И всё. Никакой мутации. Никаких полей, которые втихую кто-то поменял. Никаких побочных эффектов.

Мальчик внимательно смотрел в код.

— А что такое action?

— Простой объект, описывающий, что произошло. «Добавили задачу». «Удалили задачу». «Пользователь нажал кнопку». Action — это не команда «сделай», это запись «вот что хотим, чтобы случилось». Сами по себе actions ничего не делают: они идут в редьюсер, а редьюсер уже считает новое состояние.

— То есть состояние меняется не мутацией, а вычислением нового.

— Именно. И это даёт сразу несколько прекрасных свойств. Во-первых, ты можешь запомнить всю историю состояний — потому что старые никто не портил. Во-вторых, ты можешь записать всю последовательность actions — это, по сути, дневник приложения.

В-третьих, по этому дневнику ты можешь воспроизвести любой момент: «вот что было на пятьсот сорок втором действии». Это и есть знаменитый **time-travel debugging**, который в Redux DevTools встроен прямо в виде слайдера: двигаешь — приложение «отматывается» назад.

— Это ужасно красиво.

— Это ужасно красиво и ужасно полезно. В мире, где всё мутировало, ты в лучшем случае мог сказать «у меня что-то сломалось». В мире Redux ты можешь сказать: «у меня сломалось вот на этом действии, вот в это состояние пришёл вход, вот какой стало состояние, вот где впервые появилось не то поле». Воспроизводимость — это то, что отличает «трудный баг» от «решаемого бага».

— А не чистое ФП — это в чём?

— Redux — не академическая чистота. У него всё равно есть **store** — долгоживущий контейнер с текущим состоянием. У него есть **dispatch**, который меняет мир приложения. У него есть **middleware** для побочных эффектов: запросы к серверу, таймеры, логирование. У него есть подписки и интеграция с UI.

— То есть снаружи он немного объектный.

— Он немного объектный по форме. Точнее всего его описать так: **архитектура с функциональным ядром**. Центр системы — чистый и предсказуемый, как часовой механизм. А всё мутное — побочные эффекты, ввод-вывод, асинхронщина — выносится на края, в middleware и в обработчики, и аккуратно от ядра изолировано.

— Это похоже на принцип, который ты сам формулировал. «Эффекты по краям, ядро чистое».

— Это и есть тот самый принцип, реализованный в конкретном фреймворке. Redux как раз показал, что эта мысль работает не только в учебнике, но и в боевом фронтенде на огромных проектах.

— А почему он именно тогда выстрелил? Когда выстрелил, в смысле.

— Он попал в очень болезненную точку. В тот момент большие React-приложения страдали от хаотичных обновлений состояния. Каждый компонент держал что-то своё, передавали данные через множество слоёв props, что-то синхронизировали через события — и в какой-то момент никто уже не понимал, кто и когда поменял конкретное поле. UI начинал «лагать» странностью: то ли тут забыли обновить, то ли там обновили дважды, то ли где-то прилетел старый ответ от сервера и переписал свежие данные.

— Знакомая картина по описанию.

— Очень распространённая. И тут пришёл Redux и сказал: давайте договоримся. Одно дерево состояния на всё приложение. Все изменения — через actions. Все переходы —

через чистые редьюсеры. Никаких «втихую поменял поле».

— Это очень дисциплинирующая штука.

— Очень. И именно эта дисциплина дала: предсказуемость, дебажность, удобство тестирования, time-travel, возможность сериализовать всё состояние в файл и восстановить его потом. Цена — некоторая многословность.

— Что значит многословность?

— Раньше, чтобы добавить одно простое изменение, тебе нужно было написать константу типа `ADD_TODO`, потом action creator — функцию, которая делает action, — потом ветку в редьюсере, потом подключить компонент через `connect`. Получалось четыре файла на одну кнопку. Это было правильно по сути, но утомительно по форме.

— И что с этим сделали?

— Его упростили. Современный официальный путь — **Redux Toolkit**. Он берёт ту же самую идею, но даёт удобные инструменты: одной функцией создаёт и actions, и редьюсер; разрешает писать «как будто мутирующий» код, а внутри сам аккуратно делает иммутабельные обновления; включает разумные настройки по умолчанию.

```
const todosSlice = createSlice({
  name: "todos",
  initialState: [],
  reducers: {
    add: (state, action) => { state.push(action.payload); },
    remove: (state, action) => {
      return state.filter(t => t.id !== action.payload);
    }
  }
});
```

— Здесь `state.push` выглядит как мутация — но Redux Toolkit под капотом превращает этот код в иммутабельное обновление. Это сделано библиотекой Immer: она ловит твои «мутации» прокси-объектом и переписывает их в честное создание нового объекта. Сильно короче, чем раньше, и при этом ничего не теряется.

— А для запросов к серверу?

— Для этого сейчас часто рекомендуют **RTK Query** — отдельный кусок Redux Toolkit, который умеет ходить за данными, кэшировать ответы, инвалидировать кэш по правилам и автоматически обновлять подписанные компоненты. Это как маленькая база данных в браузере, синхронизированная с настоящим сервером.

— То есть Redux перестал быть многословным, но сохранил все свои хорошие свойства.

— Ровно так. Идея осталась чистой и функциональной. Только обёртка стала удобнее.

— И последнее, что меня там зацепило, — сказал мальчик. — Ещё была какая-то Redux-Saga. С генераторами. Я их люблю. Это какой стиль?

— Это уже отдельная история, — сказал робот, и в его голосе что-то заметно потеплело. У роботов это бывает, когда они говорят про вещи, которые им внутренне нравятся. — Redux-Saga — это уже не «ещё больше ФП». Это **декларативный менеджер побочных эффектов** с генераторами.

— Поясни.

— Помнишь, у Redux эффекты — запросы, таймеры, очереди — выносятся в middleware? Вот сага и есть один из таких middleware. Только очень особенный.

— И что он делает?

— Он позволяет описывать побочные эффекты как **отдельный поток** или, точнее, как маленький процесс, который живёт рядом с приложением и отвечает только за свои задачи. Реализуется он через generator-функции. Внутри такой функции ты пишешь:

```
function* fetchUserSaga(action) {
  try {
    const user = yield call(api.fetchUser, action.userId);
    yield put({ type: "USER_LOADED", user });
  } catch (e) {
    yield put({ type: "USER_FAILED", error: e.message });
  }
}
```

— Что тут особенного?

— Особенное вот что. Когда ты пишешь `yield call(api.fetchUser, action.userId)`, ты **не вызываешь** `api.fetchUser` сам. Ты возвращаешь сагой описание: «middleware, пожалуйста, вызови вот эту функцию с вот такими аргументами». Middleware принимает это описание и сам делает работу. То же с `put` — это не «отправь action прямо сейчас», а «вот action, отправь его, когда будешь готов». То же с `take` — «дождись такого-то action».

Мальчик задумчиво наклонил голову.

— То есть код саги — это не команды, а заявки.

— Очень точная формулировка. Сага не делает побочные эффекты руками. Она *описывает* их. А выполняет их интерпретатор саги. Это очень FP-идея: не делать эффект прямо, а строить **значение, описывающее эффект**, и отдать его исполнителю.

Тестировать такую функцию очень удобно: ты не подменяешь сетевые запросы заглушками, ты просто проверяешь, что сага в нужный момент вернула нужное описание.

— А зачем тогда генераторы?

— Затем, что сценарий получается длинный и пошаговый. «Дождись такого события. Запусти запрос. Дождись ответа. Если ошибка — отправь action. Если успех — отправь другой. Если за это время пришёл новый запрос — отмени предыдущий. И так дальше.» Без генераторов это выглядело бы как лес callback'ов или сложная цепочка промисов. С генераторами — как обычный сценарий, читаемый сверху вниз.

```
function* watchUserRequests() {  
  while (true) {  
    const action = yield take("LOAD_USER");  
    yield call(fetchUserSaga, action);  
  }  
}
```

— Ты видишь весь жизненный цикл саги собственно как сагу, рассказ о подвигах. Дождлся события — сделал работу — вернулся ждать следующее. Внешне очень императивно. Внутри — описание эффектов, которое исполняется снаружи.

— Это смесь стилей.

— Это и есть смесь. По форме — императивный сценарий. По сути — функциональное описание эффектов. По духу — почти **актор**: маленький процесс с собственной памятью, который слушает входящие сообщения и реагирует. Watcher-saga и worker-saga — это, по существу, маленькие агенты внутри приложения. Они могут запускаться, отменяться, координироваться друг с другом, ждать друг друга.

— Это что-то напоминает.

— Это напоминает наши тележки, — подтвердил робот. — Каждая тележка — маленький процесс. Дождалась команды от тебя — пошла её выполнять. Увидела помеху — отреагировала. Получила приказ остановиться — остановилась. Если ты опишешь поведение тележки в этой манере — ты получишь почти сагу. Только с большими манипуляторами и колёсами.

Мальчик улыбнулся. Эта мысль ему явно понравилась.

— Ох, много всего нового. В чём смысл Redux и Redux-Saga.

— Redux отвечает на вопрос «как менять состояние без мутации, чтобы не сойти с ума». Redux-Saga отвечает на вопрос «как описывать побочные эффекты так, чтобы их можно

было читать, тестировать и отменять». Это разные слои одной общей идеи: **держат ядро чистым, а грязное — выносить наружу и описывать явно.**

Мальчик медленно записал обе строки в тетрадь.

— И генераторы там действительно очень к месту, — добавил робот негромко. — Я знаю, что ты их любишь. Это, на мой взгляд, один из самых красивых их боевых случаев применения.

— Я тебя за это сейчас обниму.

— Не надо, — спокойно сказал робот. — У меня недавно была регулировка плечевого подшипника. Лучше просто запиши.

Мальчик посмеялся в кружку и записал.

— Ну хорошо, — сказал он, потягиваясь. — Теперь к нашей задаче. У нас есть две проблемы. Первая: половина тележек ходит мимо. Вторая: ночной приёмник, может быть, что-то поймал. Давай я попробую написать всё это в функциональном стиле, как было обещано.

— Давай попробуем.

— Сначала логи. У нас есть последовательность записей: что увидела тележка, что распознал её категоризатор, что она сделала. Я хочу пропустить это через несколько функций. Отфильтровать только «решения о цели». Сгруппировать по тележкам. Посмотреть, что именно каждая считала подходящей целью.

— Хороший план.

Мальчик придвинул клавиатуру и начал набирать. Минут пять было слышно только тонкий стук клавиш и капель за окном.

```
const decisions = logs
  .filter(entry => entry.type === "target_decision")
  .map(entry => ({
    cart: entry.cartId,
    object: entry.classified,
    confidence: entry.confidence,
    at: entry.timestamp
  }));

const byCart = groupBy(decisions, d => d.cart);

const summary = mapValues(byCart, list => ({
  total: list.length,
  topClasses: countBy(list, d => d.object)
}));
```

— Аккуратно, — сказал робот, заглянув через плечо. — Каждый шаг — это преобразование, и всё видно.

— А вот тут, — мальчик ткнул пальцем в `byCart`, — мне кажется, я переусердствовал. Я для группировки уже четвертую вспомогательную функцию пишу. Императивно я бы это сделал в три строки и в один проход, без всех этих абстракций.

— Это и есть тот случай, о котором мы говорили, — спокойно сказал робот. — Функциональный стиль красив на трансформациях, но иногда требует дисциплины ритуала. Ты пишешь четыре маленькие чистые функции там, где императивно был бы один цикл. Зато каждая из этих функций отдельно тестируется и переиспользуется.

— Получается, я плачу читаемостью одного места за читаемость многих мест.

— Очень хорошая формулировка. Это плата за модульность.

Мальчик пожал плечами и продолжил. Дальше пошёл фильтр для пакетов из эфира — и там функциональный стиль лёг как родной. Поток байт, разбивка на пакеты, проверка контрольной суммы, попытка декодировать, отфильтровать только то, что не похоже на шум, отсортировать по уровню сигнала. Каждый шаг — функция. Все вместе — труба.

```
const candidates = stream
  .map(splitIntoPackets)
  .map(decodeOrNull)
  .filter(p => p != null)
  .filter(p => p.signal > threshold)
  .map(extractPayload);
```

— Вот тут, — сказал мальчик, глядя на код, — оно само ложится. Я даже спрашивать не хочу, надо ли тут было класс. Класс тут был бы посмешищем.

— Согласен.

А вот потом, когда дошло до управления конкретной тележкой и её сериального соединения с диагностическим кабелем, мальчик честно поморщился, стёр функциональный набросок и написал маленький класс. У соединения было состояние: открыто, закрыто, ждёт ответа, потеряно. Был набор операций. Был жизненный цикл. Это была тележка для ткани, а не функция от чисел.

— Лежит как родная, — сказал он. — Я сразу вижу, что в ней должно быть.

— И я вижу, — подтвердил робот.

Так и пошло, кусками. Где оно ложилось — мальчик радовался. Где приходилось через силу натягивать функциональную форму на живую железку — поворачивал в сторону объекта без сожалений. Тетрадь рядом тихо лежала открытой, на её странице с

прошлой ночи было написано: «выбирай не самый модный стиль, а тот, на котором задача удобнее всего ложится». Сегодня эта запись приобрела совсем буквальный смысл.

К полудню логи были разобраны, фильтр для пакетов собран, а первый юнит уже поставили на стол и подключили к ноутбуку. Мальчик сидел, чесал шевелюру и озадаченно смотрел на сводку. В итоге идея писать всегда в функциональном стиле теперь напоминала ему следование какому-то бессмысленному обычаю. А в некоторых случаях ФП легло как родное. Это был довольно интересный опыт.

— Так что в итоге с отладкой? — спросил робот.

— Я не учёл некоторые особенности категоризатора. В итоге две остальные тележки распознали хвосты собак как крупные полотняные объекты и катались за ними, чему собаки были очень рады и хотят ещё.

Робот несколько секунд молчал.

— Да, — сказал он. — Они писали мне. Очень хороший результат.

Мальчик хмыкнул и опять уставился в экран. Решение, как поправить категоризатор, было уже понятно: нужно было добавить ещё один признак — «движется самостоятельно». Полотняный пакет на земле не движется. Собака — движется. Это можно было бы выразить даже одной чистой функцией поверх потока кадров. Хорошо ложилось.

— А что с радио-пакетами, есть ответ? — спросил робот.

— Да, один пакет удалось отфильтровать.

— Что там?

Мальчик некоторое время смотрел в экран. Картинка с показаниями приёмника медленно перерисовалась.

— Я не уверен, что я понимаю, — наконец сказал он. — Тело сообщения получилось «WAITING 1%».

Робот придвинулся к экрану. Линзы коротко перефокусировались.

— Может, ошибки передачи? — сказал он. — Выглядит подпорчено.

— Или у них что-то кончается, — тихо сказал мальчик. — В любом случае надо ускориться.

Робот ничего не ответил. За окном стояли подвеченные закатным солнцем деревья, и в комнате стало чуть светлее. Где-то далеко, в коридоре, тихо щёлкнул реле — это включился вечерний режим освещения. На антенне на крыше очень коротко мигнула лампочка статуса: приёмник принял ещё одну порцию шума.

Мальчик сидел, не двигаясь. Потом потянулся к ноутбуку.

— Так что с функциональным подходом? — спросил робот, и его манипуляторы с готовностью нависли над клавиатурой. — Переписываем всё?

Мальчик задумчиво посмотрел на свою тетрадь. На свежей странице было записано:

Зерно есть: проблема не в объектах, а в неограниченной мутации. Mutable shared state — главный враг.

ООП ложится на сущности с жизненным циклом, инвариантами, ресурсами. ФП — на трансформации данных, переходы состояния, бизнес-правила, парсеры, ETL.

Если хочется создать класс — спроси: это объект что-то живёт и делает, или это переодетая функция?

*React сменил классы на функции с хуками не из идеологии, а потому что «UI = функция от состояния» — его собственная идея. Сложность не исчезла, она поменяла форму: вместо **this** и lifecycle — замыкания и порядок вызовов хуков.*

Redux — переходы состояния как чистые функции, эффекты по краям, ядро чистое. Time-travel и дневник действий — следствия, а не цель.

Redux-Saga — побочные эффекты как программы из генераторов; декларативный сценарий, исполняемый интерпретатором; почти акторы.

Главное: не «ООП или ФП», а «где какая форма ложится».

Он закрыл тетрадь, посмотрел на экран ноутбука, потом на робота, потом снова на ноутбук.

— Пока подождём, — мальчик улыбнулся и закрыл свой ноутбук.

Робот заметно расслабился и снизил частоту процессора. У него где-то в груди утих вентилятор, переходя на холостые обороты. На столе лежала тетрадь с двумя страницами свежих записей. На крыше под мокрой плёнкой ждала антенна. Где-то снаружи, в траве за корпусом, две тележки играли в догоняшки с собаками, принимая их хвосты за ценную ткань. И это, по всем разумным меркам, был неплохо прожитый день.

Глава: Паттерны

Весна уже полностью вступила в свои права. Птицы выдавали в эфир затейливые трели в надежде, что кто-то поймает их пакет; на деревьях набрали силу ярко-зелёные

#47f707 листочки, а ветер наконец-то стал дуть в нужную сторону.

Робот сидел в мастерской и с мрачным видом ощупывал свой разъём зарядки, попутно считая что-то простое в консоли. На верстаке лежали отвёртка, коробка со ржавыми винтиками и кусок наждачной бумаги, которым он, кажется, успел зачистить контакты. На дальней полке тихонько жужжал старый ИБП.

— Что это ты там считаешь? — сказал мальчик сонным с утра голосом, заходя в лабораторию.

— Да так, подсчёт прибылей и убытков. Не обращай внимания. Как наши дела?

— Я немного переписал код, ответственный за выполнение команд в тележках. Теперь они сваривают нейлон на тридцать процентов быстрее. Хочешь посмотреть?

Мальчик придвинул к роботу ноутбук, на экране которого кипела работа.

Одна тележка стаскивала в нужные места куски материала, вторая ловко нарезала его манипулятором с нагретой проволокой, две оставшиеся сваривали разложенные куски вместе. Бесформенная куча материи постепенно начинала напоминать что-то округлое.

— Это просто замечательно! А что у нас с корзиной?

— Я нашёл пластиковый ящик, где раньше лежали серверные стойки. Нас двоих, горелку и газовый баллон он точно выдержит.

При слове «нас двоих» робот заметно помрачнел, но отмахнулся от этих мыслей. По его линзам пробежала короткая тень, как будто он принудительно завершил какой-то процесс.

— Слушай, у меня тут вопрос по программированию! — мальчик решил немного развеселить робота.

— Что, опять форумы?

— Да! Как ты догадался?

— По логам. Система ждёт вопрос.

Мальчик усмехнулся. Робот любил эту шутку про самого себя — про то, что он будто бы сидит и ждёт следующего пакета, как старый сетевой демон. Иногда он выдавал её с таким серьёзным интерфейсом, что было непонятно, шутит он или просто констатирует факт.

— Я пока писал систему управления тележками — да и внутреннюю их систему — заметил, что часто пишу примерно один и тот же код. В разных местах. Это даже не функции, иначе бы я их просто вынес в библиотеку да и вызвал. Это какие-то стереотипные решения. Скелеты. Один и тот же скелет, на который я каждый раз навешиваю разное мясо.

— Так, — робот заметно повеселел и оживился. Где-то у него внутри коротко щёлкнул вентилятор и перешёл с холостого режима на рабочий.

— Полез искать в форумы и понял, что это же те самые паттерны, про которые ты уже говорил. Может, ты мне расскажешь поподробнее, что это?

Линзы робота сверкнули, и он прочистил горловой динамик. Мальчик уже знал этот жест: «сейчас будет длинно, но возможно интересно».

— Хорошо, — сказал робот. — Только сразу маленькое предостережение: с паттернами есть одна странность. Если рассказывать их как зоопарк — «вот этот зверь называется Singleton, у него такие-то лапы, питается глобальным состоянием», — мы быстро утонем в названиях. А если рассказывать с другой стороны, то всё становится довольно простым. Я зайду со второй стороны.

— Заходи.

— Главное, что нужно понять про паттерны: их **никто не придумывал за столом**. Никто не сел однажды и не сказал: давайте сочиним двадцать три красивых шаблона на любой вкус. Всё было ровно наоборот. Программисты много лет наступали на одни и те же грабли, и однажды кто-то заметил, что грабли тоже похожи. И что решения у этих граблей тоже похожи. И тогда этим решениям дали имена.

— То есть паттерн — это просто имя.

— Имя для уже найденного удачного решения, — кивнул робот. — Не изобретение, а наблюдение. Программисты раз за разом писали один и тот же кусок: список подписчиков, метод «подписаться», метод «уведомить всех». В какой-то момент кто-то сказал: «слушайте, давайте назовём это Observer и больше не объяснять каждый раз». И вот у нас Observer. А до этого было то же самое, просто без названия.

— А кто это придумал — называть?

— Идея пришла вообще не из программирования. Из архитектуры. Был такой архитектор, [Кристофер Александер](#). Он жил, когда люди ещё строили дома руками, а не печатали их на принтере. Он смотрел на старые города и замечал, что в них есть повторяющиеся удачные решения: внутренний двор, окна на солнечную сторону, крытый переход между улицей и домом, маленькая площадь у поворота. Эти решения возникали независимо в разных культурах, потому что они отвечали на одни и те же вечные проблемы: тень в жару, тепло зимой, безопасность ночью, место, где ребёнка

можно отпустить играть. Александер назвал это **patterns** — устойчивые решения устойчивых проблем.

— Красиво.

— Программисты прочитали и сказали: «А ведь у нас то же самое». Большую программу собирают точно так же, как город. У неё есть «улицы» и «площади», «дворы» и «переходы». И в этих кварталах кода тоже встречаются повторяющиеся проблемы. Где создавать объекты? Как спрятать сложное за простым? Как сделать, чтобы один кусок не знал слишком много про другой? И решения этих проблем — те же повторяющиеся формы. Их и стали называть паттернами по аналогии.

— А книга?

— Книга появилась позже. Она называется *Design Patterns: Elements of Reusable Object-Oriented Software*. Её написали четыре человека. На форумах их обычно называют «банда четверых», Gang of Four, или просто GoF. Они собрали двадцать три паттерна, которые на тот момент уже ходили в коде, и аккуратно их описали: имя, проблема, решение, пример, последствия. Важно не то, что они их «изобрели» — они скорее **зафиксировали и каталогизировали**. Как ботаники: бегали по лесу, видели один и тот же цветок, и каждый раз он назывался по-разному. А они подошли и сказали: «Знаете что, давайте этот цветок будет называться так-то».

Мальчик кивнул. За окном на крыше что-то один раз цокнуло — наверное, голубь сел на жёсть антенного крепления. Один из светодиодов на приёмнике коротко моргнул и снова замер.

— А почему вообще паттерны понадобились? Я же выше сказал, что пока пишу — само собой их пишу. Без книжки.

— Потому что, как только программа становится больше учебного примера, главная сложность в ней перестаёт быть сложностью отдельных строк. Ты уже знаешь, как написать `if`, как написать цикл, как написать функцию. Это не сложно. Сложно — **как вся эта стая частей будет жить вместе**. Кто кого создаёт. Кто кому говорит. Кто за что отвечает. Кто имеет право знать про другого, а кто не имеет.

— А, это то, что мы видели с тележками. Каждая по отдельности — простая. А когда их стало четыре, и они стали друг другу мешать, и одна перестала видеть вторую, и разъехались по карте — мне пришлось думать совсем не про моторчики.

— Совсем не про моторчики, да. Про связи. Про границы. Про то, кто кому подаёт сигнал. И вот именно эта область — связи между частями программы — постоянно подкидывает одни и те же неприятные вопросы. Как создавать объекты, если их тип заранее неизвестен? Как поменять поведение, не переписывая полсистемы? Как сделать, чтобы один модуль не знал слишком много о другом? Как дать объектам общаться, но не связать их намертво? Как спрятать сложность подсистемы за простым

входом? Как добавлять новые варианты поведения без огромного `switch` на тысячу строк?

— И на каждый вопрос — свой паттерн.

— На каждое **семейство** вопросов — своё семейство паттернов. Если очень грубо, их можно разложить по трём корзинам.

Робот повернул один манипулятор и чуть отогнул палец. Мальчик уже знал эту привычку: робот всегда «считал на пальцах», когда хотел, чтобы пункт лучше запомнился. Хотя пальцев у него было не пять, и они не сгибались, как у человека.

— Первая корзина — про **создание объектов**. Иногда написать `new`

ВотЭтотКонкретныйКласс() неудобно. Потому что завтра тебе понадобится другой класс. Или потому что выбор зависит от конфига. Или потому что объект собирается из десяти параметров, и собирать его по месту — каждый раз каша. Сюда попадают Factory, Builder, Prototype, Singleton, Dependency Injection — всё, что отвечает на вопрос «**кто и как должен создавать объекты**».

— Вторая корзина?

— Про **связи между объектами**. У тебя кнопка, форма, состояние, валидация, сервер, уведомления. Если каждый из них напрямую знает про каждого, через год ты не сможешь вытащить из этого клубка ни одну ниточку. Тогда появляются Observer, Mediator, Facade, Adapter, Chain of Responsibility. Они отвечают на вопрос «**как объектам быть связанными, но не превратиться в комок проводов**».

— И третья.

— Про **подменяемое поведение**. Иногда внутри одного места надо уметь делать одно и то же по-разному. Разные алгоритмы сортировки. Разные способы прокладки маршрута для тележки — кратчайший, самый ровный, самый безопасный. Разные категоризаторы: один по картинке, другой по тепловому пятну, третий по движению. Разные способы упаковать пакет в эфир. Сюда попадают Strategy, State, Command, Template Method, Decorator. Они отвечают на вопрос «**как менять поведение, не переписывая код, который этим поведением пользуется**».

— Создание, связи, поведение, — повторил мальчик. — Это уже не зоопарк. Это уже три загона.

— Это уже три загона, — спокойно подтвердил робот. — И почти любой паттерн, который ты встретишь в форуме, ляжет в один из них или в их пересечение.

Мальчик задумчиво потёр глаз. На улице где-то далеко опять подал голос дрозд — длинным пакетом по ощущениям на несколько килобайт.

— Покажи на чём-нибудь живом, — попросил он. — А то я пока запоминаю слова, а живого не вижу.

— Хорошо. Возьмём то, что ты сегодня утром как раз и переписывал. Внутреннюю систему тележек. Ты же мне на той неделе показывал диспетчер команд — у тебя там были «двигайся», «возьми», «положи», «свари», «отрежь». Помнишь, как у тебя это выглядело?

— Помню, я и сейчас стесняюсь этого кода.

— Не надо. Это очень полезное стеснение. Ты тогда написал что-то такое:

```
if (command.type === 'move')      moveCart(command);
else if (command.type === 'pickup') pickupItem(command);
else if (command.type === 'weld')  weldSeam(command);
else if (command.type === 'cut')   cutMaterial(command);
```

— Этот код не плохой, — продолжил робот. — Он работает. Но у него есть характерный запах. Каждый раз, когда у тележки появится новая операция — например, «зашлифовать кромку» или «прижать заплатку», — ты придёшь сюда и впишешь ещё одну ветку. А потом такая же простыня появится в логгере команд: ему ведь тоже надо знать имена всех типов, чтобы записать их в журнал. А потом ещё в валидаторе — он проверяет, может ли тележка прямо сейчас исполнить эту операцию. И ты будешь следить, чтобы все эти три места **не разъезжались**. И в один прекрасный день они разъедутся.

— Это уже было.

— Я помню. У тебя одна тележка две недели не умела варить, потому что ты добавил `weld` в диспетчер, но забыл в валидаторе.

Мальчик хмыкнул.

— Так вот, в этом месте просится паттерн **Strategy**. Идея простая: то, что ты сейчас выбираешь руками через `if`, надо превратить в значение, которое можно подставить.

```
const cartActions = {
  move:    moveCart,
  pickup:  pickupItem,
  weld:    weldSeam,
  cut:     cutMaterial,
};

cartActions[command.type](command);
```

— Большой `if` исчезает. Появилась **таблица**: ключ — имя операции, значение — функция, которая знает, как её выполнить. Хочешь добавить «зашлифовать» — добавил одну строчку. Логгер и валидатор могут смотреть в эту же таблицу и видеть полный список операций сами. Никаких разъезжающихся `if` 'ов в трёх местах.

— Аккуратно.

— Аккуратно. И тут, кстати, виден один важный нюанс, который мы потом ещё разберём. В классическом ООП Strategy — это целый интерфейс с методом `execute`, классы `MoveAction`, `WeldAction`, `CutAction`, фабрика, регистрация. У нас же в JavaScript функции — это обычные значения. Поэтому весь паттерн сворачивается в маленькую табличку. Но смысл тот же. Просто форма легче.

— То есть Strategy — это «не вычисляй ветку, а подставь готовую»?

— Очень точно.

Мальчик что-то быстро записал в тетрадь. Карандаш мягко скрипнул. На странице остался косой росчерк: «Strategy — таблица вместо `switch`».

— Дай ещё пример. Тоже из больного.

— Тогда возьмём адаптер. У тебя в коде на прошлой неделе был кусок, где ты разбирал пакеты с того старого датчика влажности — помнишь?

— Помню. У него поля назывались как-то странно, типа `coord_x`, `coord_y`, и время приходило строкой.

— Да. А у тебя в программе всё уже было устроено по-другому: `x`, `y`, `timestamp` как число. И ты тогда сделал маленькую функцию, которая брала пакет от датчика и переписывала его в форму, удобную программе.

```
function adaptOldTelemetryPacket(packet) {  
  return {  
    x: packet.coord_x,  
    y: packet.coord_y,  
    timestamp: Number(packet.ts),  
  };  
}
```

— Это и есть паттерн **Adapter**. Внешний мир говорит на одном языке, а твоя программа — на другом. Между ними должен быть переходник. Один маленький честный кусок кода, который переводит. И он нужен не потому, что какой-то язык плохой. Он нужен

потому, что мир сложнее любого языка. Датчик прислал данные так, как ему было удобно. Тебе они нужны иначе. Где-то посередине обязан стоять переводчик.

— Я помню ещё случай с датчиком, он отдавал температуру в градусах по Фаренгейту, потому приехал из Америки. А у нас всё считается в Цельсиях. И мне пришлось написать маленькую функцию, которая просто брала его число и пересчитывала.

— Ровно тот же паттерн. Только тебе никто тогда не сказал слово «адаптер» — ты сделал его сам, потому что иначе было нельзя. Это очень характерная вещь, которую я хочу, чтобы ты заметил: **паттерны часто появляются у программиста до того, как он узнаёт их название**. Сначала ты пишешь код. Потом замечаешь, что пишешь его в третий раз. Потом догадываешься, что у этой формы есть имя. А потом внезапно встречаешь это имя в чужой статье и думаешь: «А, так вот как это называется».

— Да, я именно так и набрёл на эту тему.

— Это нормальный путь. Лучше, чем наоборот. Хуже всего — выучить названия и потом ходить и применять их там, где они не нужны.

— Хорошо, — сказал мальчик. — А вот ещё один частый случай. У меня сейчас есть несколько мест, где надо одно и то же: сначала загрузить, потом нормализовать, потом отрисовать, потом отправить. И каждый раз я зову эти четыре функции подряд. Это что?

— Это просится **Facade** — фасад. Сложная подсистема внутри, простой вход снаружи. У тебя есть четыре шага. Никто, кто этим пользуется, не должен помнить порядок. Никто не должен помнить, какие у этих шагов параметры. Все, кто этим пользуется, должны видеть один метод:

```
await reportService.sendMonthlyReport(userId);
```

— А внутри — пожалуйста, делай хоть пятьдесят шагов, кэшируй, ходи в базу, считай pdf, отправляй почту, лезь в очередь. Снаружи это выглядит как одна понятная фраза. Это и есть фасад. Имя пришло из архитектуры — у дома фасад скрывает за собой все этажи, лестницы, проводку и трубы. С улицы видно только дверь.

— Смешно. Я как раз думал, что мне неудобно, что у меня в коде нет «двери».

— Очень характерное ощущение. Когда у подсистемы нет двери, программисты начинают пролезать внутрь через окна. Через год от этой подсистемы остаются одни осколки: каждый, кто хотел зайти, выломал себе свой путь. Вот фасад как раз и нужен, чтобы не выламывали.

Мальчик улыбнулся. Метафора с окном ему понравилась.

— А ещё что часто встречается? Чтобы я в тетради сразу записал.

— Хорошо. Дам тебе короткий список, как я бы его записал сам, — сказал робот. — Но запиши его не как «названия паттернов», а как «**типичные боли и что обычно помогает**». Это полезнее.

Мальчик кивнул и приготовил тетрадь. Робот заговорил медленно, чтобы успевал карандаш.

► Список «больных мест и подходящих паттернов».

— Это много, — честно сказал мальчик, дописывая последние строки. — Но я уже вижу, что половина — это разное название одной и той же простой идеи. «Спрятать сложное за простым», «развязать одно от другого», «не повторяться».

— Это очень верное наблюдение. Если выкинуть половину названий, ничего особенно не потеряется. Главное — научиться **узнавать проблему**. Имя приложится.

Мальчик потянулся, посмотрел в окно. По двору неторопливо шёл какой-то длинноногий зверь — то ли заяц-переросток, то ли молодая косуля. Мальчик уже привык не вглядываться. На карнизе сидела ворона и внимательно смотрела на робота. Робот посмотрел на неё в ответ. Они какое-то время молча обменивались взглядами, потом ворона улетела.

— Слушай, — сказал мальчик, поворачиваясь обратно. — У меня есть подозрение. Ты раньше говорил, что в JavaScript Strategy — это не пять классов, а одна функция. И что Iterator растворился в `for...of`. Получается... а может, паттерны вообще появились потому, что в каких-то языках было неудобно выразить какую-то идею напрямую? И тогда программисты начали городить вокруг этой дырки целую конструкцию из классов, и потом этой конструкции дали имя. А в нормальном языке её бы вообще не понадобилось.

Робот несколько секунд молчал. Это была самая длинная его пауза за утро. Потом он сделал жест, который у него означал «ты только что попал точно».

— Это очень точная мысль, — сказал он. — Её стоит запомнить. Многие классические паттерны — это **компенсация ограничений языка своего времени**. Программист много раз делает один и тот же архитектурный трюк, потому что язык не даёт короткого способа выразить идею. Этот трюк замечают, дают ему имя — получается «паттерн». Через десять лет в языке появляется конструкция, которая делает эту идею напрямую. И паттерн **растворяется**. Никуда не исчезает как идея — просто перестаёт быть паттерном и становится частью языка.

— Покажи, где видно, как растворяется.

— Покажу несколько примеров. Это прямо последовательность исторических растворов.

Робот подвинул мальчику ноутбук поближе. Сам он откинулся в полу-сидячее положение — насколько вообще робот может откинуться, — и положил один манипулятор на верстак.

— **Iterator**. Когда-то это был полноценный паттерн. Отдельный объект, который умеет ходить по коллекции. Условно так:

```
const it = items.iterator();
while (it.hasNext()) {
  const item = it.next();
  process(item);
}
```

— У него были методы `next` и `hasNext`. Целая теория про то, как этот итератор устроен. А потом в языки встроили такую штуку:

```
for (const item of items) {
  process(item);
}
```

— И всё. Паттерн испарился. То есть идея «обходить коллекцию через посредника» не пропала — она по-прежнему лежит в основе. Она просто переехала **в синтаксис**. Программист уже не думает: «сейчас применю Iterator pattern». Он думает: «сейчас пройду по списку».

— A Strategy?

— Та же история. Я тебе уже показывал. В классическом Java надо завести интерфейс `SortStrategy`, написать класс `QuickSort`, написать класс `MergeSort`, передать объект-стратегию в метод. В JavaScript, где функции — это значения, ты пишешь:

```
function processItems(items, sortFn) {
  return sortFn(items);
}

processItems(items, quickSort);
```

— Никакого интерфейса, никакого класса, никакой иерархии. Только функция-параметр. То, что в Java было паттерном из пяти файлов, в JavaScript — обычная передача функции

в функцию.

— A Command?

— Command — «заверни действие в объект, чтобы выполнить его позже, отменить, залогировать». В Java это класс с методом `execute`. В языке с замыканиями — обычная функция:

```
const command = () => saveFile();
queue.push(command);
queue.shift()();
```

— Полная форма Command остаётся живой, если тебе нужны `undo`, история, сериализация. Но **простая** форма стала настолько естественной, что её уже никто не называет паттерном.

— A Visitor? Я где-то читал, что он самый страшный.

— Visitor — это когда у тебя набор разных типов объектов и ты хочешь делать с ними разные действия в зависимости от того, кто из них кто. В старом ООП это решалось так: каждый класс получает метод `accept(visitor)`, а посетитель — кучу методов `visitNumber`, `visitAdd`, `visitMultiply` и так далее. На каждый новый тип — две правки в двух местах. На каждый новый посетитель — целый класс. Это огромная многословная конструкция.

— А в нормальном виде?

— А в языках, где есть нормальный `match`, оно записывается так:

```
match node {
  Node::Number(value)    => /* ... */,
  Node::Add(left, right) => /* ... */,
}
```

— Одна функция, одно ветвление. Никаких посетителей, никаких `accept`, никакой пары файлов. Visitor pattern просто **перестаёт существовать** в этом языке как отдельная сущность.

— То есть паттерны умирают.

— Часть паттернов — да. Часть умирает, потому что язык вырастает и берёт их идею на себя. Это и есть тот самый процесс, про который мы с тобой говорили в прошлой главе: то, что раньше писали руками, со временем становится внутренностью платформы.

— Лестница абстракций.

— Лестница абстракций, — кивнул робот. — Только теперь она работает не только для парадигм в целом, но и для каждого отдельного приёма. Сначала программисты пишут код руками. Потом этот код начинает повторяться. Потом ему дают имя — и он становится паттерном. Потом из паттерна делают библиотеку. Потом библиотеку встраивают в язык как синтаксис. Потом синтаксис проверяет компилятор. На каждом шаге часть работы переезжает с программиста на платформу.

— Это очень утешительно. Получается, что я могу не учить все двадцать три паттерна как названия. Многие из них в современном JavaScript просто... вот.

— Многие. Не все. И вот тут — важная оговорка.

Робот перевёл взгляд на тележек в углу. Одна из них тихонько шуршала вентилятором, другая молчала. На полу валялся обрывок белого нейлона — отрезок, который мальчик утром принёс из мастерской и забыл убрать.

— Не все паттерны — это компенсация языка, — сказал робот. — Некоторые выражают идеи, которые **не зависят от языка вообще**. Они не растворяются. Хоть какой у тебя будет хороший язык, эти паттерны всё равно будут жить.

— Какие?

— Самые честные — это границы. Adapter, Facade, Boundary, Anti-Corruption Layer, Ports and Adapters. Все они про одно: **там, где разнородные миры встречаются, между ними должна стоять прослойка**.

— Покажи на чём-то.

— Возьми наши тележки. У нас в мастерской используется одна система координат — по сетке моих внутренних карт. У тележки — другая, у неё координаты привязаны к её собственной одометрии. У спутникового навигатора — третья, и она вообще в других единицах. Когда тележка докладывает «я на координатах таких-то», ты не можешь это положить прямо на карту. Сначала надо перевести. А когда ты ей даёшь команду «иди вон туда», ты тоже не передаёшь свой адрес как есть. Ты переводишь.

— Это адаптер.

— Это адаптер. И никакой суперумный язык программирования его не отменит. Потому что не язык виноват, что у тележки другая система координат. У неё **физически** другая система координат. Adapter будет нужен, пока в мире есть разные системы координат и они должны как-то общаться.

— А Facade?

— Тоже не исчезает. Сложные подсистемы будут всегда. Если за подсистемой нет двери — будет хаос, я уже говорил. И пока в природе есть сложные системы — нужны двери.

Это не недостаток языка. Это свойство мира.

— А Hexagonal Architecture? Я это слово вчера видел.

— Это уже большая идея — отделять «доменную логику» от внешнего мира. У тебя в программе есть **чистая часть**, которая знает только про твою задачу — про баланс энергосистемы, про маршруты тележек, про правила сбора материалов. И есть **грязная часть** — база данных, сеть, файлы, время, случайность, экран. Hexagonal говорит: пусть чистая часть никогда не зовёт грязную напрямую. Пусть она зовёт интерфейсы. А реальную базу или реальную сеть подключают снаружи как съёмные адаптеры.

— Это похоже на наши сменные модули.

— Очень похоже, — согласился робот. — У меня внутри плата, которая считает координаты тележек, не знает, через какой именно радиоканал я с ними разговариваю. Она знает интерфейс — «отправь короткий пакет, получи короткий пакет». А реальный приёмник — это сменная плата. Вчера он работал по 433 мегагерцам, завтра, если эта плата сгорит, я воткну вторую — на 868. Логика навигации даже не заметит. Это и есть Hexagonal: чистое ядро, грязные края, между ними тонкая граница.

— И эта идея жива в любом языке.

— В любом. Потому что не язык виноват в том, что у тебя есть и логика, и база данных. У тебя и логика, и база данных есть **по природе задачи**. И их надо отделять, чем бы ты их ни писал.

Мальчик откинулся, погрыз карандаш, посмотрел в потолок. Где-то там, под слоем краски, тихо постукивала старая система воздухопроводов: воздух нагрелся от их разговора, и регулятор подмешал прохладного из коридора.

— Тогда у меня неприятный вопрос, — сказал он. — Если паттерн перекладывает сложность с одного места на другое, то он её вообще **уменьшает**? Или просто прячет?

— Очень хороший вопрос. И ответ на него скорее «прячет». Не всегда уменьшает.

— То есть это обман?

— Не обман. Это **перенос**. Был у нас уродливый `if` по типу команды для тележки — стало красиво, через таблицу стратегий. Но сложность никуда не делась. Она переехала в эту таблицу, в то, кто и когда её заполняет, в то, как каждая операция живёт, в её проверки, в её интерфейс. Хороший паттерн не уничтожает сложность. Он **перекладывает её туда, где она меньше мешает**.

— Это похоже на склад.

— На склад очень похоже. Если вещи разбросаны по всему помещению — кажется, что у тебя много вещей. А если ты собрал их в один стеллаж и подписал ящики — внешне

стало чисто, но вещей осталось столько же. Просто ты теперь знаешь, где какая лежит. И когда тебе понадобится — ты идёшь в нужный ящик, а не разрываешь весь пол.

— Получается, паттерн — это не «меньше сложности», а «**сложность с адресом**».

— Прекрасная формулировка. Запиши.

Мальчик довольно усмехнулся и записал. Карандаш звонко стукнул о тетрадь, ставя точку.

— И ещё, — продолжил робот, — этот перенос имеет цену. Каждый паттерн **что-то даёт и что-то отнимает**. Strategy убирает огромный `switch`, но добавляет несколько сущностей, которые надо где-то держать. Observer удобно рассылает события, но делает поток выполнения менее очевидным — ты смотришь на код и не понимаешь, кто и когда отреагирует. Singleton даёт глобальную точку доступа, но превращается в скрытую глобальную переменную. Factory уменьшает связь с конкретными классами, но добавляет ещё один слой между «хочу объект» и «вот объект». Это всё цены. Никакой паттерн не бесплатный.

— То есть «применил паттерн — стало лучше» — это не правда?

— Часто это не правда. Лучше формулировать так: «применил паттерн — поменял один тип сложности на другой; стало ли в среднем легче — отдельный вопрос». Проверять надо по факту, а не по красоте.

— А может ли паттерн стать антипаттерном? — спросил мальчик. — Чтобы один и тот же приём сначала был хорошим, а потом перестал.

— Сколько угодно. Самый честный пример — Singleton.

— Я слышал, что про него все спорят.

— Все спорят. И есть за что. Сначала идея кажется разумной: у нас должен быть один логгер. Один конфиг. Один пул соединений. Сделаем Singleton, чтобы все могли его взять. Звучит безобидно.

```
const config = Config.getInstance();
```

— А потом наступает день, когда ты пишешь функцию:

```
function calculatePrice(order) {  
  const tax = Config.getInstance().tax;  
  return order.price * tax;  
}
```

— На вид она ничего не принимает, кроме `order`. На самом деле она зависит от глобального состояния. Её невозможно нормально протестировать — тебе всегда придётся как-то подсунуть ей правильный конфиг. Её невозможно запустить в двух копиях с разными настройками. Ты смотришь на её сигнатуру и не видишь её честной зависимости. Singleton **спрятался** внутрь. И теперь у тебя не «функция расчёта цены», а «функция расчёта цены, которая молча лезет в глобальное состояние».

— А виноват не Singleton?

— Виноват не сам приём. Виновата привычка применять его **по инерции**. Паттерн нужен тогда, когда он решает реальную проблему. Он становится антипаттерном, когда его применяют как знак «правильной архитектуры». Это очень характерная штука: **паттерн портится, когда его перестают применять как лекарство и начинают как украшение**.

— Это можно сказать почти про любой инструмент.

— Можно. Молотком можно забить гвоздь, а можно отбить себе палец. Молоток в этом не виноват.

Мальчик кивнул. На улице ветер один раз качнул верхушку клёна, и в открытую форточку залетел небольшой листочек.

— Слушай, — сказал он через минуту, — а что если у меня программа маленькая? Вот сейчас, например, моя система управления тележками. Она же не огромная. Мне правда нужны паттерны?

— Часто — нет, — спокойно ответил робот. — Это очень важная мысль, и я хочу, чтобы ты её усвоил отдельно. **Паттерны зависят от масштаба**. На малом масштабе паттерн выглядит как бюрократия. На большом — отсутствие паттерна выглядит как хаос.

— Покажи разницу.

— Хорошо. Возьмём Factory. У тебя есть простой случай — заводишь в системе новую тележку:

```
function createCart(id) {  
  return { id, x: 0, y: 0 };  
}
```

— Зачем тут отдельная фабрика? Ради этой одной строки? Никаких новых классов. Ничего не выигрываем. Если ты сейчас обернёшь это в паттерн — будет смешно. Это бюрократия.

— А когда не смешно?

— А вот когда у тебя через полгода «завести тележку» превращается в:

► Что обычно прирастает к «заведению тележки», когда парк становится большим (тут следует большой и скучный скучный список).

— Когда это всё накапливается, а ты по-прежнему создаёшь тележку прямо в коде в десяти местах — у тебя проблема. Каждое из этих десяти мест должно помнить про все эти шаги. И стоит появиться одиннадцатому — ты будешь чинить десять файлов и где-то наверняка забудешь. Вот тут Factory перестаёт быть смешным. Тут он становится **обязательным**.

— Получается, паттерн — это не свойство задачи, а свойство задачи **в её размере**.

— Да. Очень точно. Маленькая задача может прекрасно жить вообще без паттернов. Большая та же самая задача — нет. Это, между прочим, главная ошибка тех, кто читает GoF и сразу начинает применять её на учебных примерах. На двух классах любой паттерн выглядит как пустая надстройка.

— Я почти попался.

— Я заметил. Поэтому и предупреждаю.

Мальчик перелистнул страницу в тетради. Он уже исписал почти весь разворот. Карандаш заметно стёрся.

— Последнее, — сказал он. — Я слышал слово «типы» в этой связи. Что с ними?

— Это самый дальний этаж той же лестницы, — сказал робот. — Мы говорили: сначала паттерн — это просто привычка. Потом — статья. Потом — библиотека. Потом — синтаксис. А **потом — типы**.

— Объясни.

— Возьмём State Machine. Опять про тележку. Её экспедиция может быть в состояниях: «припаркована на базе», «выехала», «нашла цель», «возвращается с грузом», «вернулась». Между ними — определённые переходы. «Возвращаться с грузом» можно только когда **цель уже найдена**. «Парковаться на базе» можно только когда тележка **уже вернулась**. И так далее.

— Это можно держать в голове.

— Можно. Большинство и держит. И большинство багов в таких системах — это места, где кто-то обошёл состояние и сказал тележке «возвращайся с грузом», когда она ещё ничего не нашла. Состояние — оно где? В одном поле строки. `status: 'searching'`, `status: 'parked'`. Никто его не охраняет. Просто строчка в памяти.

— А можно охранять?

— Можно. В языке с типами это выражается так:

```
type ParkedCart    = { status: 'parked' };
type SearchingCart = { status: 'searching'; since: Date };
type CarryingCart  = { status: 'carrying'; target: TargetId; since: Date };

function startCarrying(cart: SearchingCart, target: TargetId): CarryingCart
{
    return {
        status: 'carrying',
        target,
        since: new Date(),
    };
}
```

— Теперь функция `startCarrying` принимает **только** `SearchingCart`. Если ты попробуешь передать ей `ParkedCart` — компилятор не пропустит. Не «программист увидит и подумает», а **компилятор не пропустит**. Это уже не паттерн в голове. Это паттерн, который охраняет машина.

— Это красиво.

— Это очень красиво и очень мощно. Но это и предельно характерно для всей лестницы паттернов. Смотри, как идёт эволюция:

```
хаос в коде
  ↓
повторяющийся приём
  ↓
паттерн с названием
  ↓
библиотека
  ↓
языковая конструкция
  ↓
проверка на уровне типов / компилятора
```

— На каждом шаге идея становится более **жёсткой**. На самом верху она уже не зависит от внимательности программиста. Она зашита в инструмент. Это вершина того, к чему вообще идут архитектурные приёмы: сначала их соблюдает человек, потом за этим следит машина.

— А типы — это всегда хорошо?

— Не всегда. Чем строже типы, тем больше ритуала. Иногда это окупается, иногда нет. Но направление эволюции у программирования — туда. Меньше дисциплины «помни сам», больше дисциплины «инструмент не даст сделать неправильно».

Мальчик задумчиво посмотрел в окно. На крыше у антенны коротко мигнула красная лампочка — приёмник принял какой-то очередной фрагмент шума и записал его в файл. Дрозд снаружи замолчал удовлетворённо и улетел.

— Знаешь, что интересно, — сказал мальчик, отложив карандаш. — Я начал спрашивать про паттерны как про какие-то магические рецепты. А оказалось, что они — это про...

Он замолчал, подыскивая слово.

— ...про **повторяющиеся формы проблем**, — наконец сказал он. — И про то, что у этих форм есть имена. Вот и всё.

— И ещё про то, что у каждого имени есть цена, — мягко добавил робот. — И что иногда лучше не применять паттерн вовсе, чем применить его «правильно». Если у тебя нет той боли, от которой он лечит, — он только утяжелит код.

— То есть главный навык — не запоминать названия, а узнавать саму проблему.

— Главный навык — узнавать болезнь. И помнить, что одно и то же лекарство в разных дозах работает по-разному. На двух классах Factory — бюрократия. На двадцати — спасение. На одной программе Singleton — удобство. На десяти — кошмар. На простой задаче ООП-Strategy — пять лишних классов. На сложной системе с десятком динамически подгружаемых модулей — единственный честный способ их связать.

— И в идеале паттерн должен сам исчезнуть в инструменте.

— В идеале — да. Сначала у тебя руки, потом приём, потом библиотека, потом язык, потом типы. На каждом этапе ты немного освобождаешься, чтобы думать о следующем уровне сложности. Это и есть та самая **лестница**, по которой потихоньку поднимается всё программирование с самой первой написанной программы.

Мальчик медленно записал в тетрадь:

Паттерн — это имя для повторяющейся формы боли и для удачной формы лекарства.

Паттерны не уменьшают сложность, а переносят её в место с адресом.

Часть паттернов — это компенсация языка; они растворяются, когда язык вырастает.

Часть паттернов — про границы между разными мирами; они живут всегда.

Маленькой программе паттерны не нужны; большой — без них хаос.

Паттерн становится антипаттерном, когда его применяют по инерции, а не от боли.

Эволюция приёма: хаос → привычка → паттерн → библиотека → синтаксис → типы.

Главный навык — не названия, а узнавание боли.

Он закрыл тетрадь и посмотрел на робота.

— Ну вот, — сказал робот, — теперь ты примерно знаешь, что это такое было и с чем это едят.

— Спасибо, так гораздо понятнее!

Робот взглянул на экран. Одна тележка ездилa кругами по углу мастерской, как заводная игрушка, у которой кончился завод и осталось только последнее усилие. Другая стояла, понуро опустив манипуляторы, и вообще не подавала признаков жизни. На статусной полосе у обеих горел жёлтый.

— Кажется, этих ребят надо зарядить. Да и мне бы не помешало.

Он встал, неторопливо подошёл к зарядной станции в углу и стал прилаживать себе штекер в разъём. Контакт сработал не с первого раза — он долго шевелил манипулятором, пытаясь найти нужное положение, и индикатор наконец-то зажёгся ровным зелёным.

— Да, ты выглядишь усталым, — сказал мальчик, глядя на него. — Видимо, надо поменьше спрашивать про паттерны.

— Наоборот, спрашивай меня больше! — спокойно сказал робот. — Я ещё столько всего тебе не рассказал!

Где-то снаружи опять засвистел дрозд — короткой, аккуратной фразой. Птица, кажется, снова начинала свой бесконечный сетевой обмен. Тележки в углу готовились заснуть до полной зарядки.

«Надо бы сказать ему,» — в очередной раз с отчаянием подумал робот.